


**NED University of Engineering & Technology, Karachi**
**Department of Computer and Information Systems Engineering**
**Bachelors in Computer Systems Engineering**

---

**CS-218: Data Structures and Algorithms**
**Complex Engineering Problem (CEP)**
**Fall Semester 2026**

# DESIGN AND IMPLEMENTATION OF A TEXT EDITOR IN C++

**Submitted By:**



| Name | Roll Number |
|---|---|
| Rumaisa Waseem | **CS-25052** |
| Manahil Imran | **CS-25061** |
| Yashfeen Raza | **CS-25055** |



**Batch: SE Batch 2025**

**Submitted To:**
**Dr. Shahab Tahzeeb**

# TABLE OF CONTENTS

**1. INTRODUCTION AND PROBLEM DESCRIPTION ..... 6**
1.1 Introduction ..... 6
1.2 Problem Description ..... 6
1.3 Project Objectives ..... 7
1.4 Scope of the Project ..... 7
1.5 Course Learning Outcomes ..... 8
**2. REQUIREMENTS AND ASSUMPTIONS ..... 9**
2.1 Functional Requirements ..... 9
2.1.1 File Operations ..... 9
2.1.2 Text Display ..... 9
2.1.3 Text Insertion and Deletion ..... 9
2.1.4 Search Operations ..... 10
2.1.5 Replace Operations ..... 10
2.1.6 Undo Operations ..... 10
2.1.7 Error Handling ..... 11
2.2 Non-Functional Requirements ..... 11
2.3 Assumptions ..... 12
2.4 Hardware and Software Requirements ..... 13
**3. SYSTEM AND DESIGN OVERVIEW ..... 14**
3.1 System Overview ..... 14
3.2 Class Design ..... 15
3.2.1 Frontends: main.cpp and EditorWindow ..... 15
3.2.2 TextDocument ..... 16
3.2.3 LineNode and Character Buffer ..... 16
3.2.4 UndoStack and UndoRecord ..... 17
3.2.5 EditOperations ..... 17
3.2.6 TextSearch and TextReplace ..... 18
3.2.7 FileManager ..... 19
3.2.8 InputValidator and DisplayManager ..... 19
3.3 Class Diagram ..... 21
3.4 Program Workflow ..... 23
3.5 Module Interaction ..... 25
**4. DATA STRUCTURES USED ..... 26**
4.1 Overview of Selected Data Structures ..... 26
4.2 Doubly Linked List ..... 26
4.2.1 Structure and Node Representation ..... 26
4.2.2 Main Operations ..... 27
4.2.3 Implementation ..... 27
4.3 Dynamic Character Array ..... 28
4.3.1 Structure and Memory Representation ..... 28
4.3.2 Main Operations ..... 29
4.3.3 Memory Management ..... 29

4.4 Bounded Stack (UndoStack) ..... 30
4.4.1 Stack Representation ..... 30
4.4.2 LIFO Principle and Main Operations ..... 30
4.4.3 Implementation and Undo Mechanism ..... 31
4.5 Additional Algorithm: Knuth-Morris-Pratt (KMP) Searching ..... 32
4.5.1 Failure Table (LPS Table) ..... 32
4.5.2 Searching Process ..... 32
4.5.3 Time Complexity ..... 33
4.6 Data Structure Summary ..... 34
**5. JUSTIFICATION FOR SELECTED DATA STRUCTURES ..... 35**
5.1 Justification for the Linked List ..... 35
5.2 Justification for the Dynamic Array ..... 36
5.3 Justification for the Stack ..... 36
5.4 Justification for Additional Algorithms ..... 37
5.5 Comparison with Alternative Data Structures ..... 38
5.6 Summary of Design Decisions ..... 39
**6. DESCRIPTION OF IMPORTANT ALGORITHMS ..... 40**
6.1 Document Creation ..... 40
6.2 Opening and Saving a File ..... 40
6.3 Text Insertion Algorithm ..... 42
6.4 Text Deletion Algorithm ..... 44
6.5 Line Insertion and Deletion ..... 45
6.6 Search Algorithm ..... 46
6.7 Replace First Occurrence ..... 48
6.8 Replace All Occurrences ..... 49
6.9 Undo Algorithm ..... 50
6.10 Error Handling and Input Validation ..... 52
**7. TIME AND SPACE COMPLEXITY ANALYSIS ..... 54**
7.1 Complexity Analysis Method ..... 54
7.2 Time Complexity of Major Operations ..... 54
7.3 Space Complexity of Major Operations ..... 55
7.4 Complexity Discussion ..... 56
7.5 Performance Considerations ..... 57
Table 7.3: Measured Execution Times of the Backend ..... 59
7.6 Summary ..... 59
**8. TESTING STRATEGY AND TEST CASES ..... 61**
8.1 Testing Objectives ..... 61
8.2 Testing Approach ..... 61
8.3 Test Environment ..... 62
8.4 Functional Test Cases ..... 62
8.5 Boundary and Invalid-Input Testing ..... 65
8.6 Integration Testing ..... 66
8.7 Test Results and Discussion ..... 67
**9. TEST EXECUTION SCREENSHOTS AND OUTPUT EXAMPLES ..... 68**

9.1 Backend Automated Test Execution ..... 68
9.2 Backend Automated Test Execution ..... 69
9.3 Overall Backend Testing Result ..... 70
**9.4 Frontend (Qt GUI) – Essential Feature Demonstration ..... 71**
9.4.1 File Operations ..... 71
9.4.1.1 Create a New Document ..... 71
9.4.1.2 Open an Existing Text File ..... 72
9.4.1.3 Save the Current Document ..... 74
9.4.1.4 Save the Document Under a New Filename ..... 74
9.4.1.5 Exit the Editor ..... 75
9.4.2 Text Display ..... 77
9.4.2.1 Display Document Contents ..... 77
9.4.2.2 Display Document Line by Line ..... 78
9.4.2.3 Display Document Information ..... 78
9.4.3 Text Insertion and Deletion ..... 79
9.4.3.1 Insert Text at a Specified Position ..... 79
9.4.3.2 Delete a Character, Word, or Text Range ..... 80
9.4.3.3 Insert a New Line ..... 84
9.4.3.4 Delete a Line or Range of Lines ..... 85
9.4.4 Search Functionality ..... 87
9.4.4.1 Search for a Word or Text Pattern ..... 87
9.4.4.2 Display Search Results and Match Locations ..... 89
9.4.5 Replace Functionality ..... 90
9.4.5.1 Replace a Specific Occurrence ..... 90
9.4.5.2 Replace All Occurrences ..... 91
9.4.6 Undo Functionality ..... 92
9.4.6.1 Undo Recent Editing Operations ..... 92
9.4.7 Basic Error Handling ..... 94
9.4.7.1 Invalid Search Input ..... 94
9.4.7.2 File-Opening Error ..... 95
9.4.7.3 Invalid Editing Operations ..... 96
9.5 Summary of Frontend Functionality ..... 97
9.6 Chapter Summary ..... 97
**10. LIMITATIONS OF THE IMPLEMENTATION ..... 98**
10.1 Interface Limitations ..... 98
10.2 Document Representation Limitations ..... 98
10.3 Editing Limitations ..... 98
10.4 Search and Replacement Limitations ..... 99
10.5 Undo Limitations ..... 99
10.6 Performance Limitations ..... 99
10.7 Future Improvements ..... 100
**11. CONCLUSION ..... 101**
**APPENDIX A: SOURCE CODE STRUCTURE ..... 102**
A.1 Project Directory Structure ..... 102

A.2 Description of Source Files ..... 103
A.3 Build Configuration ..... 103
A.4 Automated Testing File ..... 104
Automated Test Summary ..... 104
**GROUP MEMBERS' CONTRIBUTION ..... 105**
Individual Contributions ..... 105
Collaborative Contributions ..... 106

# 1. INTRODUCTION AND PROBLEM DESCRIPTION

## 1.1 Introduction

A text editor is a software application that allows users to create, view, modify, search, and save text documents. Text editors are used in programming, documentation, and everyday text processing, and their usefulness depends on how reliably and efficiently they handle frequent changes to the content.

The objective of this project is to design and implement a functional text editor in C++ using fundamental concepts of Data Structures and Algorithms (DSA). The editor supports document creation, opening and saving files, inserting and deleting text, searching, replacing, and undoing previous operations.

Three core data structures are implemented manually. A doubly linked list organizes the lines of a document. A dynamic character array stores and manipulates the text of each individual line. A stack-based history records editing operations so that they can be reversed. In addition, the Knuth-Morris-Pratt (KMP) algorithm is used for efficient pattern searching.

The project provides two interfaces built on one shared, interface-independent core: a console-based interface and a graphical user interface (GUI) developed with the Qt framework. The GUI offers a line-numbered editor, a sidebar and File menu for file operations, keyboard shortcuts, a search panel that lists every match, and light and dark themes. Together, the two interfaces demonstrate the practical use of data structures, algorithm design, memory management, and complexity analysis in a complete application.

## 1.2 Problem Description

Editing multi-line documents involves several challenges. Text must be organized so that lines can be inserted and removed efficiently, editing history must be kept so that mistakes can be reversed, and searching and replacing must remain fast as documents grow. File operations must also be safe, so that an error or an interrupted save never destroys existing data.

The main objective of this project is to develop a text editor that provides essential document management and editing functions while demonstrating the practical application of Data Structures and Algorithms.

The application allows users to create new documents, view their contents, open existing text files, and save their work, either under the current name or under a new one. It supports inserting and deleting text at specified positions, including character-level and line-level operations.

The editor also provides search and replace functions. Users can locate words or character sequences and replace a single occurrence or all matching occurrences. An undo mechanism reverses insertion, deletion, and replacement operations, which makes editing more reliable.

To meet these requirements, suitable data structures and algorithms are used to organize the document, manage edits, keep the operation history, and search text efficiently. Input validation

and error handling are included throughout, so that invalid positions, empty search patterns, incorrect inputs, and file access errors are reported clearly instead of causing failures.

## 1.3 Project Objectives

The main objectives of this project are:

1. To design and implement a functional text editor using C++.
2. To provide essential document management operations: creating, opening, saving, saving under a new name, and displaying text documents.
3. To implement text insertion and deletion at specified positions, including character-level and line-level operations.
4. To implement efficient text searching using the Knuth-Morris-Pratt (KMP) algorithm.
5. To provide text replacement for a single occurrence and for all matching occurrences.
6. To implement an undo mechanism using a stack-based data structure that maintains the editing history.
7. To ensure that the undo facility covers insertion, deletion, and replacement operations.
8. To apply a doubly linked list, a dynamic character array, and a stack for document management and text manipulation.
9. To implement the core data structures manually, using appropriate memory management techniques.
10. To incorporate input validation and error handling that manage invalid operations and keep the application reliable.
11. To analyze the time and space complexity of the major operations.
12. To test the functionality, correctness, efficiency, and reliability of the application.
13. To develop both a console interface and a graphical user interface using C++ and the Qt framework, built on one shared core.

## 1.4 Scope of the Project

The scope of this project is limited to the design and development of a functional text editor using C++ and fundamental concepts of Data Structures and Algorithms. The application concentrates on essential document management and text-editing operations.

The editor supports the following functions:

1. **Document Management:** creating new documents, opening existing text files, saving documents, saving files under new names, and opening additional editor windows. Files are written safely, and unsaved changes are always confirmed before they could be lost.
2. **Text Display:** showing the complete document, individual lines, and document statistics, namely the number of lines and characters (with a word count in the GUI).
3. **Text Editing:** inserting text at specified positions, deleting characters, words, or ranges of text, and inserting or deleting single lines or ranges of lines.
4. **Search:** finding words or character sequences with the KMP algorithm, listing every match by line and column. The search engine supports case-sensitive, case-insensitive, and whole-word matching, which the GUI exposes as options.
5. **Replace:** replacing the first occurrence, a selected occurrence, or all occurrences of a word or text pattern.

6. **Undo:** reversing previous editing operations using a stack-based mechanism with a fixed capacity. When the history is full, the oldest entry is discarded so that editing can continue.
7. **Input Validation and Error Handling:** handling invalid positions, empty search patterns, incorrect user inputs, and file access errors.
8. **User Interface:** a console-based interface and a Qt-based graphical interface with menus, buttons, and keyboard shortcuts for every file operation.

The project focuses on single-document editing and basic text processing. It does not cover advanced word-processing features such as rich-text formatting, image insertion, collaborative editing, or cloud-based document management.

## 1.5 Course Learning Outcomes

This project applies the concepts taught in the CS-218 Data Structures and Algorithms course and addresses the requirements of the Complex Engineering Problem (CEP).

The project involves selecting and implementing suitable data structures to manage the document and its editing operations. A doubly linked list organizes the document lines, a dynamic character array stores and modifies the text of each line, and a stack maintains the history of editing operations to support undo.

It also applies algorithms for insertion, deletion, searching, replacement, and undo. The Knuth-Morris-Pratt string-matching algorithm provides efficient pattern searching within the document.

The implementation uses fundamental programming concepts, including object-oriented programming, modular design, pointers, dynamic memory allocation, file handling, and input validation. The core data structures are written by hand, which shows how they work internally and how their memory is managed. The design separates the core logic from the user interface, so the same code serves both the console and the graphical version.

The time and space complexity of the major operations are analyzed using Big-O notation to understand how performance changes with document size and input length.

The project also builds software design and testing skills by combining individual components into a complete application and evaluating its correctness, functionality, and reliability.

Overall, the project gives practical experience in applying, implementing, and analyzing fundamental data structures and algorithms to solve a real-world engineering problem.

# 2. REQUIREMENTS AND ASSUMPTIONS

## 2.1 Functional Requirements

Functional requirements describe the operations and services that the text editor must provide to users. The application supports document management, text editing, searching, replacement, undo, and error handling through its console-based and Qt graphical interfaces.

### 2.1.1 File Operations

The application provides the following file management operations:

- Create a new document.
- Open an existing text file.
- Save the current document.
- Save the document under a new filename using the Save As option.
- Close the current document by creating a new one (the graphical version also provides a close button on the document tab).
- Open an additional, independent editor window (graphical version).
- Exit the application.
- Warn the user about unsaved changes before a document is replaced or the application is closed, and allow the user to save, discard, or cancel.
- Write files safely, so that a failed save does not damage an existing file.
- Preserve document contents, including the file's line-ending style, when saving and reopening files.
- Provide every file operation through clickable controls in the graphical version (sidebar and File menu) and through keyboard shortcuts.

### 2.1.2 Text Display

The application provides the following text display functionalities:

- Display the complete document.
- Display document contents line by line.
- Display a specific line of the document.
- Display the total number of lines and characters.
- Show line numbers, the cursor's line and column, and line, word, and character counts in the graphical editor.
- Update the document display after editing operations.

### 2.1.3 Text Insertion and Deletion

The application supports the following editing operations:

- Insert text at a specified position within a line.
- Insert individual characters into a document (typing in the graphical editor).
- Delete a character at a specified position.
- Delete a word at a specified position.

- Delete a specified range of characters.
- Insert a new line at a specified position.
- Delete an individual line.
- Delete a specified range of lines.
- Validate character positions and line numbers before performing editing operations.
- Maintain the document's structure after insertion and deletion.

### 2.1.4 Search Operations

The application provides text-searching functionality using the Knuth-Morris-Pratt (KMP) string-matching algorithm.

The search requirements include:

- Search for a specified word or character sequence.
- Identify all matching occurrences within the document.
- Display the line and column positions of matching results.
- Support case-sensitive and case-insensitive searching (case-insensitive matching applies to ASCII letters).
- Provide an optional whole-word matching feature.
- Provide the case-sensitivity and whole-word options as selectable settings in the graphical interface.
- Navigate to a selected search result in the graphical interface.
- Reject empty search patterns and handle searches in empty documents appropriately.

### 2.1.5 Replace Operations

The application supports the following text replacement operations:

- Replace the first occurrence of a specified string.
- Replace a single selected occurrence chosen from the list of search results (graphical version).
- Replace all occurrences of a specified string.
- Apply case-sensitive or case-insensitive matching, and whole-word matching when selected, for the search and for Replace All in the graphical version.
- Update the document after a successful replacement.
- Record replacement operations for undo functionality.
- Treat the Replace All operation as a single undo operation.

### 2.1.6 Undo Operations

The application provides undo functionality using a manually implemented stack-based data structure.

The undo requirements include:

- Undo text insertion.
- Undo text deletion.

- Undo text replacement.
- Undo line insertion and deletion.
- Restore the document to its previous state for supported editing operations.
- Maintain a bounded history of editing operations.
- Discard the oldest operation when the undo history reaches its maximum capacity.

The console version maintains a maximum of 100 undo operations, while the Qt graphical version maintains a maximum of 50 undo operations.

### 2.1.7 Error Handling

The application incorporates error-handling mechanisms to improve reliability and prevent invalid operations.

The requirements include:

- Handle invalid line numbers and character positions.
- Reject empty search patterns.
- Handle nonexistent or inaccessible files.
- Reject invalid file names.
- Handle operations performed on an empty document.
- Prevent deletion beyond the available text range.
- Validate user input before performing editing operations.
- Handle an empty undo history without causing unexpected errors.
- Report a failed save or open operation without losing the current document.
- Display appropriate error messages when an operation cannot be completed.

## 2.2 Non-Functional Requirements

The text editor is designed to meet the following quality requirements:

- **Correctness and Reliability:** Editing and file operations should produce accurate results.
- **Modularity:** The application should have separate modules for document management, editing, searching, replacement, file handling, and undo operations, independent of the user interface.
- **Readability:** Code should use meaningful names, proper structure, and comments.
- **Efficiency:** Suitable data structures and algorithms should ensure efficient text processing.
- **User-Friendly Interface:** Both console and Qt interfaces should be clear and easy to use.
- **Memory Management:** Dynamic memory should be allocated and released properly.
- **Data Integrity:** Document contents should remain consistent during editing, undo, and file operations.
- **Maintainability:** The code should be easy to understand, modify, and extend.
- **Portability:** The application should support compatible C++17 compilers, CMake, and Qt environments on Windows, Linux, and macOS.
- **Testability:** Individual modules and overall functionality should be tested for correctness and reliability.

## 2.3 Assumptions

The design and implementation are based on the following assumptions:

- The application provides both console-based and Qt graphical interfaces.
- Documents are maintained in memory during editing and stored as plain-text files.
- UTF-8 text encoding is supported.
- A doubly linked list manages document lines, with each line stored in a dynamic character array.
- Line numbers are one-based, while internal indexing is zero-based.
- Console insertion and deletion positions use zero-based offsets, while search results use one-based column positions. The graphical version shows all columns as one-based.
- Text searching uses the KMP algorithm.
- Searching is case-sensitive by default, with optional case-insensitive and whole-word matching. The console search is always case-sensitive, and the graphical version offers the options.
- Case-insensitive matching applies to ASCII letters only.
- Spaces are treated as ordinary characters, and empty search patterns are not allowed.
- Undo supports the specified editing operations, with a maximum history of 100 operations in the console version and 50 in the Qt version.
- The oldest undo operation is discarded when the history reaches its limit.
- The application supports plain-text editing rather than advanced rich-text formatting.
- Supported files retain their existing line-ending style when opened and saved.

## 2.4 Hardware and Software Requirements

The development and execution of the text editor require a suitable computer system, a C++ development environment, and the necessary software tools. The application uses the C++17 standard and CMake as its build system. Qt is required for the graphical user interface, while the console version can be compiled independently without Qt.

**Table 2.1: Hardware and Software Requirements**

| Component | Specification |
|---|---|
| Programming Language | C++ |
| C++ Standard | C++17 |
| Compiler | GCC, MinGW, or another compatible C++17 compiler |
| Build System | CMake 3.16 or higher |
| IDE or Editor | Qt Creator, Visual Studio Code, or another compatible C++ IDE |
| Operating System | Windows or another supported platform |
| GUI Framework | Qt 6 (Qt 5 is also supported, if configured) |
| Qt Modules | Qt Widgets |
| Optional Module | Qt SVG for graphical icons |
| External Libraries | Standard C++ Library and Qt framework |
| File Format | Plain-text files |
| Text Encoding | UTF-8 |

# 3. SYSTEM AND DESIGN OVERVIEW

## 3.1 System Overview

The Text Editor application provides two interfaces: a console-based interface and a Qt graphical user interface (GUI). Both interfaces use the same backend components to perform document editing, searching, replacing, file management, displaying, and undo operations.

The console interface uses a numbered menu that displays the current file name, document modification status, content preview, line count, character count, and available operations. Users can select commands from the menu to perform different tasks.

**Command Processing:**

1. The application initializes the `TextDocument` object and the required service objects, including `EditOperations`, `TextSearch`, `TextReplace`, `FileManager`, `DisplayManager`, and `InputValidator`.
2. The editor screen and main menu are displayed, allowing the user to select an operation.
3. The selected menu option is validated using `InputValidator`. Invalid menu inputs are rejected, and the user is allowed a maximum of five input attempts.
4. After a valid selection, the application requests the required parameters, such as line number, character position, text, search pattern, replacement text, or file name. These parameters are validated before the operation is performed.
5. The request is forwarded to the appropriate backend class:
    - `EditOperations` handles text insertion, deletion, modification, and undo operations.
    - `TextSearch` searches for specific text patterns.
    - `TextReplace` performs text replacement operations.
    - `FileManager` handles document creation, opening, saving, and file management.
    - `DisplayManager` displays document contents and statistics.
6. The selected operation is executed, and its result is checked. If the operation is unsuccessful, an appropriate error message is displayed.
7. After completing the operation, the result or error message is displayed, and the application returns to the main menu for further commands.

When the user selects the Exit option, the application checks whether the document contains unsaved changes. The user can choose to save the document, discard the changes, or cancel the exit operation. The application terminates after the changes are successfully saved or discarded.

The Qt graphical interface follows the same processing approach. Its `EditorWindow` class manages the backend objects and converts user actions, such as button clicks, keyboard shortcuts, and text input, into corresponding method calls. It provides an interactive editing environment with a text editor, search functionality, file operations, and undo support.

Both interfaces share the same backend components and operate on the same document structure. This modular design ensures consistent functionality, reduces code duplication, and separates user interaction from document-processing logic.

## 3.2 Class Design

The system consists of several classes and supporting structures responsible for document management, text editing, searching, replacing, file handling, input validation, display, and undo operations. The application follows a modular object-oriented design in which each class performs a specific responsibility.

The original design concepts are mapped to the implementation as follows: `TextEditorApp` functionality is distributed between the console entry point (`main()` in `main.cpp`) and the graphical interface (`EditorWindow`). The `Document` concept is implemented through `TextDocument`, while `LineList` and `CharBuffer` functionality is handled internally by the `LineNode` structure. The `UndoManager` functionality is implemented through `UndoStack` and `EditOperations`, while the `SearchEngine` functionality is provided by `TextSearch`.

The UML class diagram illustrates the relationships between the frontend and backend components, including their attributes, methods, inheritance, composition, and dependencies.

Public document operations use one-based line numbers, while internal line indexing is zero-based. Character positions are represented as zero-based UTF-8 byte offsets.

### 3.2.1 Frontends: main.cpp and EditorWindow

The application provides two frontends: a console-based interface and a Qt graphical user interface.

**Console Frontend (main.cpp)**

The console frontend is implemented through the `main()` function in `main.cpp`. It initializes the shared `TextDocument`, file name, modification status, and required service objects, including `EditOperations`, `TextSearch`, `TextReplace`, `FileManager`, `DisplayManager`, and `InputValidator`.

The `printEditorScreen()` function displays document information, a preview of its contents, statistics, and the numbered menu.

User selections are validated through `InputValidator::readIntInRange()`. The menu supports options from 0 to 18, with a maximum of five input attempts. Commands are processed through a switch statement, and exceptions are handled to prevent unexpected program termination. Before exiting, the application resolves any unsaved changes.

**Graphical Frontend (EditorWindow)**

The graphical frontend is implemented through the `EditorWindow` class in `QtMain.cpp`. It inherits from `QMainWindow` and uses the same backend components as the console interface.

It provides a line-numbered text editor, search panel, sidebar, menus, toolbar, and undo functionality. File commands, including New, New Window, Open, Save, Save As, and Exit,

are implemented as shared Qt actions. These actions can be accessed through sidebar buttons, the File menu, and keyboard shortcuts.

The GUI supports up to 50 undo operations.

Text entered through the graphical editor is processed using `EditOperations::replaceDocumentText()`. Since Qt uses UTF-16 character positions while the core uses UTF-8 byte offsets, the frontend converts positions before passing them to the backend.

### 3.2.2 TextDocument

`TextDocument` represents the active text document. It maintains the document contents using a doubly linked list of line nodes.

It maintains three main attributes:

- `head_`: Pointer to the first line node.
- `tail_`: Pointer to the last line node.
- `lineCount_`: Total number of lines.

The constructor initializes the document with one empty line. Its public functions include `lineCount()`, `isEmpty()`, `getLine()`, `lineLength()`, `getAllText()`, `clear()`, `loadText()`, and `swap()`.

The class also contains private functions such as `nodeAt()`, `insertLineRaw()`, `deleteLineRaw()`, and `removeLineRaw()`. These modification functions are accessible to the friend class `EditOperations`, ensuring that document modifications are controlled through the editing and undo mechanism.

Copy construction and copy assignment are disabled to prevent unsafe shallow copying of dynamically allocated memory.

### 3.2.3 LineNode and Character Buffer

`LineNode` is a private nested structure of `TextDocument`. Each node represents one document line and contains the following attributes:

- `data`: Pointer to a dynamically allocated character array.
- `length`: Number of bytes currently used.
- `capacity`: Allocated buffer size.
- `prev`: Pointer to the previous node.
- `next`: Pointer to the next node.

The `prev` and `next` pointers connect the nodes to form a doubly linked list, allowing navigation between document lines.

The character buffer is not implemented as a separate class. Instead, it is maintained through the `data`, `length`, and `capacity` attributes of `LineNode`. The buffer stores UTF-8 encoded text and maintains a null terminator (`'\0'`).

The `setText()` function updates the buffer and increases its capacity when required. Dynamically allocated memory is released using `delete[]` when a node is destroyed.

### 3.2.4 UndoStack and UndoRecord

`UndoStack` maintains the history of editing operations using a bounded doubly linked list. Each `UndoRecord` stores information about an editing operation, including its type, line number, byte position, previous text, and replacement text.

The supported editing operation types are:

- `InsertText`
- `DeleteText`
- `InsertLine`
- `DeleteLine`
- `ReplaceText`
- `ReplaceDocument`

The main functions of `UndoStack` include `push()`, `pushCoalescing()`, `peek()`, `pop()`, and `discardTop()`.

The stack supports a maximum of 100 records in the console version and 50 records in the graphical version. When the capacity is reached, the oldest record is discarded to make room for a new operation.

Consecutive typing or deletion operations can also be combined into a single undo record through `pushCoalescing()`. This reduces unnecessary entries in the undo history and makes undo functionality more efficient.

### 3.2.5 EditOperations

`EditOperations` manages all modifications to the document. It maintains a reference to `TextDocument` and owns an `UndoStack` to record editing history.

Its main functions include:

- `insertText()`
- `insertCharacter()`
- `deleteCharacter()`
- `deleteWord()`
- `deleteRange()`
- `insertLine()`
- `deleteLine()`
- `deleteLineRange()`
- `replaceText()`

- `replaceDocumentText()`
- `undoLastOperation()`

Before modifying the document, the class validates the requested operation and records the necessary undo information. Invalid operations return `false` without changing the document.

The `replaceDocumentText()` function identifies the changed region by comparing the common prefix and suffix of the old and new text. This allows large text changes to be recorded as a single undo operation.

The class also provides functions for checking the undo history, retrieving the number of stored operations, and clearing the undo records.

### 3.2.6 TextSearch and TextReplace

`TextSearch` and `TextReplace` provide the searching and replacing functionality of the application.

**TextSearch**

`TextSearch` implements the Knuth-Morris-Pratt (KMP) algorithm to search for patterns within document lines. It returns `Match` structures containing the one-based line number and zero-based byte column.

The `searchText()` function supports the following search options:

- Case-sensitive searching.
- Case-insensitive searching.
- Whole-word searching.

These options are controlled through the `matchCase` and `wholeWord` parameters.

The search operation reports its results through `SearchStatus`, which includes `Found`, `NotFound`, `EmptyPattern`, and `EmptyDocument`.

**TextReplace**

`TextReplace` uses `TextSearch` to locate matching patterns and `EditOperations` to perform replacements.

Its main functions include:

- `replaceFirstOccurrence()`
- `replaceOccurrenceAt()`
- `replaceAllOccurrences()`

The class also validates replacement parameters and counts matching occurrences.

Replacement operations are recorded through `EditOperations`, allowing users to undo their changes. The Replace All operation is treated as a single undo operation.

### 3.2.7 FileManager

`FileManager` manages document file operations. It maintains references to the shared `TextDocument`, current file name, and modification status.

Its main functions include `createNewDocument()`, `openFile()`, `saveFile()`, `saveAs()`, `closeFile()`, and `resolveUnsavedChanges()`.

The `createNewDocument()` function creates a new empty document, while `openFile()` loads the contents of an existing file.

When opening a file, its contents are first read into temporary storage and parsed into a temporary document. The active document is updated only after successful loading.

During saving, the contents are written to a temporary file before it replaces the target file. This approach reduces the risk of losing existing data if saving fails.

The class also detects and preserves supported LF and CRLF line-ending styles. File-related errors are reported through exceptions without unnecessarily modifying the existing document state.

Before closing the application or replacing the current document, `FileManager` handles unsaved changes by allowing the user to save, discard, or cancel the operation.

### 3.2.8 InputValidator and DisplayManager

`InputValidator` and `DisplayManager` provide input validation and document display functionality for the console interface.

#### InputValidator

`InputValidator` provides static functions for validating menu choices, line numbers, character positions, ranges, search patterns, and file names.

Its main functions include:

- `isValidLineNumber()`
- `isValidPosition()`
- `isValidRange()`
- `isSearchStringEmpty()`
- `isDocumentEmpty()`
- `isValidFilename()`
- `fileExists()`
- `validateMenuChoice()`
- `validateUserInput()`
- `readLine()`

- `readIntInRange()`

The `readLine()` and `readIntInRange()` functions handle console input and support a maximum of five attempts for the relevant input process.

**DisplayManager**

`DisplayManager` maintains a constant reference to `TextDocument`, allowing document information to be displayed without modifying its contents.

Its main functions include `displayDocument()`, `displayLine()`, `displayAllLines()`, `countLines()`, `countCharacters()`, and `displayStatistics()`.

These functions display document contents and statistics, including the total number of lines and characters.

Together, `InputValidator` and `DisplayManager` improve the reliability and usability of the console interface by ensuring that user input is validated and document information is presented clearly.

## 3.3 Class Diagram

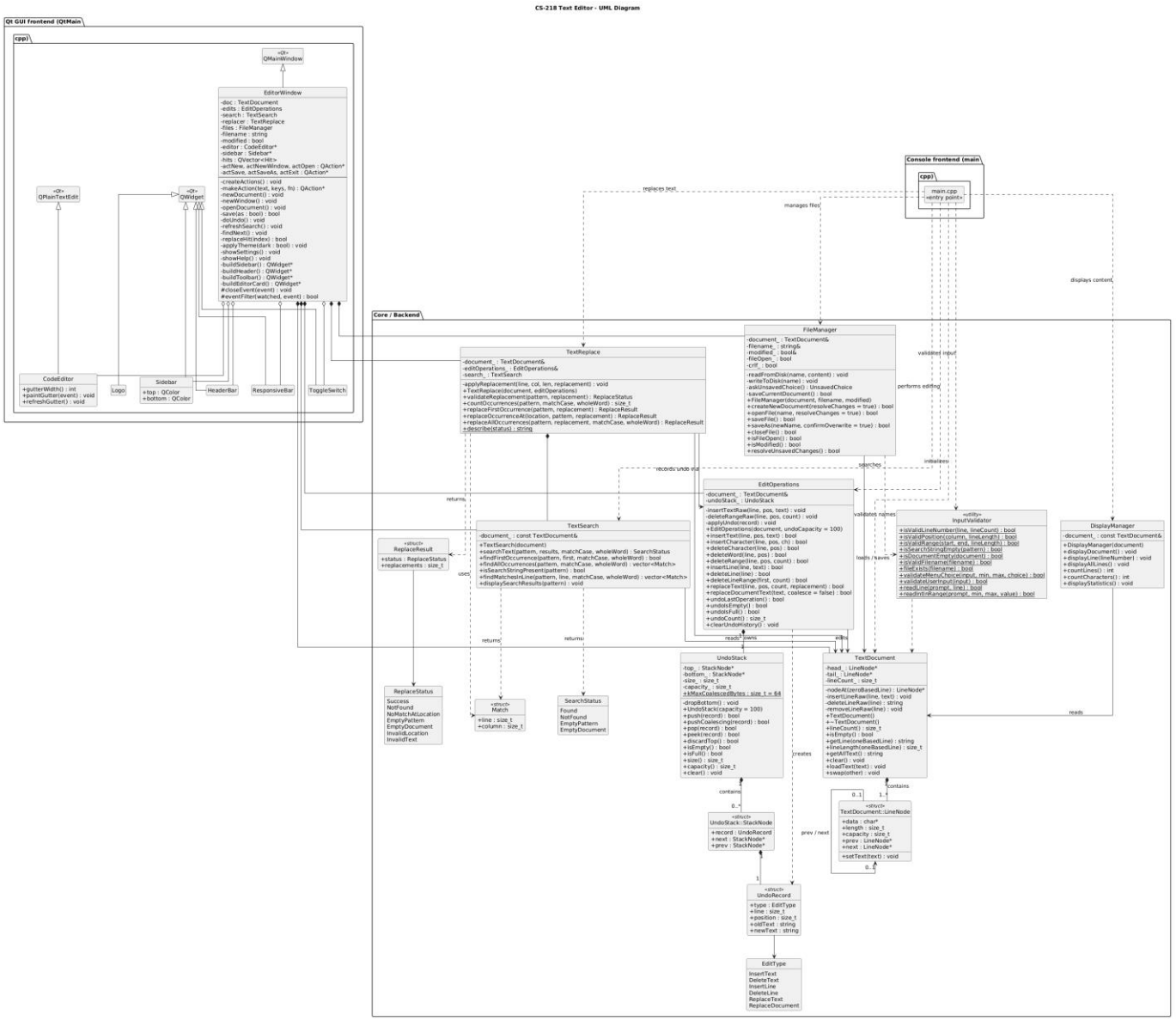



*Figure 3.1: UML Class Diagram*

**Figure 3.1** illustrates the UML class diagram of the CS-218 Text Editor application. It represents the overall structure of the system, including its frontend and backend components, classes, attributes, methods, and relationships.

The diagram is divided into three main parts: the Core / Backend, the Qt GUI Frontend, and the Console Frontend.

The Core / Backend contains the main classes responsible for document management, editing, searching, replacing, undo operations, file handling, input validation, and document display.

The Qt GUI Frontend contains the `EditorWindow` class and its supporting Qt widgets, which provide the graphical interface for user interaction. The Console Frontend represents `main.cpp` as the application entry point, responsible for initializing the required objects and processing menu-based commands.

The relationships in the diagram show how the classes communicate and share responsibilities. `TextDocument` maintains document lines through a doubly linked list of `LineNode` structures. `EditOperations` manages document modifications and owns an `UndoStack` containing editing history. `TextSearch` and `TextReplace` provide searching and replacement functionality, while `FileManager` handles file operations.

The class diagram demonstrates the modular organization of the application and shows how both frontends use the shared backend components.

## 3.4 Program Workflow

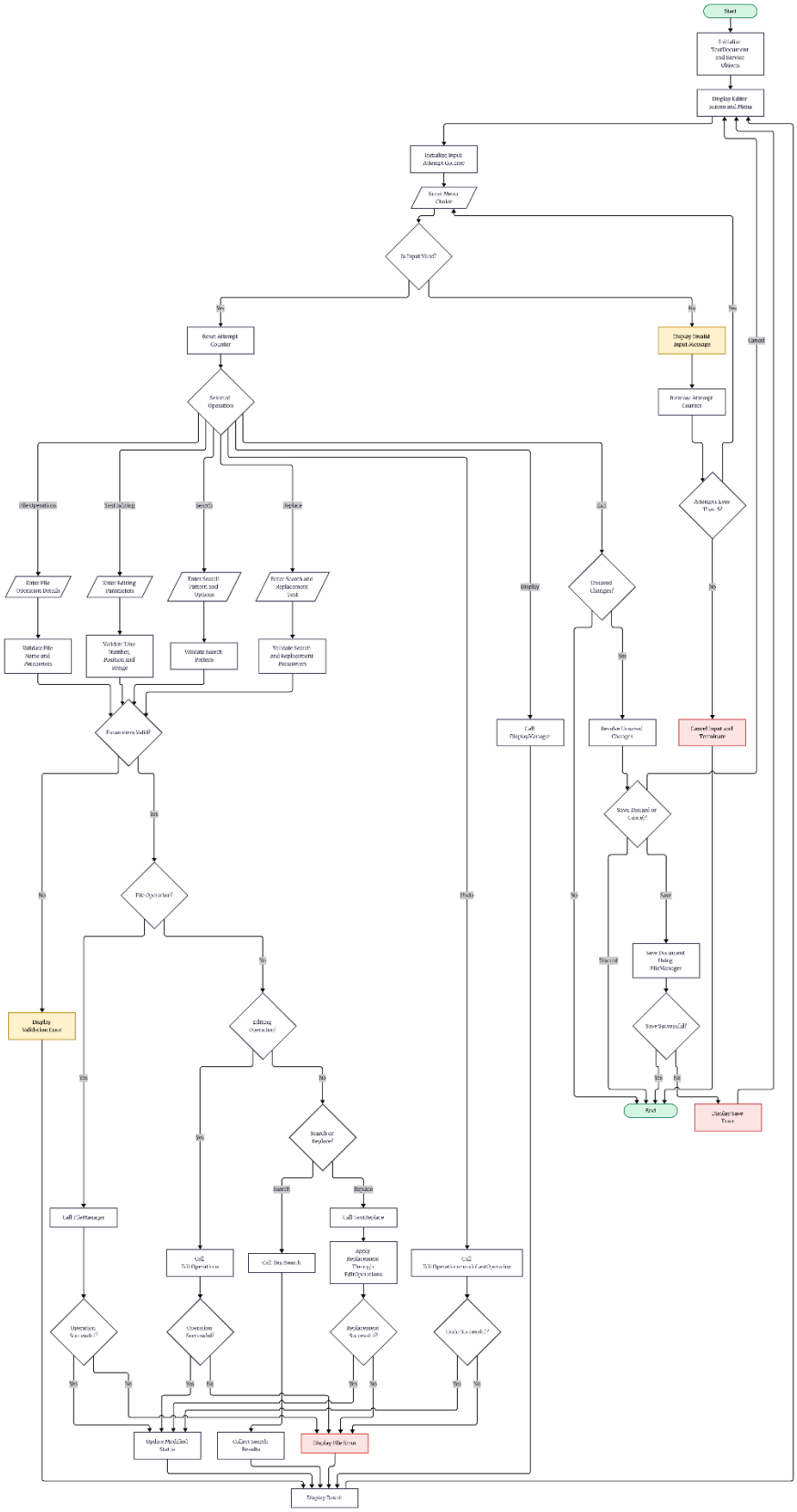



*Figure 3.2: Program Flowchart*

**Figure 3.2** illustrates the program workflow of the CS-218 Text Editor application. It describes the sequence of operations performed by the console interface, starting from application initialization and ending with application termination.

When the program starts, `main()` initializes the shared `TextDocument` object and the required service objects. The editor screen and numbered menu are then displayed.

The program follows these main steps:

- **Display the Editor Screen:** Displays the current file name, document status, content preview, statistics, and available menu options.
- **Read and Validate Menu Choice:** Accepts the user's menu selection and validates it using `InputValidator`.
- **Check Input Attempts:** If the menu input is invalid, the program displays an error message and allows another attempt. After five consecutive invalid attempts, the input is cancelled and the program terminates.
- **Enter Required Parameters:** Requests the necessary information for the selected command, such as line number, character position, text, search pattern, replacement text, or file name.
- **Validate Parameters:** Checks the entered parameters before forwarding the request to the appropriate backend component.
- **Execute Selected Operation:** Calls the responsible class to perform the requested operation.
- **Display Result:** Displays the operation result or an appropriate error message before returning to the main menu.

The selected operation is handled by the corresponding backend component:

- `FileManager` handles file-related operations.
- `EditOperations` performs text editing and undo operations.
- `TextSearch` searches for text patterns.
- `TextReplace` performs text replacement.
- `DisplayManager` displays document contents and statistics.

**Error and Exit Handling:**

- Invalid menu input is rejected and requested again. After five consecutive invalid entries, the input is cancelled and the program ends.
- Invalid parameters, such as a character position beyond the end of a line, are rejected by `InputValidator` or the relevant backend function. Unsuccessful editing operations return `false` without modifying the document.
- Runtime failures, such as file-related errors, are reported through exceptions that are caught and handled by `main()`.
- Selecting Exit starts the unsaved-changes procedure. The user can save changes, discard them, or cancel the exit operation. The application terminates only when the changes are saved or discarded, or when the document is unmodified.

This workflow ensures that user commands are validated, operations are processed by the appropriate classes, and errors are handled without unnecessarily affecting the document.

## 3.5 Module Interaction

The modules interact through well-defined communication paths. The frontend handles user interaction and forwards requests to the appropriate backend classes rather than directly modifying the internal document structure.

The main module interactions are described below:

- **Editing:** The frontend receives and validates user input through `InputValidator` and calls `EditOperations`. The `EditOperations` class validates the requested position, records an `UndoRecord` in its `UndoStack`, and modifies the appropriate `LineNode` within `TextDocument`. When the operation succeeds, the frontend updates the document's modification status.
- **Searching:** The frontend calls `TextSearch::searchText()`, which reads document lines through a read-only reference to `TextDocument` and returns matching positions. Since searching does not modify the document, no undo record is created.
- **Replacing:** The frontend calls `TextReplace` to perform replacement operations. It uses `TextSearch` to locate matching patterns and `EditOperations` to apply the modifications. The changes are recorded in the undo history. For Replace All, the updated document text is passed to `EditOperations::replaceDocumentText()` so the entire replacement operation can be undone as a single action.
- **Undo:** The frontend calls `EditOperations::undoLastOperation()`, which retrieves the most recent record from `UndoStack`, applies the inverse operation to `TextDocument`, and removes the processed record. In the Qt version, the editor then refreshes its displayed text using the updated document contents.
- **Files:** The frontend communicates with `FileManager` to create, open, save, and manage documents. `FileManager` reads or writes file contents and updates the shared `TextDocument`. After creating or opening a new document, the frontend clears the previous undo history because the existing records no longer correspond to the new document contents.
- **Display:** `DisplayManager` and the graphical interface access `TextDocument` through read-only references to display document contents and statistics. The interface refreshes its display after relevant operations.

Because the backend modules share references to the same `TextDocument` object rather than maintaining separate document copies, all components operate on consistent document data. This approach improves modularity, reduces unnecessary data duplication, and ensures consistent behavior between the console and graphical interfaces.

# 4. DATA STRUCTURES USED

## 4.1 Overview of Selected Data Structures

The CS-218 Text Editor uses three manually implemented data structures and one auxiliary structure for efficient text searching. These structures are designed to manage document contents, maintain editing history, and support efficient pattern matching.

The main data structures used in the implementation are:

- **Doubly Linked List (`TextDocument` and `LineNode`):** Stores the document as a sequence of lines, allowing traversal in both directions.
- **Dynamic Character Array (`data`, `length`, and `capacity`):** Stores the text of each individual line using dynamically allocated memory.
- **Bounded Stack (`UndoStack`):** Maintains the history of editing operations using a manually implemented doubly linked list.
- **KMP Failure Table (`std::vector`):** Supports efficient pattern searching using the Knuth-Morris-Pratt algorithm.

The implementation does not use standard library containers such as `std::list` or `std::stack` for the document or undo history. However, standard library components are used for supporting operations. These include `std::string` for text handling, `std::vector` for search results and the KMP failure table, and file streams for reading and writing documents.

## 4.2 Doubly Linked List

### 4.2.1 Structure and Node Representation

The document is stored as a doubly linked list in which each node represents one line of text. Each node contains a pointer to its character buffer, the length and capacity of that buffer, and pointers to the previous and next nodes.

The `LineNode` structure is implemented as a nested structure inside the `TextDocument` class.

Its main members are:

```
struct LineNode {
    char* data;              // Character buffer
    std::size_t length;      // Bytes currently in use
    std::size_t capacity;    // Allocated buffer size
    LineNode* prev;          // Previous line
    LineNode* next;          // Next line
};
```

The `TextDocument` class maintains three main attributes:

- `head_`: Points to the first node in the list.
- `tail_`: Points to the last node in the list.

- `lineCount_`: Stores the total number of lines.

The nodes are connected through their `prev` and `next` pointers, forming a doubly linked list. The first node has a null previous pointer, while the last node has a null next pointer.

For example, a document containing three lines is represented as follows:

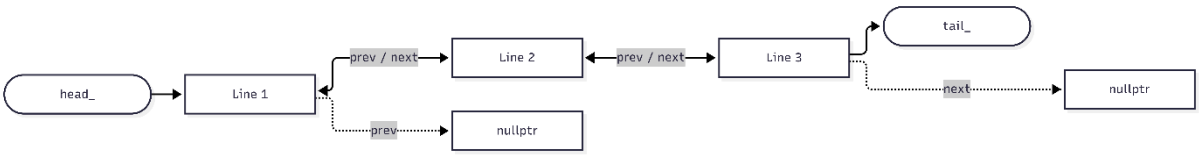



*Figure 4.1: Doubly Linked List Representation of TextDocument*

A doubly linked list is used instead of a singly linked list because it supports traversal from both directions. This allows the implementation to locate a line by starting from whichever end is closer.

### 4.2.2 Main Operations

The main operations performed on the linked list are:

- **Traversal:** `nodeAt()` locates a node using its zero-based index. It starts from `head_` when the requested index is closer to the beginning; otherwise, it starts from `tail_`.
- **Line Retrieval:** `getLine()` retrieves the contents of a specified line using its one-based line number. It returns the text as a `std::string`.
- **Line Length:** `lineLength()` returns the number of bytes stored in a specified line without copying its contents.
- **Insertion:** `insertLineRaw()` creates a new node and connects it to the list. It handles insertion at the beginning, end, or middle of the document.
- **Deletion:** `removeLineRaw()` disconnects the selected node from its neighbours, releases its memory, and updates the line count.

Once the required node has been located, inserting or unlinking a node requires only a constant number of pointer updates. However, locating the node requires traversal through the list.

If the document contains L lines, locating a node using `nodeAt()` takes O(L) time in the worst case and approximately O(L/2) traversal steps at most when starting from the nearer end.

### 4.2.3 Implementation

The linked list is implemented manually using dynamic memory allocation through `new` and `delete`.

The constructor of `TextDocument` initializes the document with one empty line. This ensures that the document always contains at least one line.

The `insertLineRaw()` function creates a new node and connects it to the existing list. The `removeLineRaw()` function removes a node while maintaining the links between the remaining nodes.

The destructor of `LineNode` releases its dynamically allocated character buffer using `delete[]`. Similarly, `TextDocument::clear()` traverses the list and deletes every node.

The `loadText()` function first normalizes line endings, converting CRLF and CR sequences into LF. It then constructs a temporary document and replaces the existing contents using `swap()` only after successful construction.

This approach helps preserve the original document if loading fails during memory allocation or parsing.

## 4.3 Dynamic Character Array

### 4.3.1 Structure and Memory Representation

Each `LineNode` stores its text in a dynamically allocated character array. The array is managed through three main attributes:

- `data`: Pointer to the allocated character array.
- `length`: Number of bytes currently used to store the text.
- `capacity`: Total number of bytes allocated for the array.

The buffer stores text using UTF-8 encoding and maintains a null terminator (`'\0'`) at the end of the stored text.

The following conditions are maintained:

```
length < capacity
data[length] == '\0'
```

For example, the text `The fox` stored in a buffer with a capacity of 16 bytes can be represented as:

```
Index:   0   1   2   3   4   5   6   7   8 ... 15
Data:   'T' 'h' 'e' ' ' 'f' 'o' 'x' '\0'  ? ...  ?
```

In this example:

```
length   = 7
capacity = 16
```

The `length` attribute identifies the actual end of the text, while `capacity` indicates the allocated memory available for the line.

Since UTF-8 is used, a non-ASCII character may occupy between two and four bytes. Therefore, the implementation uses byte offsets internally rather than assuming that every character occupies one byte.

### 4.3.2 Main Operations

The dynamic character array supports the following operations:

- **Insertion:** `insertTextRaw()` allocates a new character array large enough to hold the existing text and the inserted text. It copies the prefix, inserted text, and remaining suffix into the new buffer, then releases the old array.
- **Deletion:** `deleteRangeRaw()` uses `std::memmove()` to shift the remaining bytes to the left and update the length.
- **Replacement:** `setText()` updates the contents of a line. It reuses the existing allocation when sufficient space is available and allocates a larger buffer when necessary.
- **Resizing:** The buffer capacity is increased when the existing allocation cannot hold the required text.

Before insertion or deletion, the relevant positions and ranges are validated. The implementation also prevents insertion or deletion at invalid UTF-8 byte boundaries.

### 4.3.3 Memory Management

Memory for the character arrays is allocated using `new[]` and released using `delete[]`.

The `setText()` function uses geometric capacity growth. When a larger buffer is required, its capacity is increased to at least twice the previous capacity, or to the exact required size if that is larger. An initial capacity of 16 bytes is used when no buffer has been allocated.

In contrast, the raw insertion operation allocates a buffer of the exact required size.

The implementation creates and fills a new buffer before deleting the previous one. This ensures that if memory allocation fails, the original buffer remains available and unchanged.

The implementation also uses `std::numeric_limits<std::size_t>::max()` to check for potential size overflow during memory allocation and text insertion.

Deletion is performed in place using `std::memmove()`, avoiding the need to allocate another character array.

These memory-management techniques help maintain data integrity and reduce unnecessary memory operations.

## 4.4 Bounded Stack (UndoStack)

### 4.4.1 Stack Representation

The undo history follows the Last In, First Out (LIFO) principle. The most recent editing operation is stored at the top of the stack, while the oldest operation is stored at the bottom.

The `UndoStack` class is implemented manually using a doubly linked list of `StackNode` objects.

Each `StackNode` contains an `UndoRecord` and two pointers:

- `next`: Points toward the older node.
- `prev`: Points toward the newer node.

The stack maintains four main attributes:

- `top_`: Points to the newest undo record.
- `bottom_`: Points to the oldest undo record.
- `size_`: Stores the current number of undo records.
- `capacity_`: Defines the maximum number of records allowed.

Each `UndoRecord` stores the information required to reverse an editing operation. Its main fields are:

- `type`: Identifies the editing operation.
- `line`: Stores the zero-based line index.
- `position`: Stores the zero-based byte position or document offset.
- `oldText`: Stores the previous text.
- `newText`: Stores the inserted or replacement text.

The supported edit types are:

- `InsertText`
- `DeleteText`
- `InsertLine`
- `DeleteLine`
- `ReplaceText`
- `ReplaceDocument`

The meaning of the record fields depends on the type of operation. For example, an `InsertText` record stores the inserted text so that it can be removed during undo, whereas a `DeleteText` record stores the deleted text so that it can be restored.

### 4.4.2 LIFO Principle and Main Operations

The LIFO principle is important for undo functionality because editing operations must be reversed in the opposite order in which they were performed.

For example, suppose a user inserts the text `very` into a line and then deletes the word `quick`.

The first operation creates undo record A, while the second creates undo record B.

The correct undo order is:

```
First Operation:  Insert "very "  -> Record A
Second Operation: Delete "quick"  -> Record B

Undo Order:

Record B -> Record A
```

Undoing the newest operation first ensures that the stored positions and text information remain valid.

The `UndoStack` provides the following main operations:

- **push():** Adds a new undo record to the top of the stack.
- **pushCoalescing():** Combines consecutive compatible editing operations into a single undo record when possible.
- **peek():** Retrieves the newest record without removing it.
- **pop():** Retrieves and removes the newest record.
- **discardTop():** Removes the newest record without copying its contents.
- **clear():** Deletes all records and resets the stack.
- **isEmpty():** Checks whether the stack contains any records.
- **isFull():** Checks whether the stack has reached its capacity.

The console version supports a maximum of 100 undo records, while the Qt graphical version supports 50 records.

When the stack reaches its capacity, the oldest record is automatically discarded to make room for a new operation. This keeps the undo history within its allocated limit.

### 4.4.3 Implementation and Undo Mechanism

The `push()` operation first allocates a new `StackNode`. If the stack is already full, the oldest record is removed through `dropBottom()`. The new record is then linked at the top.

The `dropBottom()` operation requires constant-time pointer updates because the stack maintains a direct reference to its oldest node.

The `pushCoalescing()` function combines compatible consecutive editing operations when they occur on the same line. These operations include consecutive insertions, deletions, and certain replacement combinations.

The combined text is limited to a maximum of 64 bytes using the `kMaxCoalescedBytes` constant. If two operations cannot be combined, a separate undo record is created.

When the user performs an editing operation, the required undo information is recorded before the document is modified.

During undo, `EditOperations::undoLastOperation()` retrieves the newest record using `peek()`, applies the inverse operation to the document, and removes the record using `discardTop()` only after the inverse operation succeeds.

This approach preserves the undo record if an exception occurs while reversing an operation.

The `applyUndo()` function also checks the stored information before reversing certain replacement operations. If the expected text is not present at the recorded position, the operation reports an error rather than applying an incorrect modification.

## 4.5 Additional Algorithm: Knuth-Morris-Pratt (KMP) Searching

The editor uses the Knuth-Morris-Pratt (KMP) algorithm to search for text patterns efficiently.

KMP improves pattern searching by avoiding unnecessary comparisons when a mismatch occurs. Instead of restarting the search from the beginning of the pattern, it uses previously calculated information about matching prefixes and suffixes.

### 4.5.1 Failure Table (LPS Table)

The KMP algorithm uses a failure table, also known as the LPS (Longest Proper Prefix which is also a Suffix) table.

Each entry in the table stores the length of the longest proper prefix of the pattern that is also a suffix ending at the current position.

The `buildFailureTable()` function in `TextSearch.cpp` constructs this table before searching begins.

For example, consider the pattern `ABABC`:

```
Pattern:  A  B  A  B  C
Index:    0  1  2  3  4
LPS:      0  0  1  2  0
```

If a mismatch occurs after some characters have matched, the algorithm uses the LPS table to determine the next position in the pattern. This avoids repeating comparisons that have already been completed.

### 4.5.2 Searching Process

The searching process is implemented through two main functions:

- `buildFailureTable()`: Constructs the LPS table for the search pattern.
- `scanLine()`: Searches an individual document line using the KMP algorithm.

The `TextSearch::searchText()` function first validates the search pattern and checks whether the document contains searchable text. It then builds the failure table and searches the document line by line.

The search supports the following options:

- **Case-Sensitive Search:** Compares characters without changing their case.
- **Case-Insensitive Search:** Converts ASCII letters to a common case during comparison using `std::tolower()`.
- **Whole-Word Search:** Accepts a match only when the characters before and after the pattern are not considered word characters.

For whole-word matching, letters, digits, underscores, and non-ASCII bytes are treated as word characters.

The implementation reports accepted matches as `Match` structures containing the one-based line number and zero-based byte column.

Accepted matches are non-overlapping because the search restarts its matching state after recording a successful occurrence. Searching is performed independently on each line, so a pattern cannot extend across a line break.

### 4.5.3 Time Complexity

Let:

- P = Length of the search pattern in bytes.
- C = Total number of text bytes across all document lines.
- L = Number of document lines.

The time complexity of the KMP algorithm is:

| Operation | Time Complexity |
|---|---|
| Failure table construction | O(P) |
| Searching one line | $O(P + C_i)$ |
| Searching the complete document | O(P + C) |

Here, $C_i$ represents the number of bytes in an individual line.

The failure table is constructed once for each search operation and reused while scanning all document lines.

Therefore, the overall searching time is linear in the combined pattern and document sizes, excluding the additional cost of storing the returned match results.

The KMP algorithm provides efficient searching without repeatedly comparing previously matched text.

## 4.6 Data Structure Summary

Table 4.1 summarizes the main data structures used in the CS-218 Text Editor, their purposes, and their supported operations.

**Table 4.1: Data Structure Summary**

| Data Structure | Purpose | Main Operations |
|---|---|---|
| Doubly Linked List | Stores document lines using connected `LineNode` objects. | Insert line, delete line, traverse, retrieve line |
| Dynamic Character Array | Stores the text of each individual document line. | Insert text, delete text, replace text, resize buffer |
| Bounded Stack | Maintains the history of editing operations for undo functionality. | Push, pop, peek, discard top, clear |
| KMP Failure Table (`std::vector`) | Supports efficient pattern searching using the KMP algorithm. | Build failure table, scan line, identify matches |

The selected data structures provide the foundation for the text editor's main functionality. The doubly linked list organizes document lines, while dynamic character arrays manage the contents of individual lines. The bounded stack maintains editing history and supports undo operations. The KMP failure table improves the efficiency of searching.

Together, these structures provide an organized and efficient approach to document storage, text modification, undo management, and pattern searching.

# 5. JUSTIFICATION FOR SELECTED DATA STRUCTURES

## 5.1 Justification for the Linked List

A text document consists of multiple lines that may be inserted, deleted, or restored through undo operations. A doubly linked list is suitable for managing these lines because inserting or removing a node requires updating only the pointers of neighbouring nodes. Other lines do not need to be shifted or copied.

In an array of lines, inserting or deleting a line in the middle requires shifting the following elements. When the array reaches its capacity, additional memory allocation and copying may also be required.

The doubly linked list is preferred over a singly linked list for three main reasons:

- **Bidirectional traversal:** The `nodeAt()` function can start from either `head_` or `tail_`, selecting the nearer end to reduce traversal distance.
- **Efficient deletion:** A node can be unlinked by updating its `prev` and `next` pointers.
- **Efficient insertion at the end:** The `tail_` pointer allows new lines to be appended in O(1) time, which is useful when loading a document from a file.

**Advantages:**

- Insertion and deletion require only a few pointer updates after locating the target node.
- No shifting of other lines is required.
- Memory allocation grows and shrinks according to the number of lines.
- Existing nodes remain at their allocated memory addresses until they are deleted.

**Disadvantages:**

- Random access is not supported. Accessing a particular line requires traversal from the nearer end, taking $O(\min(i, L - i))$ time for a zero-based line index i.
- Each node requires additional memory for two pointers.
- Linked-list traversal has poorer cache locality than contiguous array storage.

The traversal cost is particularly relevant to searching and Replace All operations. If these operations repeatedly call `getLine()` for each line, the list may be traversed again for every access. Consequently, locating all lines individually can require approximately $L^2/4$ link traversals for a document containing L lines.

For documents containing a few thousand lines, this overhead is generally manageable. However, sequential traversal would be more efficient for very large documents.

## 5.2 Justification for the Dynamic Array

Within each document line, characters must be stored compactly and accessed efficiently by their column positions. A dynamic array provides contiguous memory, O(1) indexed access, and efficient block-copy operations.

Each `LineNode` maintains a dynamically allocated character array through three attributes:

- `data`: Pointer to the allocated character array.
- `length`: Number of text bytes currently stored.
- `capacity`: Total allocated buffer size.

The dynamic array is appropriate because character-level editing affects only one line rather than the entire document.

When inserting or deleting a character, the characters following the editing position may need to be shifted. For a line containing C bytes, insertion at position p requires shifting approximately $C - p$ bytes in the worst case. This operation is limited to the affected line.

A linked list of individual characters would introduce additional pointer overhead for every character. It would also make direct column access inefficient because reaching a particular character would require traversal.

The dynamic array therefore provides a better balance between memory usage, direct character access, and editing performance.

## 5.3 Justification for the Stack

Undo functionality requires the most recent editing operation to be reversed first. This follows the Last-In, First-Out (LIFO) principle, making a stack an appropriate data structure.

The `UndoStack` uses a bounded doubly linked list to maintain editing history. Each entry contains an `UndoRecord` describing the operation, including its type, line number, position, previous text, and replacement text.

The stack provides the following operations:

- `push()`: Adds a new undo record.
- `pushCoalescing()`: Combines compatible consecutive editing operations.
- `pop()`: Retrieves and removes the most recent record.
- `peek()`: Examines the most recent record without removing it.
- `discardTop()`: Removes the most recent record.
- `clear()`: Removes all stored undo records.

Push, pop, and peek operations take O(1) time. The stack also maintains a fixed capacity to prevent unlimited memory growth.

The console version supports a maximum of 100 undo records, while the graphical version supports 50. When the capacity is reached, the oldest record is discarded through the `bottom_` pointer, allowing removal in O(1) time.

A linked-list implementation was selected because it supports efficient removal of the oldest record without shifting the remaining entries. An array-based stack with a circular buffer could also achieve O(1) removal, but the linked-list implementation provides direct pointer-based management of the history.

Additionally, `pushCoalescing()` combines consecutive typing or deletion operations where appropriate. This prevents individual keystrokes from unnecessarily consuming the available undo history.

## 5.4 Justification for Additional Algorithms

The Knuth-Morris-Pratt (KMP) algorithm is used for searching text patterns within document lines.

A naive string-search algorithm compares the pattern at every possible text position. Its worst-case time complexity is O(C × P), where C is the number of text bytes and P is the pattern length.

KMP improves this performance by preprocessing the search pattern and constructing a prefix table. When a mismatch occurs, the algorithm uses previously matched information to avoid repeating unnecessary comparisons.

Its time complexity is O(C + P) for a line, with O(P) additional space for the prefix table.

The implementation also supports different search conditions through the `matchCase` and `wholeWord` parameters. This allows users to perform case-sensitive, case-insensitive, and whole-word searches.

Other string-searching algorithms were considered:

- **Boyer-Moore:** Can perform well on average, particularly with longer patterns, but requires additional preprocessing and more complex matching rules.
- **Rabin-Karp:** Uses hashing to locate possible matches, but hash collisions require additional verification.
- **Naive Search:** Is simple to implement but can require significantly more comparisons in the worst case.

KMP was selected because it provides a guaranteed linear-time searching approach, requires relatively little additional memory, and is straightforward to implement and explain. It also demonstrates the pattern-matching concepts covered in the course.

The KMP complexity applies to searching an individual line. The additional cost of locating lines through the linked list must also be considered when analysing the complete document-search operation.

## 5.5 Comparison with Alternative Data Structures

Table 5.1 compares the selected document representation with alternative data structures.

**Table 5.1: Comparison of Document Representations**

| Representation | Character Editing | Line Insertion / Deletion | Main Drawback |
|---|---|---|---|
| Single contiguous character array | O(N) in the worst case | O(N) | Editing may shift large portions of the document; locating a line requires scanning line breaks. |
| Array of lines (std::vector<std::string>) | O(C) in the worst case | O(L) | Line insertion and deletion may shift subsequent elements. |
| Gap buffer | O(1) near the gap; O(N) to move the gap | O(N) in the worst case | Efficient near one editing position but less suitable for frequent operations at different positions. |
| Piece table | O(1) for appending to the add buffer | Depends on the descriptor index | Supports efficient editing and natural undo concepts but requires more complex management and indexing. |
| Rope / balanced tree | Typically O(log N) | Typically O(log N) | Higher implementation complexity and additional memory overhead. |
| Doubly linked list of lines + dynamic array per line (Chosen) | O(D + C) | O(D) to locate and O(1) to relink | Locating a line requires linear traversal from the nearer end. |

**Notation used in Table 5.1:**

- N = Total number of document characters or bytes.
- L = Total number of document lines.
- C = Number of characters or bytes in the affected line.
- D = Number of links traversed to locate the required line.

The complexity values represent typical or worst-case behaviour under the stated operations. Actual performance depends on the implementation and the position of the requested operation.

The selected representation keeps editing costs relatively local. Character operations affect only one line, while line insertion and deletion require updating a small number of pointers after locating the target node.

Its main limitation is the cost of locating a particular line. Other representations can provide faster access through indexing or tree structures, but they introduce additional memory requirements, data movement, or implementation complexity.

For the scale and requirements of this project, the combination of a doubly linked list and dynamic character arrays provides a practical balance between efficiency, memory management, and implementation simplicity.

## 5.6 Summary of Design Decisions

The selected data structures and algorithms satisfy the main requirements of the text editor:

- **Doubly linked list:** Supports line insertion and deletion without shifting other document lines.
- **Dynamic array per line:** Provides direct character access and efficient character-level editing.
- **Bounded undo stack:** Maintains editing history using the LIFO principle while limiting memory consumption.
- **KMP algorithm:** Provides guaranteed linear-time pattern searching with case-sensitive, case-insensitive, and whole-word options.

The two-level document representation separates line management from character storage. This allows each data structure to handle the operations for which it is most suitable.

The implementation also demonstrates important data structures and algorithms concepts, including pointer manipulation, dynamic memory allocation, linked-list traversal, stack operations, bounded history management, string matching, and time-complexity analysis.

The main limitation of the selected design is the linear cost of locating a line. This limitation is considered in the complexity analysis in Chapter 7, where possible performance improvements can also be discussed.

# 6. DESCRIPTION OF IMPORTANT ALGORITHMS

This chapter describes the main algorithms used in the CS-218 Text Editor. It explains document creation, file handling, text insertion and deletion, line management, searching, replacing, undo operations, and input validation. Each algorithm is presented through its implementation process and corresponding pseudocode.

## 6.1 Document Creation

A document is created using the `TextDocument` constructor. Initially, the constructor sets `head_` and `tail_` to `nullptr` and initializes `lineCount_` to zero. It then calls `insertLineRaw(0, "")` to create the first line.

This operation allocates a `LineNode` containing an empty string with an initial character-buffer capacity of 16 bytes. The new node becomes both the head and tail of the linked list, and `lineCount_` becomes one.

Consequently, a newly created document contains one empty line. The `isEmpty()` function returns `true` because the document contains no actual text, even though its linked list contains one line.

When the user selects the New command, `FileManager::createNewDocument()` creates another empty `TextDocument` and exchanges it with the current document using `swap()`. Since `swap()` exchanges the document pointers and line count rather than copying every line, the operation takes O(1) time. The old document contents are released when the temporary document is destroyed.

The file name and modified status are then reset, and the default line-ending style of the platform is selected.

**Pseudocode:**

```
CREATE_DOCUMENT()
    head <- NULL
    tail <- NULL
    lineCount <- 0

    INSERT_LINE_RAW(0, "")       // Create one empty line

    return document
```

## 6.2 Opening and Saving a File

The `FileManager` class handles opening, creating, and saving documents. File operations are designed to protect the current document from incomplete or unsuccessful operations.

**Opening a File**

The `openFile(name)` function performs the following steps:

1. Validates the supplied file name using `InputValidator::isValidFilename()`.
2. Opens the file in binary mode and reads its contents in blocks of 64 KiB into a temporary string.
3. Throws `std::runtime_error` if the file cannot be opened and `std::ios_base::failure` if a reading error occurs.
4. Loads the contents into a temporary `TextDocument` using `loadText()`.
5. Normalizes CRLF (`\r\n`) and CR (`\r`) line endings into LF (`\n`) and creates one linked-list node for each line.
6. Preserves a final empty line when the file ends with a line break.
7. Resolves any unsaved changes in the current document by asking the user to save, discard, or cancel.
8. If the operation is approved, swaps the temporary document into the shared document and updates the file name, line-ending style, and modified status.

The current document is not replaced until the new file has been read and loaded successfully. Therefore, a failed read or loading operation leaves the existing document unchanged.

**Saving a File**

The `saveFile()` and `saveAs()` functions retrieve the document contents using `getAllText()`. If the document uses CRLF line endings, the LF characters are converted back to CRLF before writing.

The file is written to a temporary file inside a unique hidden directory next to the target file. The output stream is flushed and closed before the temporary file is renamed to the target.

If the rename operation fails, the implementation attempts a fallback procedure by moving the original file aside and moving the temporary file into its place. If that operation also fails, the original file is restored.

A guard object ensures that the temporary directory is removed when the operation finishes. Save As also requests confirmation before overwriting a different existing file.

The modified flag is cleared only after successful saving. If saving fails, an exception is reported and the modified flag remains set.

**Pseudocode:**

```
OPEN_FILE(name)
    if not VALID_FILENAME(name):
        DISPLAY_ERROR()
        return false

    content <- READ_ALL_BYTES(name)     // Read in 64 KiB blocks

    newDoc <- CREATE_TEMPORARY_DOCUMENT()
    LOAD_TEXT(newDoc, content)          // Normalize line endings

    if CURRENT_DOCUMENT_HAS_UNSAVED_CHANGES:
        choice <- ASK_SAVE_DISCARD_CANCEL()

        if choice = CANCEL:
```

```
            return false

        if choice = SAVE and not SAVE_CURRENT_DOCUMENT():
            return false

    SWAP(document, newDoc)

    filename <- name
    modified <- false
    crlf <- content contains "\r\n"

    return true
SAVE(name)
    text <- GET_ALL_TEXT(document)

    if crlf:
        convert every "\n" to "\r\n"

    CREATE_TEMPORARY_FILE()
    WRITE(text)
    FLUSH()
    CLOSE()

    if RENAME_TEMPORARY_FILE(name) fails:
        RESTORE_ORIGINAL_FILE_IF_NECESSARY()

    modified <- false
    return true
```

## 6.3 Text Insertion Algorithm

The `EditOperations::insertText(line, position, text)` function inserts text at a specified position within an existing document line.

Before performing the insertion, the function validates the following conditions:

- The inserted text must not be empty.
- The inserted text must not contain carriage-return or newline characters.
- The specified line number must be within the document's valid range.
- The insertion position must not exceed the current line length.
- The position must not fall inside a multi-byte UTF-8 character.

The implementation uses zero-based byte positions internally, while public line numbers are one-based.

After validation, an `UndoRecord` of type `InsertText` is pushed onto the undo stack. Recording the operation before modifying the document ensures that the information required for undo is available.

If the raw insertion fails, the newly added undo record is discarded to prevent the undo history from becoming inconsistent.

The raw insertion operation allocates a new character array large enough to hold the original text, the inserted text, and the null terminator. It then copies the original prefix, inserted text, and original suffix into their appropriate positions.

Finally, the old buffer is released, and the node's `data`, `length`, and `capacity` attributes are updated.

**Pseudocode:**

```
INSERT_TEXT(line, pos, text)

    if text is empty or contains CR or LF:
        return false

    if line < 1 or line > lineCount:
        return false

    cur <- GET_LINE(line)

    if pos > LENGTH(cur):
        return false

    if pos < LENGTH(cur) and
       cur[pos] is a UTF-8 continuation byte:
        return false

    record <- UndoRecord(
        InsertText, line - 1, pos, "", text
    )

    PUSH(record)

    try:
        node <- NODE_AT(line - 1)

        buf <- ALLOCATE(
            LENGTH(cur) + LENGTH(text) + 1
        )

        COPY cur[0 .. pos) TO buf[0 .. pos)

        COPY text TO buf[pos .. pos + LENGTH(text))

        COPY cur[pos ..] TO buf[pos + LENGTH(text) ..]

        buf[LENGTH(cur) + LENGTH(text)] <- '\0'

        FREE node.data

        node.data <- buf
        node.length <- LENGTH(cur) + LENGTH(text)
        node.capacity <- ALLOCATED_CAPACITY

        return true

    catch:
        DISCARD_NEW_UNDO_RECORD()
        return false
```

## 6.4 Text Deletion Algorithm

The `EditOperations::deleteRange(line, position, count)` function removes a specified range of bytes from a document line.

Before deletion, the function validates the line number, starting position, and deletion count. The requested range must remain within the existing line, and the count must be at least one.

The removed text is copied into an `UndoRecord` of type `DeleteText`. This information allows the deleted text to be restored during an undo operation.

After recording the operation, the function uses `memmove()` to shift the remaining characters to the left, overwriting the deleted section.

The line's length is reduced by the number of deleted bytes, and the null terminator is placed at the new end of the string.

The `deleteCharacter()` function removes one complete UTF-8 character, which may occupy between one and four bytes. The `deleteWord()` function skips leading whitespace and removes the next word according to its word-selection logic.

Line and character statistics are calculated when required rather than maintained through separate counters.

**Pseudocode:**

```
DELETE_RANGE(line, pos, count)

    if line < 1 or line > lineCount:
        return false

    if count < 1:
        return false

    cur <- GET_LINE(line)

    if pos > LENGTH(cur):
        return false

    if count > LENGTH(cur) - pos:
        return false

    removed <- cur[pos .. pos + count)

    record <- UndoRecord(
        DeleteText, line - 1, pos, removed, ""
    )

    PUSH(record)

    node <- NODE_AT(line - 1)

    MEMMOVE(
        node.data + pos,
```

```
        node.data + pos + count,
        node.length - pos - count
    )

    node.length <- node.length - count

    node.data[node.length] <- '\0'

    return true
```

## 6.5 Line Insertion and Deletion

The document stores lines as nodes in a doubly linked list. Each node contains a character buffer and pointers to its previous and next nodes.

### Line Insertion

The `insertLine(n, text)` function inserts a new line at a specified position. The valid insertion range is from 1 to `lineCount + 1`.

The inserted text must not contain line breaks. Before modifying the document, the function creates an `UndoRecord` of type `InsertLine`.

The `insertLineRaw()` function allocates a new node and connects it to the linked list. Four cases are handled:

1. Inserting into an empty list.
2. Inserting before the current head.
3. Inserting after the current tail.
4. Inserting between two existing nodes.

The head, tail, and neighbouring pointers are updated as required, and the line count is increased.

### Line Deletion

The `deleteLine(n)` function removes a specified line after validating its position.

Before removal, the original line text is stored in an `UndoRecord` of type `DeleteLine`.

The `removeLineRaw()` function locates the target node and updates its neighbouring pointers. If the node is the first or last node, the head or tail pointer is updated accordingly.

The removed node is deleted, releasing its dynamically allocated character buffer, and the line count is decreased.

A special condition applies when the document contains only one line. Instead of removing the final node, the implementation clears its contents and keeps one empty line. The undo record stores the information required to restore the original text.

The `deleteLineRange(first, count)` function removes multiple consecutive lines. It stores the deleted contents together in one undo record, with line breaks separating the original lines. Undo reconstructs the deleted lines and restores their positions.

**Pseudocode:**

```
INSERT_LINE(n, text)

    if n < 1 or n > lineCount + 1:
        return false

    if text contains CR or LF:
        return false

    PUSH(UndoRecord(InsertLine, n - 1, 0, "", text))

    INSERT_LINE_RAW(n - 1, text)

    return true
REMOVE_LINE_RAW(i)

    n <- NODE_AT(i)

    if lineCount = 1:
        n.SET_TEXT("")
        return

    if n.prev != NULL:
        n.prev.next <- n.next
    else:
        head <- n.next

    if n.next != NULL:
        n.next.prev <- n.prev
    else:
        tail <- n.prev

    DELETE n

    lineCount <- lineCount - 1
```

## 6.6 Search Algorithm

The `TextSearch` class implements the Knuth-Morris-Pratt (KMP) algorithm to locate patterns within document lines.

The `searchText(pattern, results, matchCase, wholeWord)` function first checks whether the search pattern is empty or the document contains no text. It returns `EmptyPattern` or `EmptyDocument` when either condition is encountered.

For a valid search request, the algorithm builds the KMP failure table, also known as the Longest Proper Prefix which is also a Suffix (LPS) table.

The LPS table stores the length of the longest proper prefix that is also a suffix for each pattern position. This information allows the algorithm to avoid repeating comparisons after a mismatch.

The search then processes each document line and collects matching positions in `Match` structures containing the line number and zero-based byte column.

The algorithm supports:

- Case-sensitive searching.
- Case-insensitive searching.
- Whole-word searching.

For whole-word searches, a match is accepted only when its surrounding characters satisfy the word-boundary conditions. Matches are collected line by line and do not overlap.

The console displays matching positions using one-based line and column numbers. If no match is found, the function returns `NotFound`.

**Pseudocode:**

```
BUILD_LPS(P)

    m <- LENGTH(P)

    lps[0] <- 0
    len <- 0
    i <- 1

    while i < m:

        if P[i] = P[len]:
            len <- len + 1
            lps[i] <- len
            i <- i + 1

        else if len != 0:
            len <- lps[len - 1]

        else:
            lps[i] <- 0
            i <- i + 1

    return lps
KMP_SEARCH_LINE(T, P, lps)

    matched <- 0

    for i <- 0 to LENGTH(T) - 1:

        while matched > 0 and
              T[i] != P[matched]:

            matched <- lps[matched - 1]

        if T[i] = P[matched]:
```

```
            matched <- matched + 1

        if matched = LENGTH(P):

            start <- i - LENGTH(P) + 1

            if not wholeWord or
               WORD_BOUNDARIES_ARE_VALID(T, start, i):

                REPORT(start)

                matched <- 0

            else:
                matched <- lps[matched - 1]
```

The LPS table is built once for the search pattern and reused for each line. The time complexity of KMP searching one line is O(C + P), where C is the line length and P is the pattern length. The LPS table requires O(P) additional space.

## 6.7 Replace First Occurrence

The `TextReplace::replaceFirstOccurrence(pattern, replacement)` function replaces the first matching occurrence of a pattern in the document.

Initially, the function validates the search pattern and replacement text. It rejects empty patterns, empty documents, and replacement text containing line breaks.

The `findFirstOccurrence()` function is then used to locate the first match in document order.

If no matching occurrence exists, the function returns `NotFound`. If the pattern and replacement text are identical, no modification is necessary.

Otherwise, the function calls `EditOperations::replaceText(line, column, patternLength, replacement)` to perform the replacement.

The editing operation stores the original and replacement text in an undo record of type `ReplaceText` before updating the document.

The graphical interface also supports replacing a selected search result through `replaceOccurrenceAt()`. This function verifies that the selected location still contains the expected pattern before applying the replacement.

**Pseudocode:**

```
REPLACE_FIRST(pattern, replacement)

    if pattern is empty:
        return EmptyPattern

    if document is empty:
        return EmptyDocument
```

```
    if replacement contains a line break:
        return InvalidText

    if not FIND_FIRST(pattern, line, col):
        return NotFound

    if pattern != replacement:

        REPLACE_TEXT(
            line,
            col,
            LENGTH(pattern),
            replacement
        )

    return Success
```

## 6.8 Replace All Occurrences

The `TextReplace::replaceAllOccurrences()` function replaces every matching occurrence of a pattern in the document.

Directly modifying the original text during the search could change the positions of later matches. To avoid this problem, the algorithm builds a new document string in a separate buffer.

The process works as follows:

1. Validate the search pattern and replacement text.
2. Initialize an empty output string and a replacement counter.
3. Process each document line independently.
4. Retrieve all matching positions from the original line using `findMatchesInLine()`.
5. Copy the unchanged text before each match into the output string.
6. Append the replacement text.
7. Continue processing from the end of the original match.
8. Append the remaining unchanged text after the final match.
9. Preserve line breaks between document lines.
10. Pass the completed text to `EditOperations::replaceDocumentText()`.

All match positions refer to the original document. Therefore, earlier replacements cannot invalidate the positions of later matches.

The replacement text is not searched again during the same operation. This ensures that replacements such as changing `a` into `aa` terminate correctly.

The `replaceDocumentText()` function compares the old and new document contents, identifies the changed region using their common prefix and suffix, and records the operation for undo.

A single undo operation restores the original document contents, even when several occurrences were replaced.

**Pseudocode:**

```
REPLACE_ALL(pattern, replacement, matchCase, wholeWord)

    if pattern is empty:
        return EmptyPattern

    if document is empty:
        return EmptyDocument

    if replacement contains a line break:
        return InvalidText

    total <- 0
    newText <- ""

    for each line n from 1 to lineCount:

        old <- GET_LINE(n)

        matches <- FIND_MATCHES_IN_LINE(
            pattern, n, matchCase, wholeWord
        )

        copied <- 0

        for each match m in matches:

            append old[copied .. m.column)
                   to newText

            append replacement to newText

            copied <- m.column + LENGTH(pattern)

        append old[copied ..] to newText

        total <- total + LENGTH(matches)

        if n < lineCount:
            append '\n' to newText

    if total = 0:
        return NotFound

    REPLACE_DOCUMENT_TEXT(newText)

    return Success(total)
```

The algorithm processes the original lines and constructs the resulting document in one pass over the matching and unchanged text. The resulting text is submitted as one editing operation, allowing the entire Replace All operation to be undone together.

## 6.9 Undo Algorithm

The text editor implements undo functionality using a bounded stack of `UndoRecord` structures.

Every successful modifying operation stores the information required to reverse that operation. The `undoLastOperation()` function retrieves the most recent record and applies its inverse operation.

The process consists of three main steps:

1. Use `peek()` to examine the most recent undo record.
2. Call `applyUndo()` to reverse the recorded operation.
3. Use `discardTop()` to remove the record after the inverse operation has been applied successfully.

The inverse operation depends on the original edit type:

- **InsertText:** Deletes the inserted text.
- **DeleteText:** Reinserts the previously deleted text.
- **InsertLine:** Removes the inserted line.
- **DeleteLine:** Restores the deleted line or lines.
- **ReplaceText:** Replaces the new text with the original text.
- **ReplaceDocument:** Restores the previous document contents.

For replacement operations, the implementation verifies that the expected new text is still present at the recorded position before applying the inverse.

For document-level replacements, the original changed region is restored and the document is reloaded as required.

The undo stack also supports coalescing consecutive compatible typing and deletion operations to reduce unnecessary history entries.

Both the console and Qt graphical interfaces use the same backend undo mechanism through `EditOperations`. After an undo operation, the GUI refreshes its editor contents from the updated document.

**Pseudocode:**

```
UNDO_LAST()

    if UNDO_STACK is empty:
        return false

    r <- PEEK(UNDO_STACK)

    switch r.type:

        case InsertText:
            DELETE_RANGE_RAW(
                r.line, r.position, LENGTH(r.newText)
            )

        case DeleteText:
            INSERT_TEXT_RAW(
                r.line, r.position, r.oldText
            )
```

```
        case InsertLine:
            REMOVE_LINE_RAW(r.line)

        case DeleteLine:
            RESTORE_SAVED_LINE_OR_LINES(r)

        case ReplaceText:
            VERIFY_TEXT(r.newText)
            REPLACE_WITH_OLD_TEXT(r.oldText)

        case ReplaceDocument:
            VERIFY_TEXT(r.newText)
            RESTORE_OLD_DOCUMENT_TEXT(r.oldText)

    DISCARD_TOP(UNDO_STACK)

    return true
```

If the undo stack is empty, the function returns `false` without modifying the document.

## 6.10 Error Handling and Input Validation

The text editor uses multiple levels of validation and error handling to prevent invalid operations and protect document contents.

At the console level, input functions validate user entries before commands are executed. At the backend level, editing and searching functions validate their parameters and return appropriate status values when an operation cannot be performed.

Exceptions are used for failures that cannot be handled through ordinary validation, particularly file access and memory-related errors.

The console application catches reported exceptions and displays appropriate error messages rather than terminating unexpectedly.

**Table 6.1: Error Handling and Input Validation**

| Situation | Handling |
|---|---|
| Menu input is not a number or is outside the range 0–18 | `readIntInRange()` displays an invalid-input message and requests another entry. After five failed attempts, the input is cancelled. |
| Input contains control characters or exceeds 1024 characters | `validateUserInput()` rejects the input and displays an appropriate message. |
| Line number is outside the valid range | The console restricts the entry to the valid range, and backend functions return `false` for invalid line numbers. |
| Column is beyond the end of a line or inside a multi-byte UTF-8 character | `insertText()` returns `false`. The console displays: "Insertion was rejected. Check the line and column." |
| Deletion count is zero, too large, or starts beyond the end of the line | `deleteRange()` returns `false`. The console displays: "Deletion was rejected. Check the range." |

| | |
|---|---|
| Empty search pattern | The search operation returns `EmptyPattern` and displays: "Error: the search string cannot be empty." |
| Search or replacement is requested on an empty document | The operation returns `EmptyDocument` without modifying the document. |
| Search pattern is not found | The operation returns `NotFound`, and the document remains unchanged. |
| Replacement text contains a line break | The operation returns `InvalidText`, and the document remains unchanged. |
| Invalid file name, including empty names, excessive length, illegal characters, or reserved device names | `isValidFilename()` rejects the name and displays: "Error: filename is invalid." |
| Missing or unreadable file | A `std::runtime_error` is thrown and caught in `main()`. The existing document remains unchanged. |
| Failed write due to disk space or permission problems | An exception is reported. Temporary-file writing protects the original file, and the modified flag remains set. |
| Unsaved changes when selecting New, Open, or Exit | `resolveUnsavedChanges()` asks the user to save, discard, or cancel the operation. |
| Empty undo history | `undoLastOperation()` returns `false`. The console displays: "There is no operation to undo." |
| Size overflow or inconsistent undo record | `std::length_error` or `std::logic_error` is reported and caught by the application. |

The combination of input validation, return-status checking, exception handling, and temporary-file operations helps maintain document consistency and reduces the risk of data loss.

These mechanisms ensure that invalid commands are rejected before they modify the document and that file-related failures do not unnecessarily destroy the user's existing work.

# 7. TIME AND SPACE COMPLEXITY ANALYSIS

## 7.1 Complexity Analysis Method

The complexity analysis evaluates the time and memory requirements of the major operations performed by the text editor. The analysis is based on the implemented data structures, which consist of a doubly linked list of lines, a dynamic character array for each line, and a bounded undo stack.

The following variables are used throughout this chapter:

- N = Total number of text bytes in the document.
- L = Total number of lines in the document.
- C = Number of bytes in the affected line.
- T = Length of the text being inserted, deleted, or used as a replacement.
- P = Length of the search pattern.
- K = Number of matching occurrences.
- H = Number of records in the undo history (maximum 100 in the console version and 50 in the GUI).
- D = Distance of the target line from the nearer end of the linked list, where $D \leq \frac{L}{2}$
- F = Size of the file in bytes, which is approximately proportional to N.

The analysis considers both the cost of locating a line in the linked list and the cost of manipulating its character buffer. Since the implementation uses a linked list rather than an indexed array, accessing a particular line requires traversing the list.

## 7.2 Time Complexity of Major Operations

Table 7.1 presents the theoretical time complexity of the main operations.

**Table 7.1: Time Complexity of Major Operations**

| Operation | Best Case | Average Case | Worst Case | Explanation |
|---|---|---|---|---|
| Display document | Θ(N) | Θ(N) | Θ(N) | `getAllText()` traverses the linked list and copies the document contents into a string. |
| Display all numbered lines | O(N + L) | O(N + L²) | O(N + L²) | `displayAllLines()` repeatedly calls `getLine(i)`, causing repeated linked-list traversal. |
| Find a line | O(1) | O(L) | O(L) | `nodeAt()` starts from the nearer end of the linked list. |
| Insert a line | O(T) | O(L + T) | O(L + T) | The target position is located, the new text is copied, and the node is linked into the list. |
| Delete a line | O(C) | O(L + C) | O(L + C) | The target node is located, its contents are saved for undo, and the node is removed. |

| | | | | |
|---|---|---|---|---|
| Insert characters | O(C + T) | O(L + C + T) | O(L + C + T) | The line is located, a new buffer is allocated, and the existing and inserted characters are copied. |
| Delete characters | O(C) | O(L + C) | O(L + C) | The line is located, the removed text is stored, and the remaining characters are shifted using `memmove()`. |
| Search text (KMP) | O(1) | O(P + N + $L^2$) | O(P + N + $L^2$) | KMP builds the failure table and searches the document. Repeated indexed line access introduces additional traversal cost. |
| Replace text | O(P + C + T) | O(P + N + $L^2$ + K·T) | O(P + N + $L^2$ + K·T) | Replace First searches for a match, while Replace All processes all matching occurrences and constructs the updated document. |
| Undo | O(1) | O(L + C + T) | O(N + L) | Most operations require locating a line and reversing a local edit. Document-level replacements may require rebuilding the document. |
| Open or save file | Θ(F) | Θ(F) | Θ(F) | File operations read or write the complete file, with additional processing for line-ending conversion and document construction. |

The table shows that operations involving individual lines depend on the distance required to locate the corresponding node. Once the node is located, insertion and deletion of lines require only a constant number of pointer updates.

Character-level operations additionally depend on the size of the affected line because the dynamic array must be copied or shifted.

The KMP algorithm itself provides linear-time pattern matching. However, repeatedly retrieving lines by their index introduces an additional linked-list traversal cost.

## 7.3 Space Complexity of Major Operations

Table 7.2 describes the additional memory requirements of the document representation and major operations.

**Table 7.2: Space Complexity of Major Operations**

| Component or Operation | Space Complexity | Explanation |
|---|---|---|
| Document storage | O(N + L) | Stores the document text and the linked-list nodes associated with each line. |
| Linked-list nodes | Θ(L) | Each line requires one node containing previous and next pointers and other node attributes. |

| | | |
|---|---|---|
| Character buffer | O(C) per line | Each line maintains a dynamically allocated character array with additional capacity for the null terminator. |
| Undo history | O(H·S) | Stores up to H undo records, where S represents the maximum relevant record size. |
| Search algorithm | O(P + K + C) | Stores the KMP failure table, matching results, and temporary line data. |
| File loading | O(F) additional temporary space | File contents, temporary document storage, and normalized text may coexist during loading. |
| Replace All | O(N) | Constructs the updated document text and stores the information required for undo. |

The document requires O(N + L) memory because it stores both character data and the linked-list nodes. Each node contains pointers to its neighbouring nodes, while its character buffer stores the actual line contents.

The undo history is bounded by its configured capacity. However, the memory occupied by individual records depends on the amount of text stored in them. For example, a record representing a large deletion or document replacement can require substantially more memory than a record representing a single character insertion.

Similarly, file loading temporarily requires additional memory because the original file contents and the newly constructed document may coexist before the document is committed.

## 7.4 Complexity Discussion

**Efficient operations**

The doubly linked list provides efficient line insertion and deletion because only the neighbouring pointers need to be updated after locating the target node. No other lines need to be shifted or copied.

Character-level operations are also localized to individual lines. Insertion and deletion affect the dynamic array of the selected line rather than the entire document. Therefore, their processing costs depend mainly on the length of the affected line.

The undo stack provides constant-time push, pop, and peek operations. Removing the oldest record when the stack reaches its capacity also takes O(1) time because the implementation maintains access to both ends of the linked list.

The `getAllText()` function traverses the document once, making its time complexity Θ(N + L). Similarly, file loading and saving process the document contents in linear time.

**Operations involving traversal and shifting**

The main limitation of the selected document representation is the time required to locate a particular line.

The `nodeAt()` function starts from either `head_` or `tail_`, depending on which end is closer. Therefore, accessing a line requires O(L) time in the worst case.

This limitation becomes more noticeable in operations that repeatedly access lines by their indexes.

For example, `searchText()`, `replaceAllOccurrences()`, and `displayAllLines()` retrieve lines individually through indexed access. Each retrieval may traverse a significant portion of the linked list. Consequently, repeatedly accessing all lines can introduce an $O(L^2)$ traversal cost.

Character insertion also requires copying the existing line into a newly allocated buffer. Character deletion uses `memmove()` to shift the remaining characters towards the beginning of the buffer.

Although these operations involve copying or shifting characters, their effects remain limited to the selected line.

**Effect of document size**

For documents containing a few thousand lines, the additional traversal cost may remain manageable. However, as the number of lines increases, repeated linked-list traversal becomes a significant performance limitation.

The measurements in Section 7.5 demonstrate this behaviour. Search time increases from approximately 18 milliseconds for 5,000 lines to approximately 1.8 seconds for 50,000 lines.

The document size increases approximately tenfold, while the search time increases substantially more.

The line-by-line traversal measurements also show a similar increase. This indicates that repeated linked-list access contributes significantly to the execution time.

## 7.5 Performance Considerations

Several implementation decisions influence the performance and memory requirements of the text editor.

**Dynamic array resizing**

The `insertTextRaw()` function allocates a new character buffer for each insertion and copies the existing contents into it. Therefore, inserting text into a line requires $O(C + T)$ time.

This approach is straightforward and maintains a compact buffer, but repeated insertions can involve unnecessary memory allocation and copying.

Using geometric capacity growth would reduce repeated reallocations, particularly when editing long lines. The existing `setText()` function already supports capacity growth.

**Memory usage**

Each document line requires a linked-list node containing its data pointer, length, capacity, and neighbouring pointers. It also requires a dynamically allocated character buffer.

Consequently, documents containing many short lines may consume noticeably more memory than their actual text size.

File loading also temporarily increases memory usage because the file contents and newly constructed document may exist simultaneously before the document is committed.

**Repeated replacement operations**

Calling Replace First repeatedly to replace K occurrences requires multiple searches. Each search may involve traversing the linked list and scanning the document.

In contrast, Replace All processes the original lines and constructs the updated document in one operation. It also records the complete replacement as a single undo operation.

This makes Replace All more efficient than repeatedly performing individual replacements when many occurrences must be changed.

**Undo history**

The undo history is limited to 100 records in the console application and 50 records in the graphical interface.

Consecutive typing and deletion operations can be combined using `pushCoalescing()`, reducing the number of individual records stored in the history.

The implementation also supports a maximum coalesced text size of 64 bytes. When the history reaches its configured capacity, the oldest record is discarded.

**Console screen refreshing**

The console interface refreshes the editor screen after each command. Displaying the document and calculating its statistics requires traversing the document contents.

Therefore, repeated screen refreshing introduces additional processing time for large documents.

**Possible optimizations**

The following improvements could reduce the performance limitations:

- Sequential traversal: Search, Replace All, and display operations could traverse the linked list directly rather than repeatedly calling `getLine(i)`. This would reduce the traversal component from $O(L^2)$ to $O(L)$.
- Geometric buffer growth: Increasing character-buffer capacity geometrically would reduce repeated memory allocations during text insertion.

- Running statistics: Maintaining line and character counts would reduce the need to recalculate statistics after every operation.
- Alternative document structures: A gap buffer, piece table, or balanced tree could improve performance for larger documents, although these structures would increase implementation complexity.

**Table 7.3: Measured Execution Times of the Backend**

| L (Lines) | N (Bytes) | Search | Line-by-Line Walk Only | Replace All | Undo of Replace All | Insert at Middle Line | Insert at Line 1 |
|---|---|---|---|---|---|---|---|
| 1,000 | 52,889 | 0.65 ms | 0.41 ms | 1.97 ms | 0.34 ms | 0.0025 ms | 0.0002 ms |
| 5,000 | 268,889 | 18.1 ms | 16.8 ms | 37.1 ms | 1.06 ms | 0.0147 ms | 0.0001 ms |
| 20,000 | 1,088,889 | 241.8 ms | 235.1 ms | 548.4 ms | 8.15 ms | 0.0638 ms | 0.0001 ms |
| 50,000 | 2,738,889 | 1,820.9 ms | 1,816.6 ms | 3,805.3 ms | 18.07 ms | 0.2530 ms | 0.0001 ms |

The measurements were obtained using the backend compiled with g++ 13.3 and the `-O2` optimization flag on a single virtual CPU running Ubuntu 24.04.

Each test document contained L lines, with approximately 55 characters per line. Search, Replace All, and undo operations were timed once, while insertion times represent the average of 1,000 operations.

The measurements are indicative and may vary depending on the hardware and execution environment.

The results demonstrate that the execution time of search and Replace All increases significantly as the number of lines grows. Between 20,000 and 50,000 lines, the search time increases approximately 7.5 times, while the number of lines increases only 2.5 times.

The line-by-line traversal measurements show a similar growth pattern. This supports the theoretical analysis that repeated linked-list traversal is a major performance limitation.

The insertion measurements also demonstrate the effect of line location. Insertion at the first line is considerably faster than insertion near the middle because the first line can be accessed directly through `head_`, whereas a middle line requires traversal.

## 7.6 Summary

The complexity analysis demonstrates that the selected data structures provide efficient memory management and localized editing operations.

The doubly linked list allows lines to be inserted and deleted without shifting other lines. The dynamic character arrays provide direct column access and efficient character manipulation. The bounded undo stack maintains editing history with constant-time stack operations, while the KMP algorithm provides linear-time pattern matching within individual lines.

However, the linked-list representation introduces a performance limitation when multiple lines are accessed individually by index. Repeated traversal can increase the complexity of searching, replacing, and displaying large documents to $O(L^2)$ in the traversal component.

The measured results support this analysis and demonstrate the relationship between document size and execution time.

Overall, the implementation provides a practical demonstration of linked lists, dynamic arrays, stacks, string-matching algorithms, memory management, and algorithmic complexity analysis.

# 8. TESTING STRATEGY AND TEST CASES

## 8.1 Testing Objectives

Testing was performed to verify that the text editor operates correctly, invalid input is rejected without damaging the document, and undo operations restore previous document states accurately. Testing also verifies the behaviour of the doubly linked list, character buffers, search and replacement algorithms, UTF-8 character handling, and undo history.

The main objectives of testing were:

- To verify that a new document is initialized with one empty line.
- To check that text and lines can be inserted, deleted, and accessed correctly.
- To verify that undo operations restore the original document contents.
- To test case-sensitive, case-insensitive, and whole-word searching.
- To verify Replace First and Replace All operations.
- To ensure that invalid positions, empty inputs, and incorrect menu choices are rejected.
- To verify that UTF-8 characters are not incorrectly divided during editing.
- To test undo stack capacity, record ordering, and eviction behaviour.
- To ensure that invalid operations do not unintentionally modify the document.

The testing process covered normal operations, boundary conditions, invalid inputs, and interactions between the backend components.

## 8.2 Testing Approach

The application was tested using an automated, non-interactive test driver named `tests.cpp`. The test driver executes individual test cases and reports their results in the console using PASS or FAIL messages.

The following testing methods were applied:

- **Unit testing of document management:** TextDocument was tested for constructor initialization, empty-line access, text loading, line numbering, line separators, and trailing newline handling.
- **Editing and undo testing:** EditOperations was tested for text insertion, deletion, line insertion, line deletion, range deletion, text replacement, and restoration through undo operations.
- **Search testing:** TextSearch was tested using case-sensitive, case-insensitive, and whole-word matching. Empty patterns and patterns that do not exist were also tested.
- **Replacement testing:** TextReplace was tested for Replace First, Replace All, replacement validation, and undoing replacement operations.
- **UTF-8 testing:** Tests checked that insertion inside a multi-byte UTF-8 character is rejected and that complete UTF-8 characters can be deleted and restored.
- **Undo stack testing:** UndoStack was tested for record insertion, stack capacity, eviction of the oldest record, retrieval order, and empty-stack behaviour.
- **Input validation testing:** InputValidator was tested using valid and invalid menu choices, line numbers, character positions, and empty search strings.

- **Boundary-value testing:** Tests covered positions at and beyond the end of a line, invalid line numbers, empty input, and operations performed with an empty undo history.
- **Regression testing:** The complete test driver can be executed again after source-code modifications to check whether previously working functionality remains correct.

Each test is identified by a unique identifier, making it easier to locate failures and verify individual features.

## 8.3 Test Environment

The automated tests were executed using the following environment:

| Component | Specification |
|---|---|
| Programming Language | C++ |
| C++ Standard | C++17 |
| Compiler | g++ |
| Compiler Flags | -std=c++17 -Wall -Wextra |
| Operating System | Windows (PowerShell terminal) |
| Test Driver | tests.cpp |
| Test Executable | tests |
| Testing Type | Automated backend testing |
| Test Execution | Non-interactive console execution |

The test driver was compiled with the backend source files required for document management, editing, searching, replacement, undo operations, and input validation.

The tests were executed by running the compiled test program from the project directory. The driver automatically performed the test cases and displayed the corresponding PASS or FAIL result.

The test output was captured in two screenshots. The first screenshot shows the document, editing, and search tests, while the second screenshot shows the remaining search, replacement, UTF-8, undo stack, and input validation tests, together with the final test summary.

## 8.4 Functional Test Cases

The automated test driver executed 55 functional and validation test cases. These tests were grouped according to the backend component or functionality being tested.

**Table 8.1: Functional Test Cases**

| Test ID | Feature | Test Description | Expected Result | Status |
|---|---|---|---|---|
| TD-01 | Document creation | Create a new TextDocument | One empty line is created | Pass |
| TD-02 | Empty line access | Read the empty document line | Empty line is returned | Pass |
| TD-03 | Text loading | Load text containing three lines | Three lines are created correctly | Pass |
| TD-04 | Line access | Retrieve a line using one-based numbering | Correct line is returned | Pass |
| TD-05 | Document retrieval | Retrieve complete document text | Line separators are preserved | Pass |
| TD-06 | Trailing newline | Load text ending with a newline | Final empty line is created | Pass |
| ED-01 | Text insertion | Insert text at a valid position | Text is inserted correctly | Pass |
| ED-02 | Undo insertion | Undo a text insertion | Original text is restored | Pass |
| ED-03 | Text deletion | Delete a specified range | Selected text is removed | Pass |
| ED-04 | Undo deletion | Undo a deletion | Deleted text is restored | Pass |
| ED-05 | End-position insertion | Insert text at the end of a line | Text is appended successfully | Pass |
| ED-06 | Invalid insertion | Insert beyond the end of a line | Insertion is rejected | Pass |
| ED-07 | Empty insertion | Insert empty text | Insertion is rejected | Pass |
| ED-08 | Line-break validation | Insert a line break using single-line insertion | Request is rejected | Pass |
| ED-09 | Line validation | Insert using an invalid line number | Request is rejected | Pass |
| ED-10 | Character deletion | Delete a character | Selected character is removed | Pass |
| ED-11 | Undo character deletion | Undo character deletion | Original character is restored | Pass |
| ED-12 | Line insertion | Insert a new line | Line is added correctly | Pass |
| ED-13 | Line deletion | Delete an existing line | Selected line is removed | Pass |
| ED-14 | Undo line deletion | Undo line deletion | Deleted line is restored | Pass |
| ED-15 | Line-range deletion | Delete a range of lines | Selected lines are removed | Pass |
| ED-16 | Undo line-range deletion | Undo deletion of multiple lines | All deleted lines are restored | Pass |
| ED-17 | Text replacement | Replace a specified text range | Replacement is applied correctly | Pass |
| ED-18 | Undo replacement | Undo a text replacement | Original text is restored | Pass |

| | | | | |
|---|---|---|---|---|
| ED-19 | Empty undo history | Undo without previous operations | Operation returns false | Pass |
| SR-01 | Case-sensitive search | Search for lowercase "cat" | Three matches are found | Pass |
| SR-02 | Match coordinates | Check returned match positions | Lines are one-based and columns are zero-based | Pass |
| SR-03 | Case-insensitive search | Search without case matching | Four matches are found | Pass |
| SR-04 | Whole-word search | Search for complete occurrences of "cat" | Substring inside "concatenate" is excluded | Pass |
| SR-05 | Empty search | Search using an empty pattern | Request is rejected | Pass |
| SR-06 | Missing pattern | Search for text that does not exist | NotFound status is returned | Pass |
| RP-01 | Replace First | Replace the first occurrence | Only one occurrence changes | Pass |
| RP-02 | Undo Replace First | Undo the first replacement | Original text is restored | Pass |
| RP-03 | Replace All | Replace all matching substrings | All intended occurrences change | Pass |
| RP-04 | Undo Replace All | Undo a Replace All operation | All replacements are reversed in one undo operation | Pass |
| RP-05 | Empty replacement pattern | Replace using an empty search pattern | Request is rejected | Pass |
| RP-06 | Replacement validation | Use replacement text containing a newline | Request is rejected | Pass |
| UTF-01 | UTF-8 insertion | Insert inside a multi-byte character | Invalid insertion position is rejected | Pass |
| UTF-02 | UTF-8 deletion | Delete a complete UTF-8 character | Entire character is removed | Pass |
| UTF-03 | UTF-8 undo | Undo UTF-8 character deletion | Original character is restored | Pass |
| US-01 | Undo stack | Push the first record | Record is stored | Pass |
| US-02 | Undo stack | Push the second record | Second record is stored | Pass |
| US-03 | Stack capacity | Fill the undo stack | Stack reports that it is full | Pass |
| US-04 | Record eviction | Push another record when full | Oldest record is evicted | Pass |
| US-05 | Record retrieval | Pop from a non-empty stack | Newest record is returned first | Pass |
| US-06 | Record ordering | Pop the next record | Next newest record is returned | Pass |
| US-07 | Empty stack | Pop from an empty stack | Operation is rejected | Pass |
| IV-01 | Menu validation | Enter a valid menu choice | Choice is accepted | Pass |

| | | | | |
|---|---|---|---|---|
| IV-02 | Menu validation | Enter non-numeric input | Input is rejected | Pass |
| IV-03 | Menu validation | Enter an out-of-range menu choice | Input is rejected | Pass |
| IV-04 | Line validation | Enter a valid line number | Line number is accepted | Pass |
| IV-05 | Line validation | Enter line number zero | Line number is rejected | Pass |
| IV-06 | Position validation | Enter a position at the end of a line | Position is accepted | Pass |
| IV-07 | Position validation | Enter a position beyond the line end | Position is rejected | Pass |
| IV-08 | Search validation | Check an empty search string | Empty search input is detected | Pass |

The results in Table 8.1 correspond to the actual test identifiers and PASS messages displayed by the automated test driver.

## 8.5 Boundary and Invalid-Input Testing

Boundary and invalid-input testing was performed as part of the automated test suite. The purpose was to ensure that the application handles invalid parameters safely and does not perform unintended modifications.

The following boundary conditions were covered:

- Creating and accessing an empty document.
- Inserting text at the end of a line.
- Rejecting insertion beyond the end of a line.
- Rejecting empty text and text containing line breaks in single-line insertion.
- Rejecting invalid line numbers, including line zero.
- Rejecting empty search patterns.
- Rejecting replacement patterns that are empty.
- Rejecting replacement text containing newline characters.
- Returning NotFound when a search pattern does not exist.
- Rejecting undo operations when the history stack is empty.
- Ensuring that insertion inside a multi-byte UTF-8 character is rejected.
- Verifying that complete UTF-8 characters can be deleted and restored.
- Checking the behaviour of the undo stack when its capacity is reached.

**Table 8.2: Boundary and Invalid-Input Test Summary**

| Test Area | Boundary or Invalid Condition | Expected Behaviour | Status |
|---|---|---|---|
| Document | Empty document creation | One empty line is maintained | Pass |
| Text insertion | Position equal to line length | Insertion is accepted | Pass |
| Text insertion | Position beyond line length | Insertion is rejected | Pass |
| Text insertion | Empty text | Insertion is rejected | Pass |
| Text insertion | Text containing newline | Insertion is rejected | Pass |

| | | | |
|---|---|---|---|
| Line validation | Line number zero | Request is rejected | Pass |
| Search | Empty search pattern | Request is rejected | Pass |
| Search | Pattern not found | NotFound is returned | Pass |
| Replacement | Empty search pattern | Request is rejected | Pass |
| Replacement | Replacement contains newline | Request is rejected | Pass |
| UTF-8 | Position inside a multi-byte character | Insertion is rejected | Pass |
| Undo | Empty undo history | Undo returns false | Pass |
| Undo stack | Capacity exceeded | Oldest record is removed | Pass |
| Input validation | Non-numeric menu input | Input is rejected | Pass |
| Input validation | Menu selection outside the valid range | Input is rejected | Pass |

These tests confirm that the backend performs validation before applying modifications and handles invalid requests without unintended changes.

## 8.6 Integration Testing

Integration testing was performed to verify that the backend components work together correctly when several operations are executed on the same document.

The following interactions were covered by the automated tests:

- **Document management and editing:** TextDocument was used with EditOperations to insert and delete text while maintaining the linked-list structure and character buffers.
- **Editing and undo:** Text insertion, deletion, and replacement operations were combined with UndoStack to verify that previous document states could be restored.
- **Searching and replacement:** TextSearch was used to locate matching text, while TextReplace performed the required replacement through EditOperations.
- **Replace All and undo:** Multiple occurrences were replaced and then restored using one undo operation.
- **UTF-8 and editing:** Multi-byte character positions were validated before insertion and deletion operations were performed.
- **Undo stack management:** Multiple records were pushed and retrieved to verify capacity, eviction, and last-in-first-out ordering.
- **Input validation:** Valid and invalid menu choices, line numbers, and character positions were passed through the validation functions.

The tests also checked the interaction between document lines and their character buffers. Operations involving different lines and line lengths were used to verify that editing one line did not unintentionally modify another.

The automated test driver concentrated on backend functionality. The console application and Qt interface use these same backend components, but the 55 test results should not be interpreted as 55 separate GUI or file-management tests.

## 8.7 Test Results and Discussion

The automated test driver completed successfully with the following results:

| Result | Number |
|---|---|
| Total tests executed | 55 |
| Tests passed | 55 |
| Tests failed | 0 |
| Success rate | 100% |

All 55 automated test cases passed successfully. No failed tests were reported during execution.

The results demonstrate the correct behaviour of the tested document-management, editing, searching, replacement, UTF-8 handling, undo stack, and input-validation operations.

The document-management tests confirmed that a new document contains one empty line, that lines can be loaded and retrieved correctly, and that a trailing newline creates a final empty line.

The editing tests confirmed that text and lines can be inserted and deleted, that invalid insertion requests are rejected, and that undo restores previous document contents. The search tests confirmed that case-sensitive, case-insensitive, and whole-word searches return the expected matches. The replacement tests verified that Replace First modifies one occurrence and Replace All modifies all intended occurrences using a single undo operation.

The UTF-8 tests confirmed that the application rejects insertion positions inside multi-byte characters while allowing complete character deletion and restoration. The undo stack tests also verified record ordering, capacity handling, and eviction of the oldest record.

The input-validation tests demonstrated that valid menu choices and positions are accepted, while non-numeric input, out-of-range choices, invalid line numbers, and positions beyond the end of a line are rejected.

Despite the successful results, the testing process has some limitations. The automated test driver primarily verifies backend functionality. It does not provide comprehensive testing of the Qt interface's visual appearance, responsiveness, or event handling. File operations and the complete graphical workflow should also be evaluated separately.

Overall, the results indicate that all 55 tested cases passed and that no functional failures were detected in the tested backend components.

# 9. TEST EXECUTION SCREENSHOTS AND OUTPUT EXAMPLES

This chapter presents screenshots demonstrating the successful execution of the automated backend tests and the essential functionality of the Qt graphical user interface (GUI).

The chapter is divided into two main parts. The first part presents the automated backend test results, while the second part demonstrates the implementation of the required text editor features through the graphical interface.

## 9.1 Backend Automated Test Execution

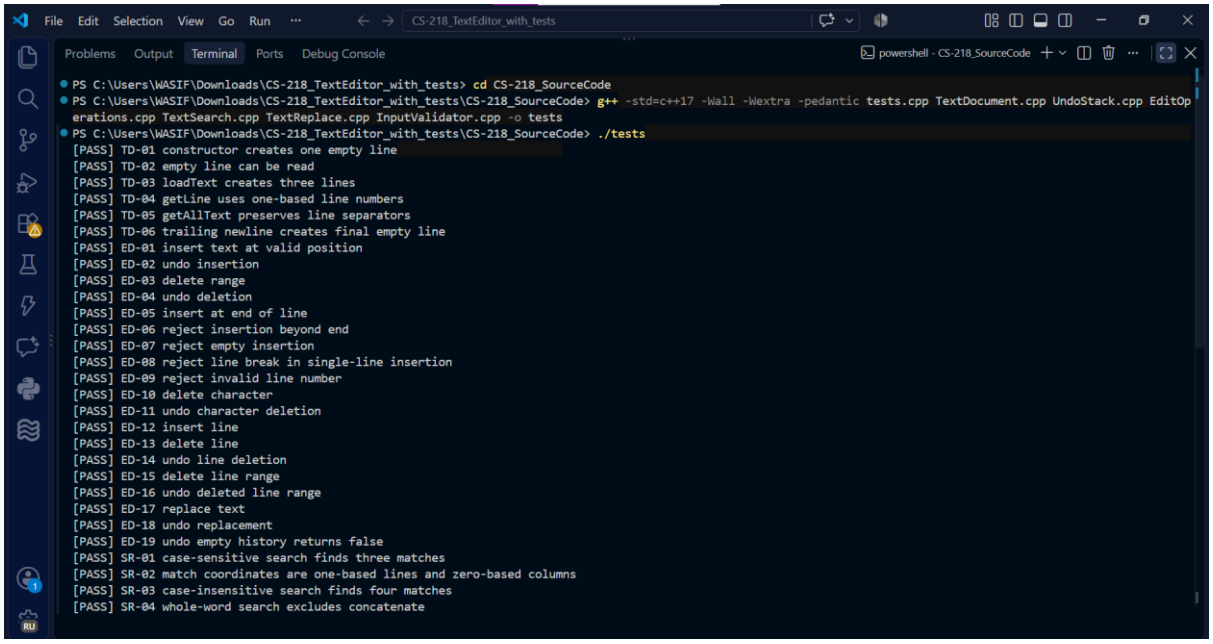



*Figure 9.1: Backend Automated Test Execution – Document, Editing, and Search Tests*

Figure 9.1 shows the first part of the automated backend test execution. It contains the results of document-management, editing, undo, and initial search tests.

The document-management tests (TD-01 to TD-06) verify document construction, empty-line access, text loading, line numbering, preservation of line separators, and handling of trailing newlines.

The editing tests (ED-01 to ED-19) verify text insertion, deletion, line operations, replacement, and undo functionality. They also check invalid insertion requests and the behaviour of undo when the history is empty.

The search tests (SR-01 to SR-04) verify case-sensitive matching, match coordinates, case-insensitive searching, and whole-word searching.

All tests displayed in this screenshot report PASS, indicating that the tested backend operations produced their expected results.

## 9.2 Backend Automated Test Execution

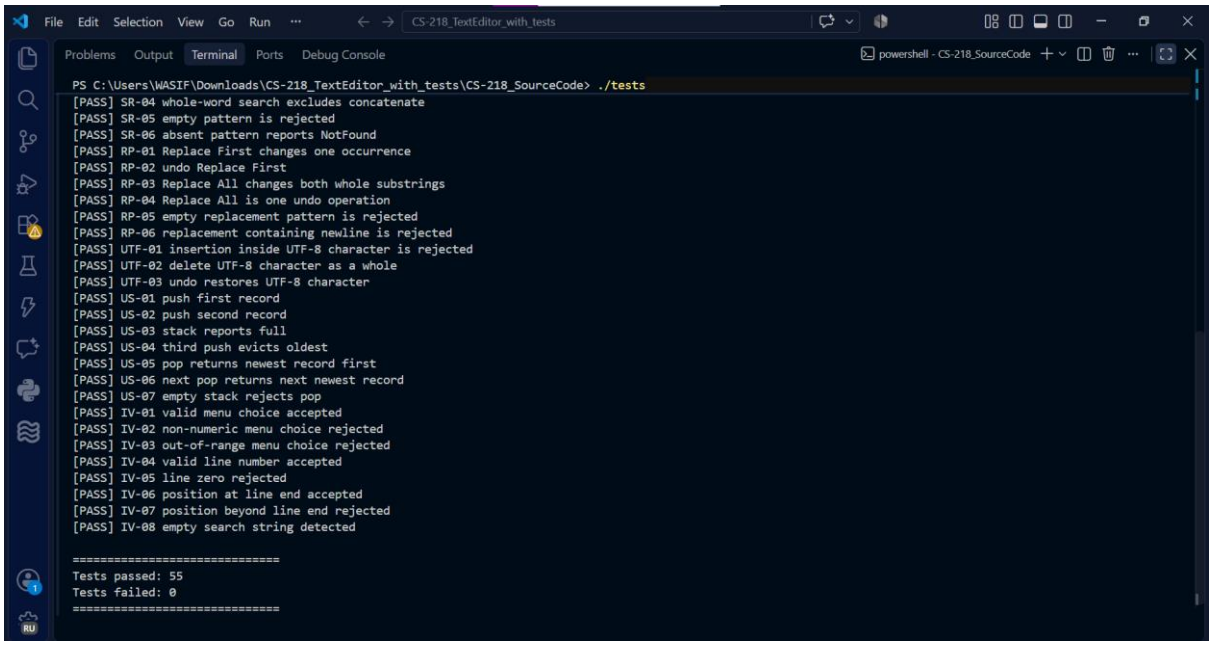



*Figure 9.2: Backend Automated Test Execution – Replacement, UTF-8, Undo Stack, and Validation Tests*

Figure 9.2 presents the remaining automated backend test results.

The search tests (SR-05 to SR-06) verify the rejection of empty search patterns and the NotFound response when a pattern does not exist.

The replacement tests (RP-01 to RP-06) verify Replace First, Replace All, undo functionality, and rejection of invalid replacement requests.

The UTF-8 tests (UTF-01 to UTF-03) verify that insertion inside a multi-byte character is rejected, complete UTF-8 characters can be deleted, and undo restores deleted characters.

The undo stack tests (US-01 to US-07) verify record insertion, capacity, eviction of the oldest record, retrieval order, and empty-stack behaviour.

The input validation tests (IV-01 to IV-08) verify valid and invalid menu choices, line numbers, character positions, and empty search input.

The final summary displayed by the test driver reports:

- Tests passed: 55
- Tests failed: 0

These results confirm that all 55 automated backend test cases completed successfully.

## 9.3 Overall Backend Testing Result

The two screenshots together provide evidence of the complete automated backend test execution.

The test driver executed 55 test cases covering document management, editing, searching, replacement, UTF-8 character handling, undo stack operations, and input validation.

All 55 test cases passed, and no failures were reported.

**Final Result: 55 out of 55 tests passed (100% success rate).**

These results demonstrate that the tested backend components performed according to their expected functionality.

# 9.4 Frontend (Qt GUI) – Essential Feature Demonstration

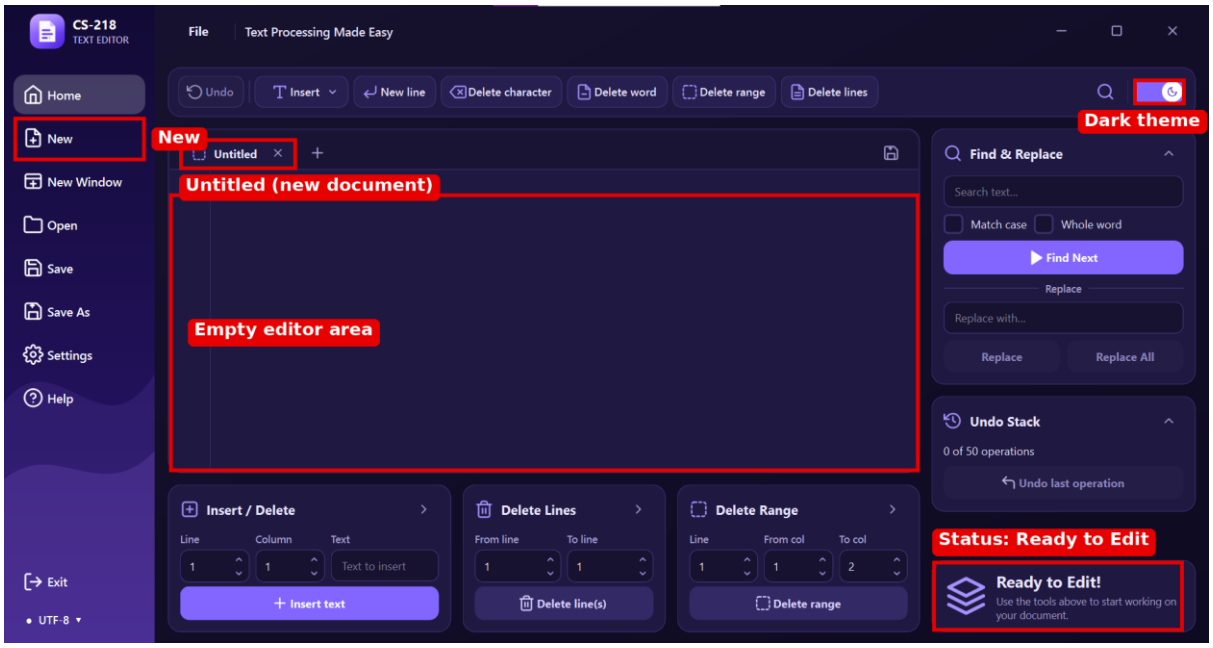
This section presents screenshots of the Qt graphical user interface to demonstrate the essential features of the text editor.

The GUI provides an interactive environment through which users can create, open, edit, save, search, replace, and manage text documents. It uses the same backend components as the console application, ensuring consistent document-processing functionality.

The following subsections demonstrate the essential features required for the text editor.

## 9.4.1 File Operations

### 9.4.1.1 Create a New Document



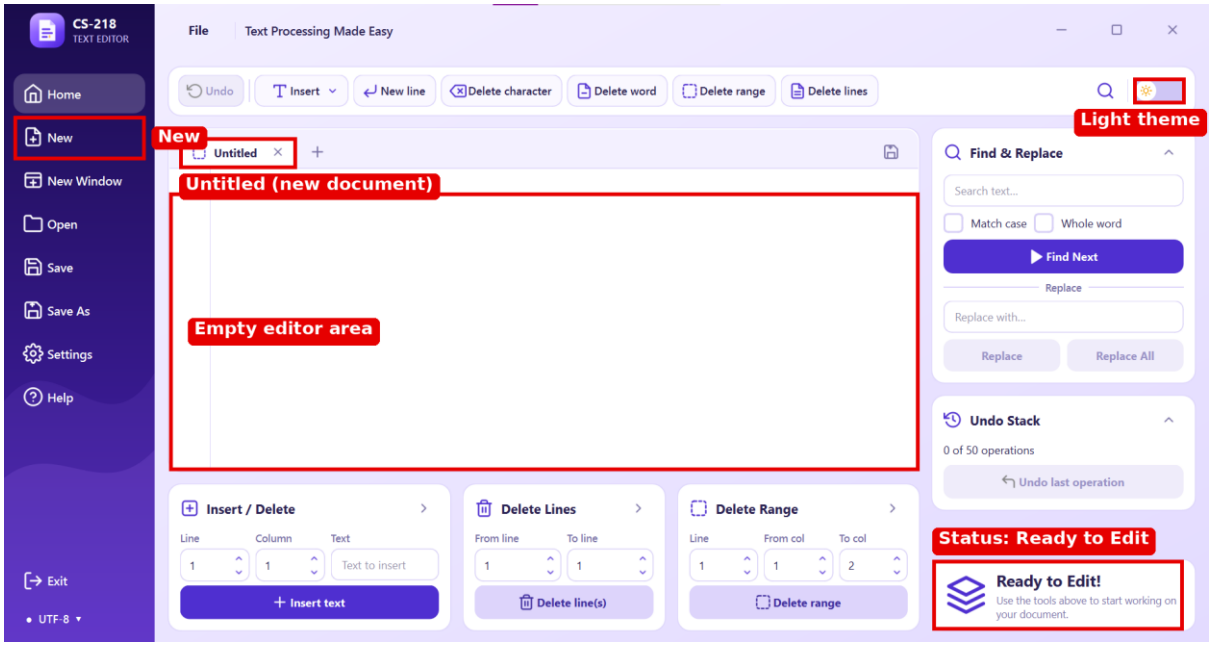
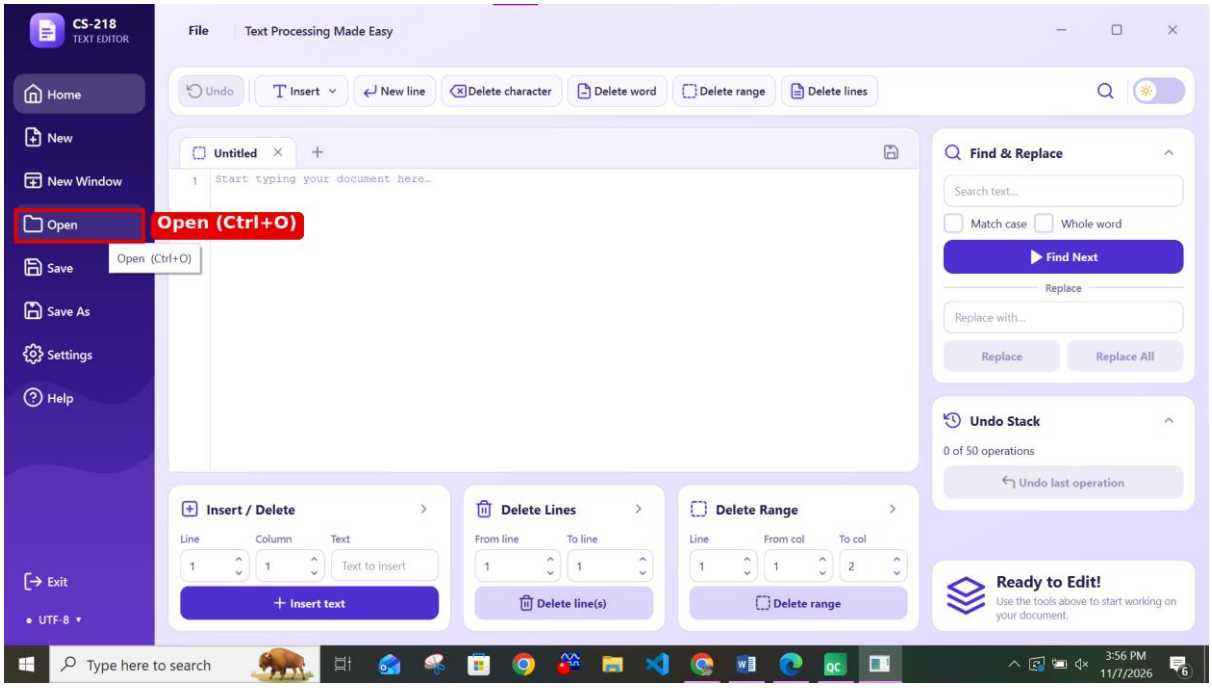



*Figure 9.3: Creating a New Document*

Figure 9.3 demonstrates the creation of a new document through the Qt graphical interface. The New option allows users to start a fresh document. The editor displays an empty document ready for text entry.

### 9.4.1.2 Open an Existing Text File



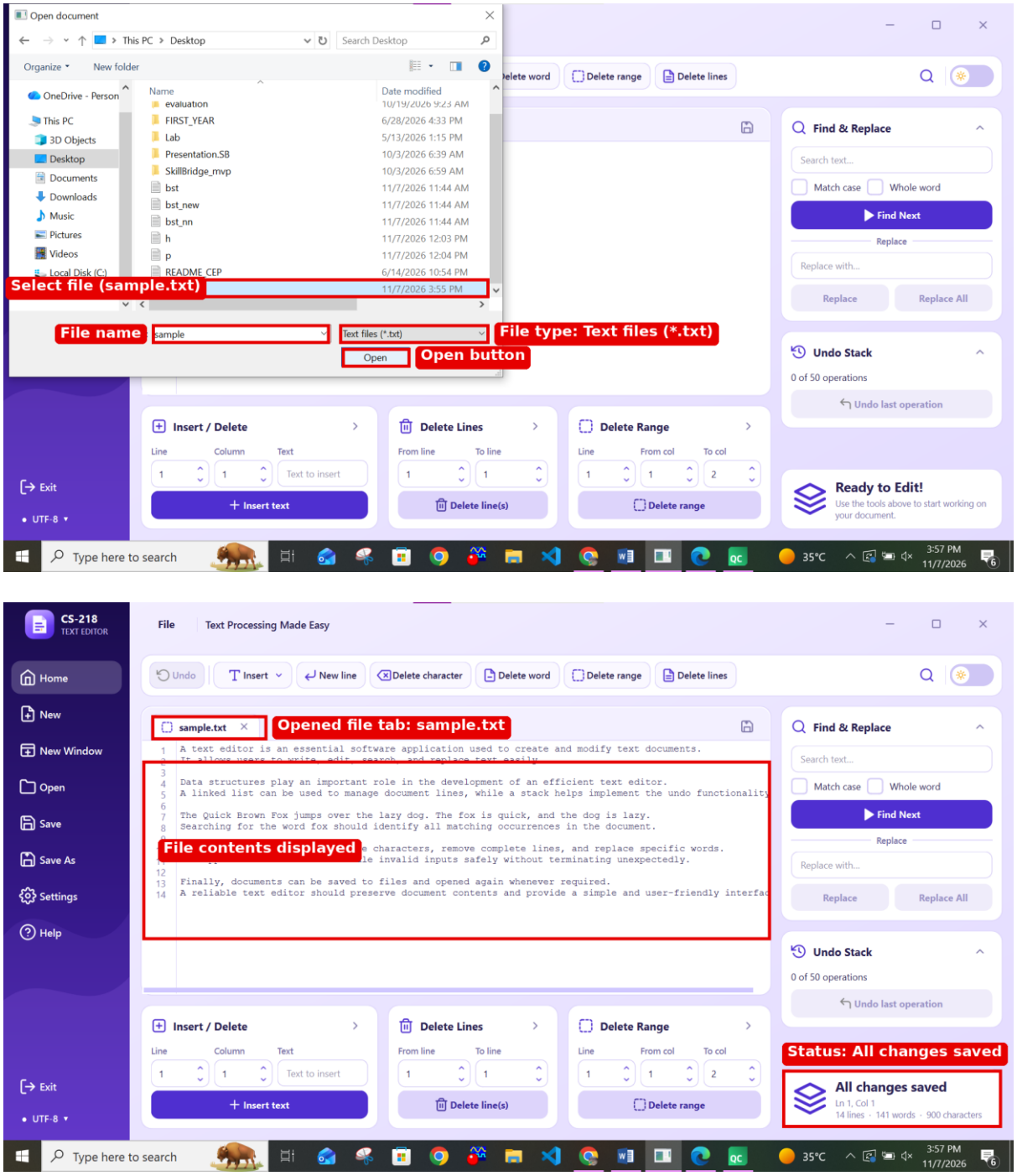



*Figure 9.4: Opening an Existing Text File*

Figure 9.4 demonstrates how users can open an existing text file using the Open option. After selecting a valid file, its contents are loaded and displayed in the main editing area.

### 9.4.1.3 Save the Current Document

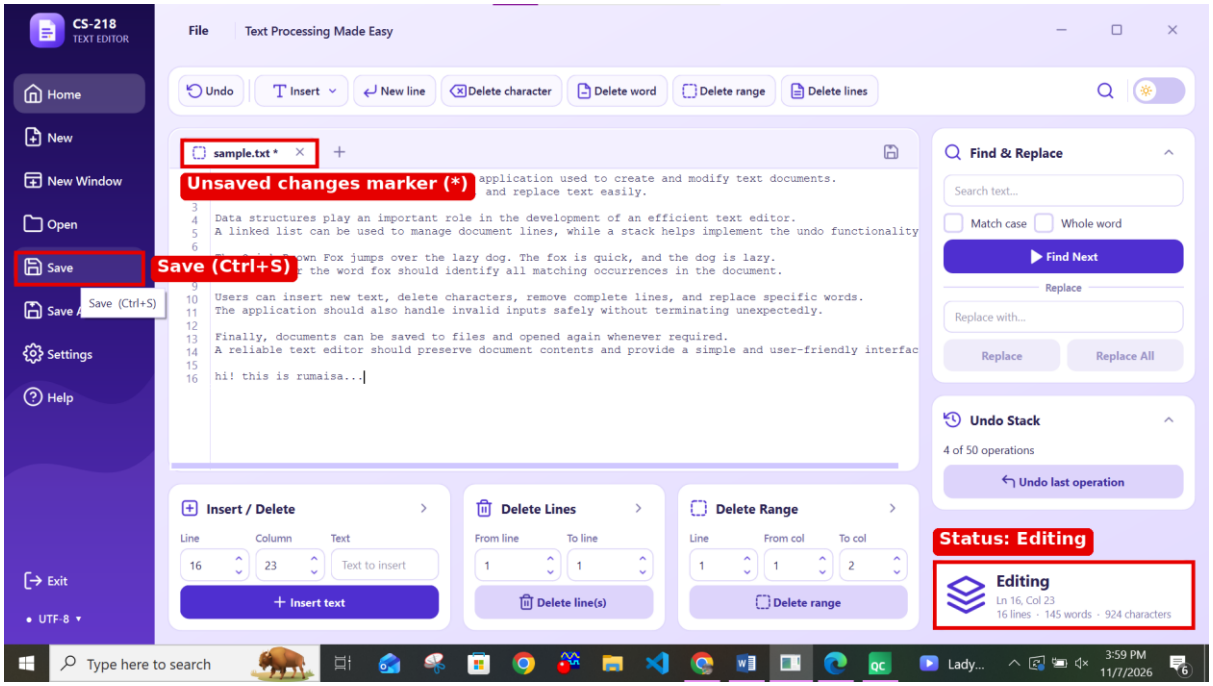
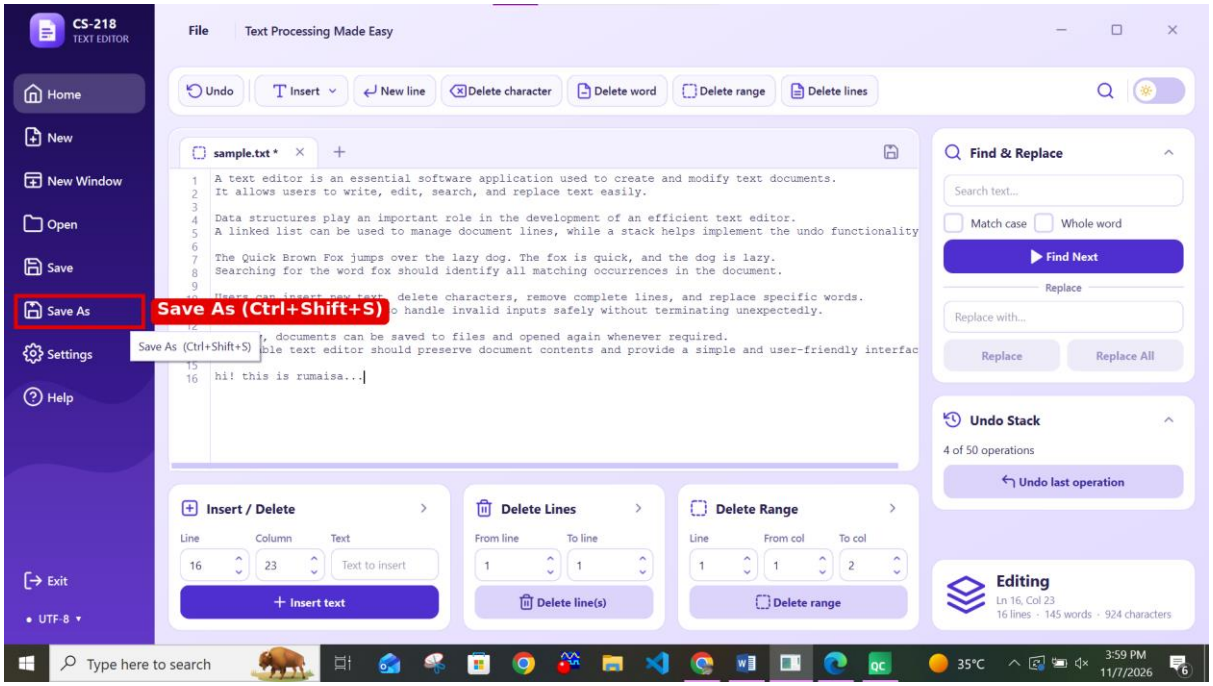



*Figure 9.5: Saving the Current Document*

Figure 9.5 demonstrates the Save functionality. Users can save modifications to the currently associated file, preserving the latest document contents.

### 9.4.1.4 Save the Document Under a New Filename



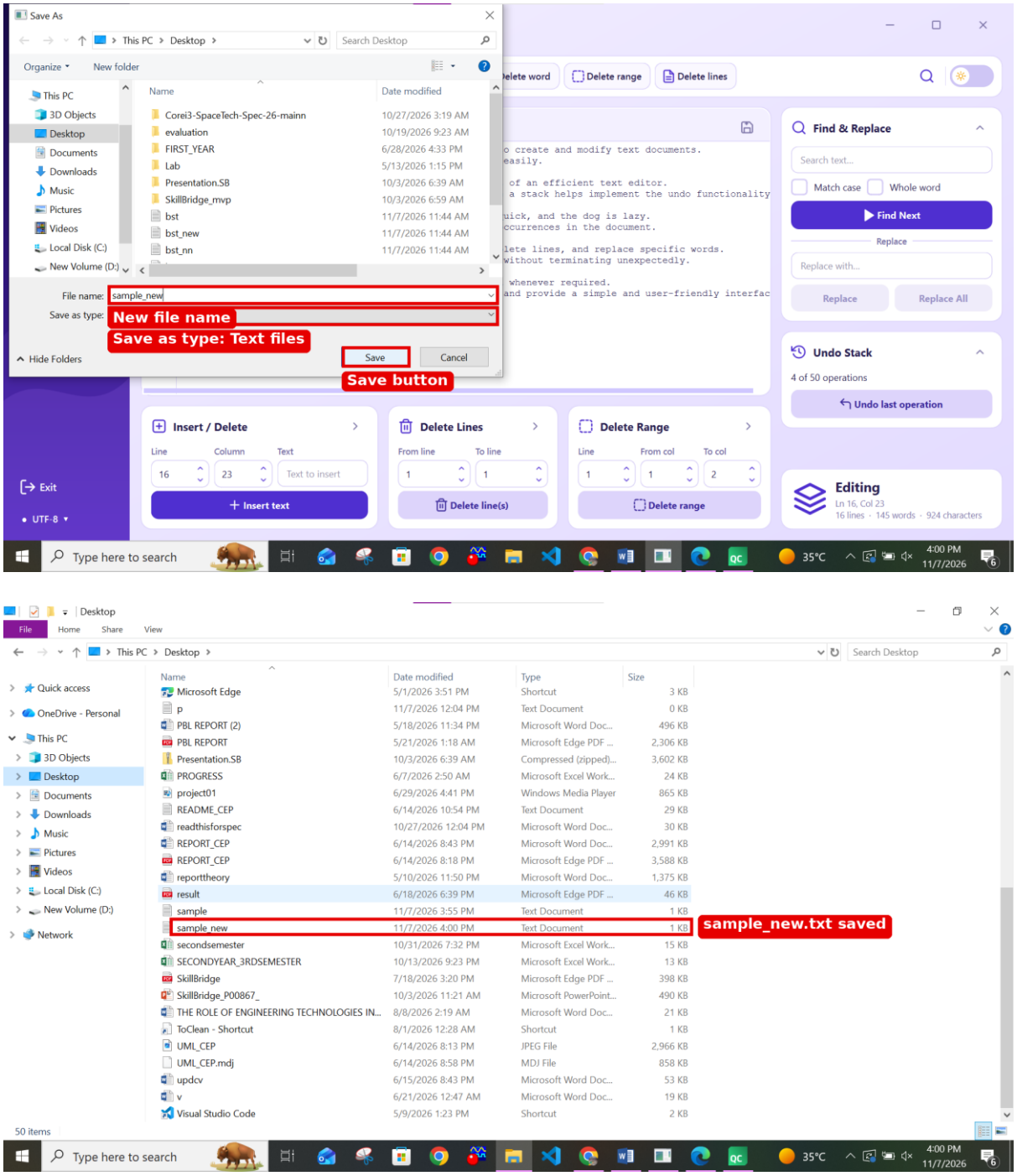



*Figure 9.6: Save As Functionality*

Figure 9.6 demonstrates the Save As functionality. It allows users to save the current document under a new filename or at a different file location.

### 9.4.1.5 Exit the Editor

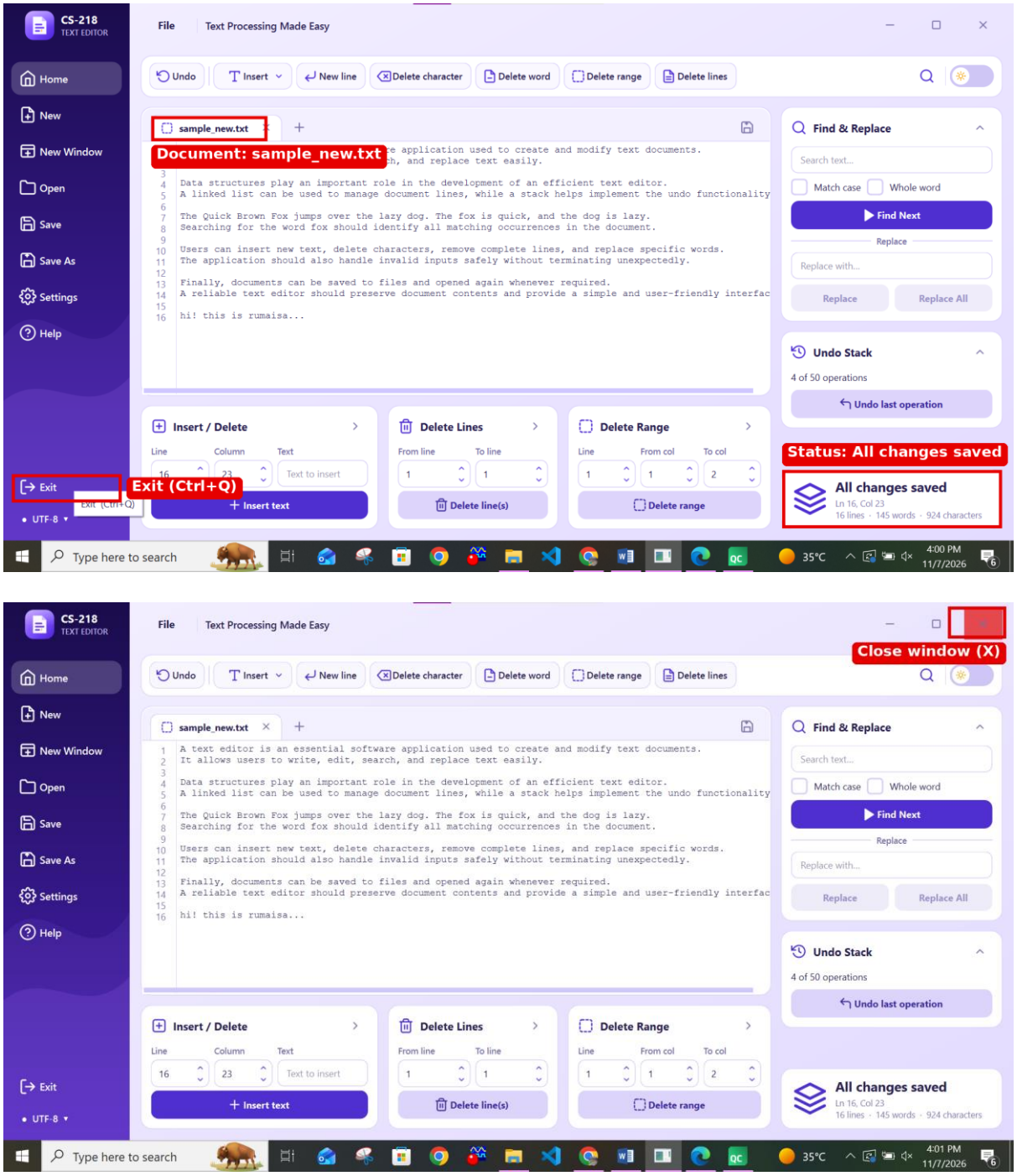



*Figure 9.7: Exiting the Text Editor*

Figure 9.7 demonstrates the application exit functionality. Users can close the graphical editor after completing their work. If unsaved-change confirmation is implemented, the screenshot can also demonstrate that confirmation.

## 9.4.2 Text Display

### 9.4.2.1 Display Document Contents

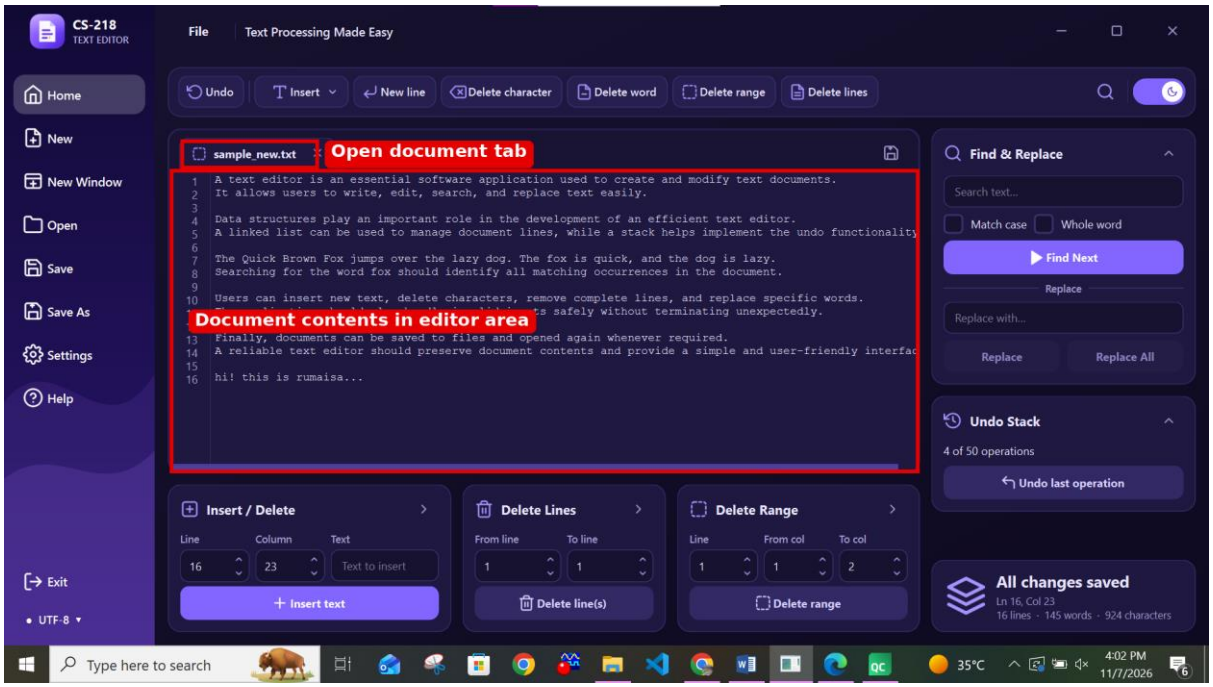



*Figure 9.8: Displaying Document Contents*

Figure 9.8 demonstrates how the graphical interface displays the contents of a text document in the main editing area. Users can view and edit the document directly.

### 9.4.2.2 Display Document Line by Line

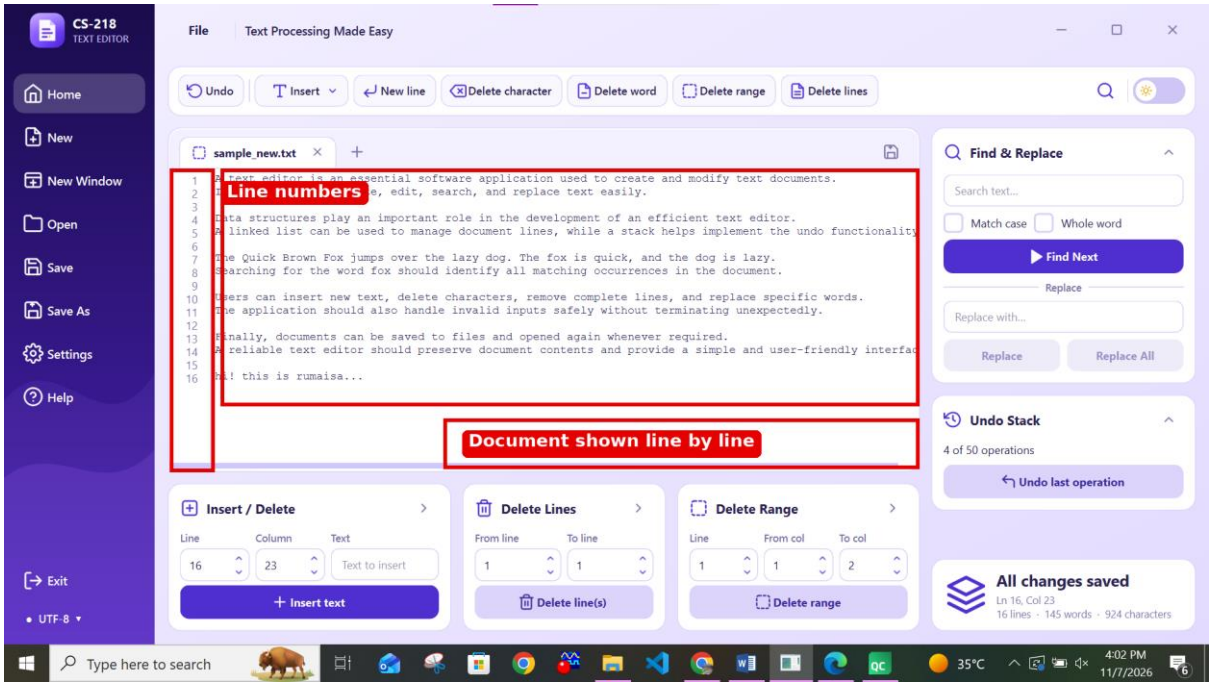



*Figure 9.9: Displaying Document Lines*

Figure 9.9 demonstrates the line-by-line display functionality. The editor displays document contents as individual lines, while line numbers help users identify and navigate different sections of the document.

### 9.4.2.3 Display Document Information

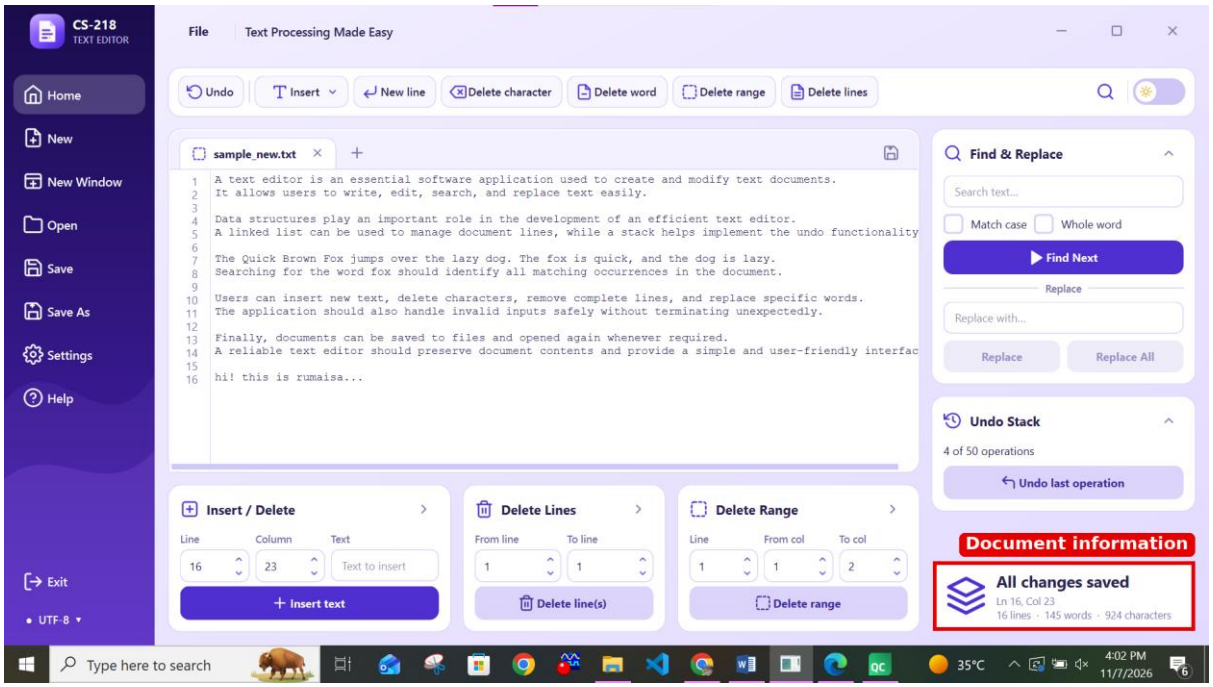
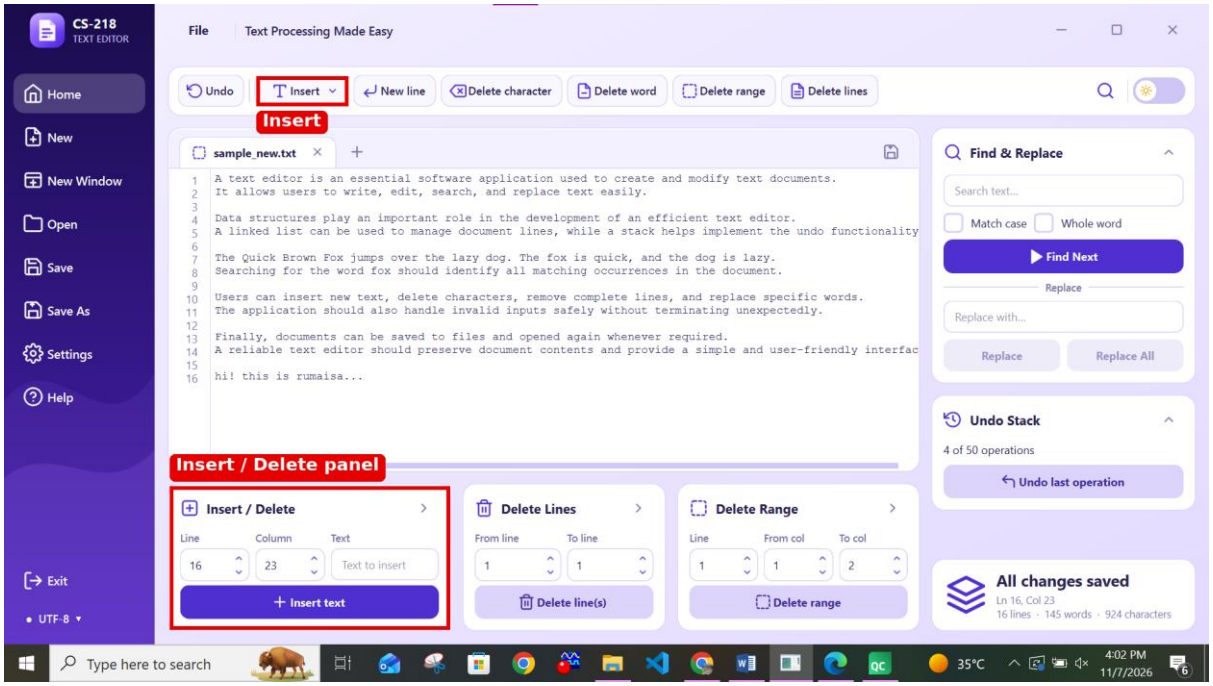


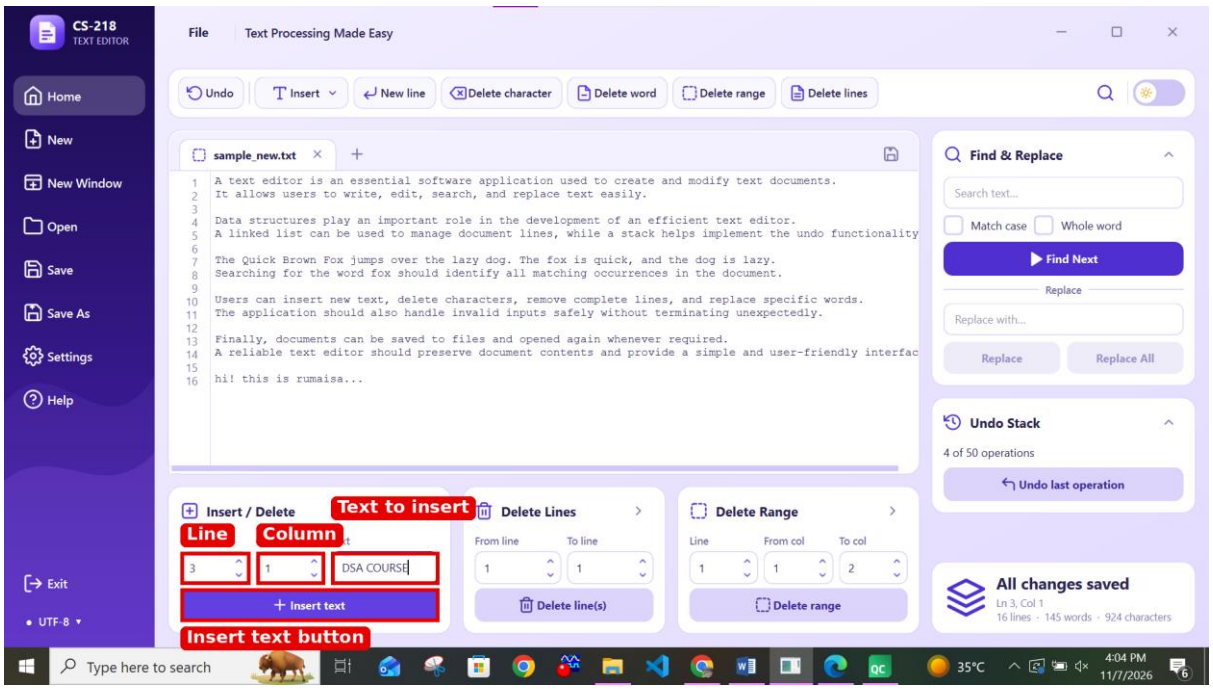
*Figure 9.10: Displaying Document Information*

Figure 9.10 demonstrates the display of useful document information, such as the total number of lines and characters. This information helps users understand the size and structure of the current document.

## 9.4.3 Text Insertion and Deletion

### 9.4.3.1 Insert Text at a Specified Position



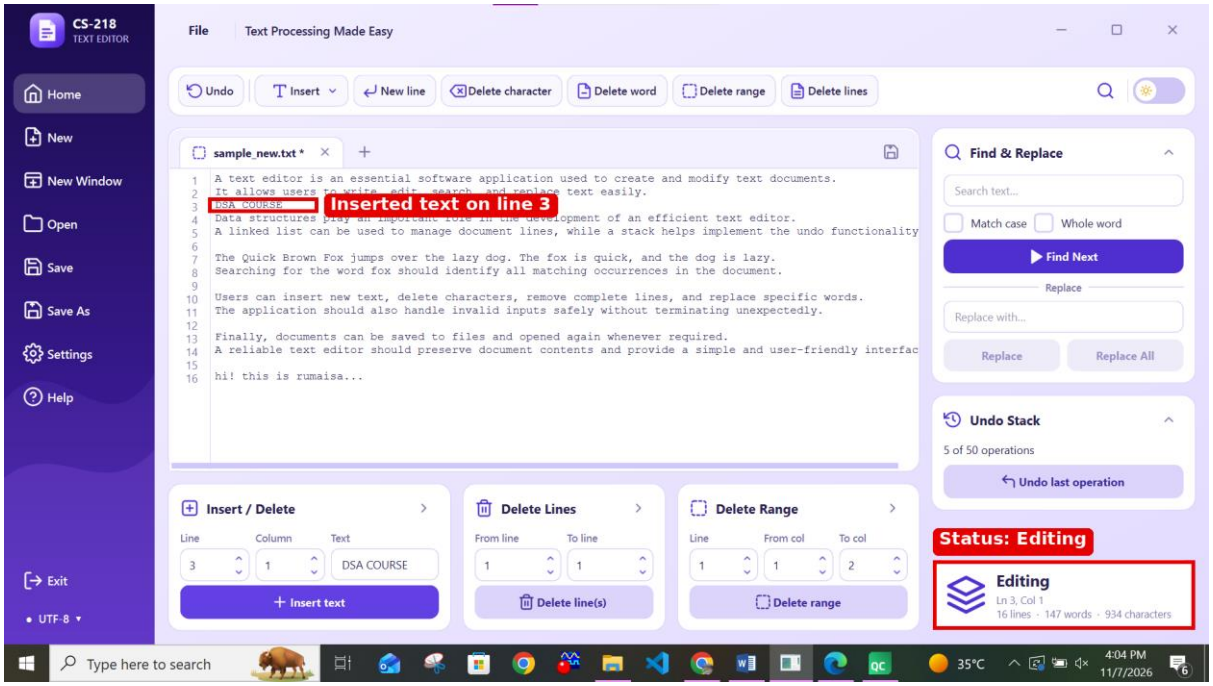
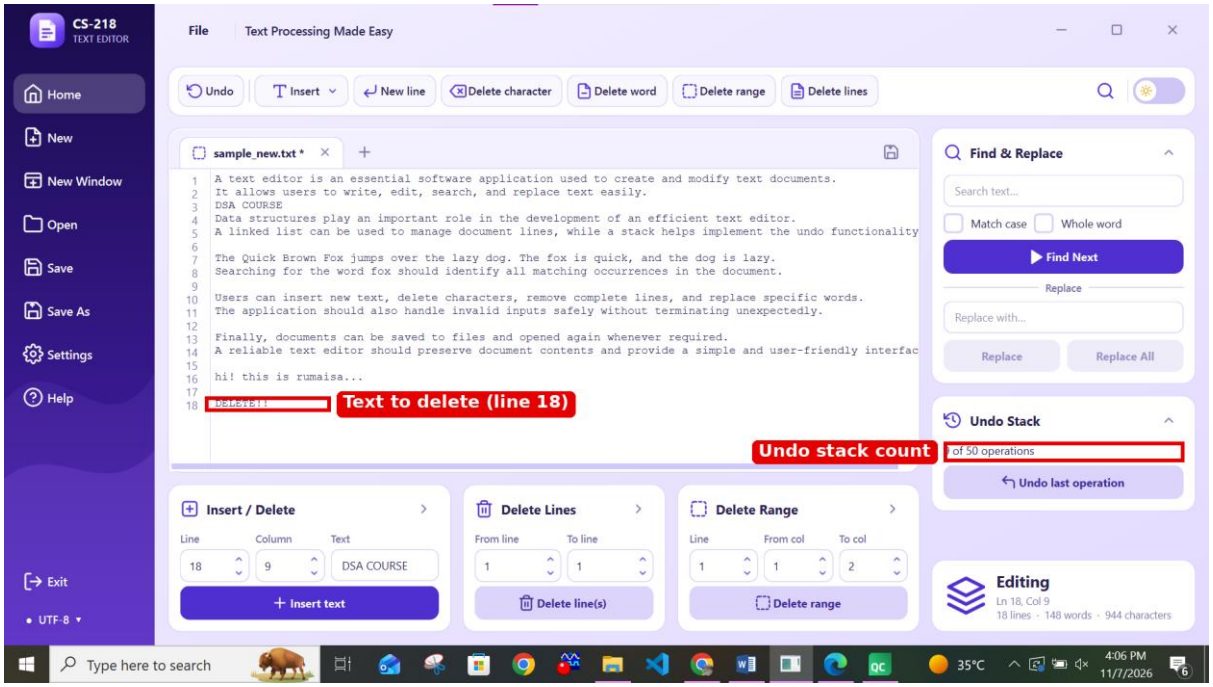
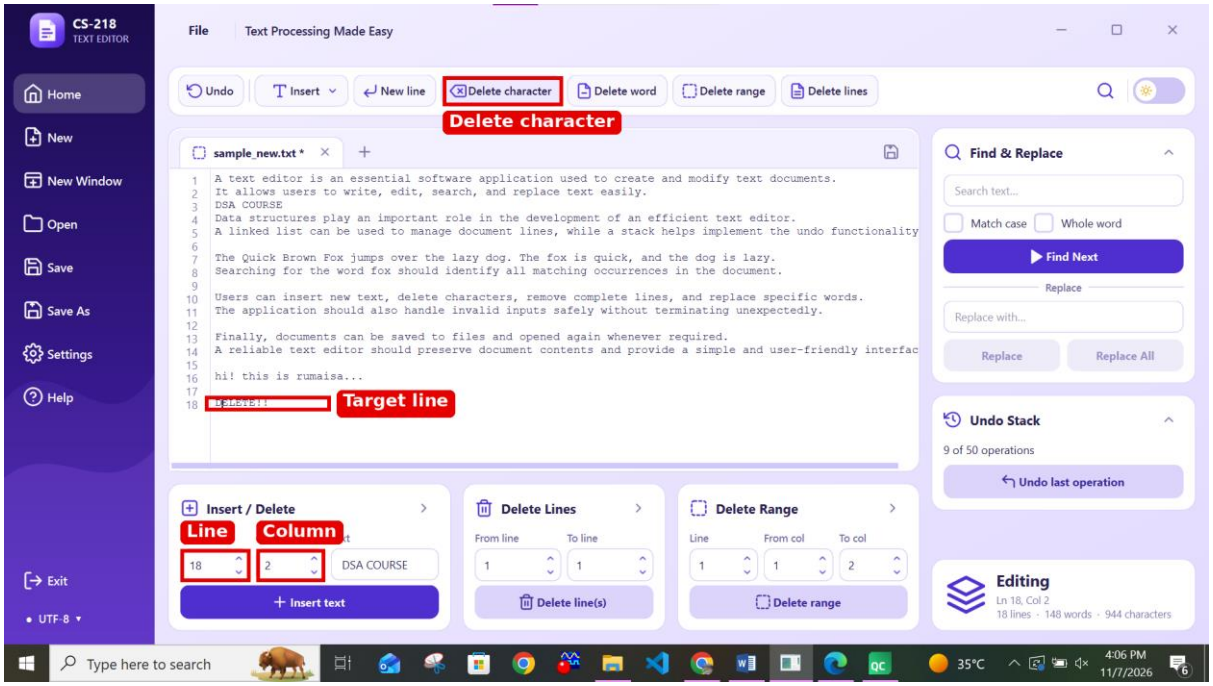


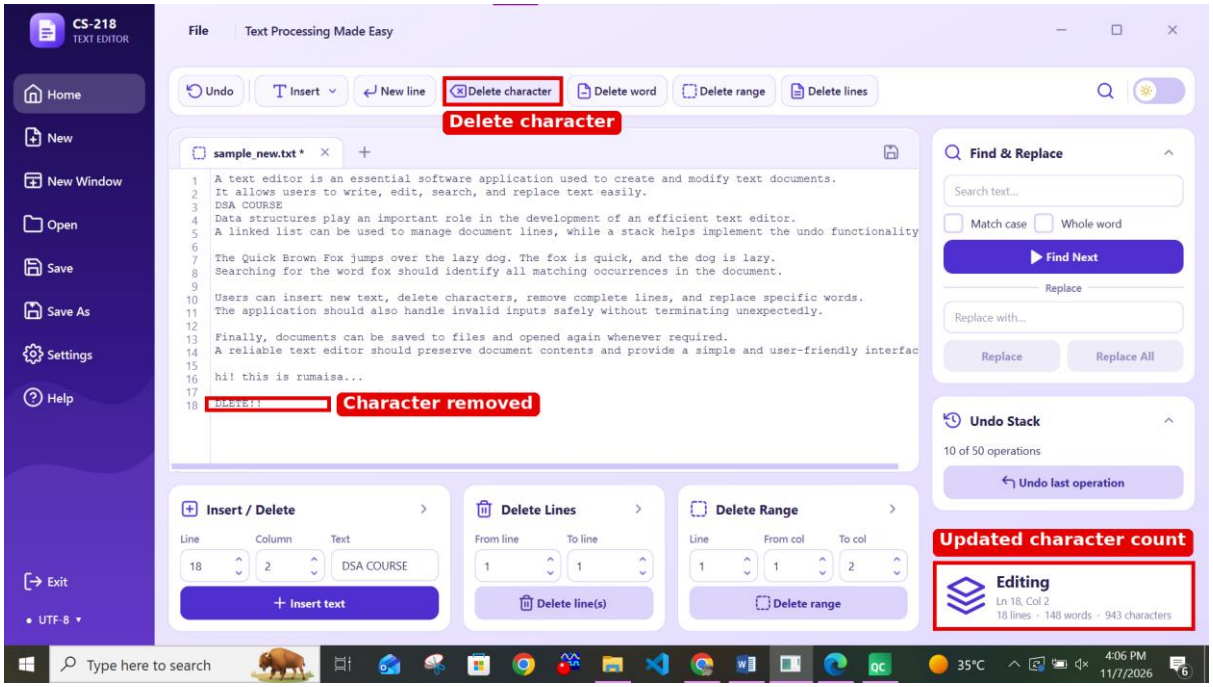

*Figure 9.11: Text Insertion at a Specified Position*

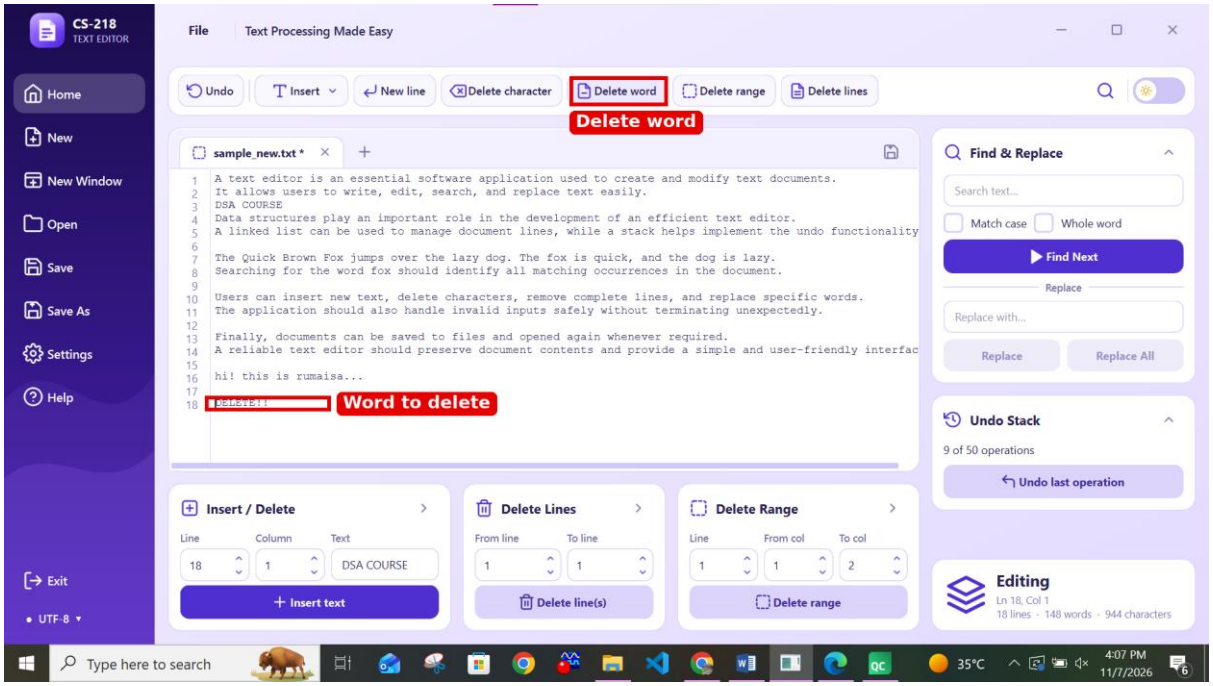
Figure 9.11 demonstrates the text insertion functionality. Users can place the cursor at the required position and enter additional text. The inserted text appears at the selected position without replacing the remaining document contents.

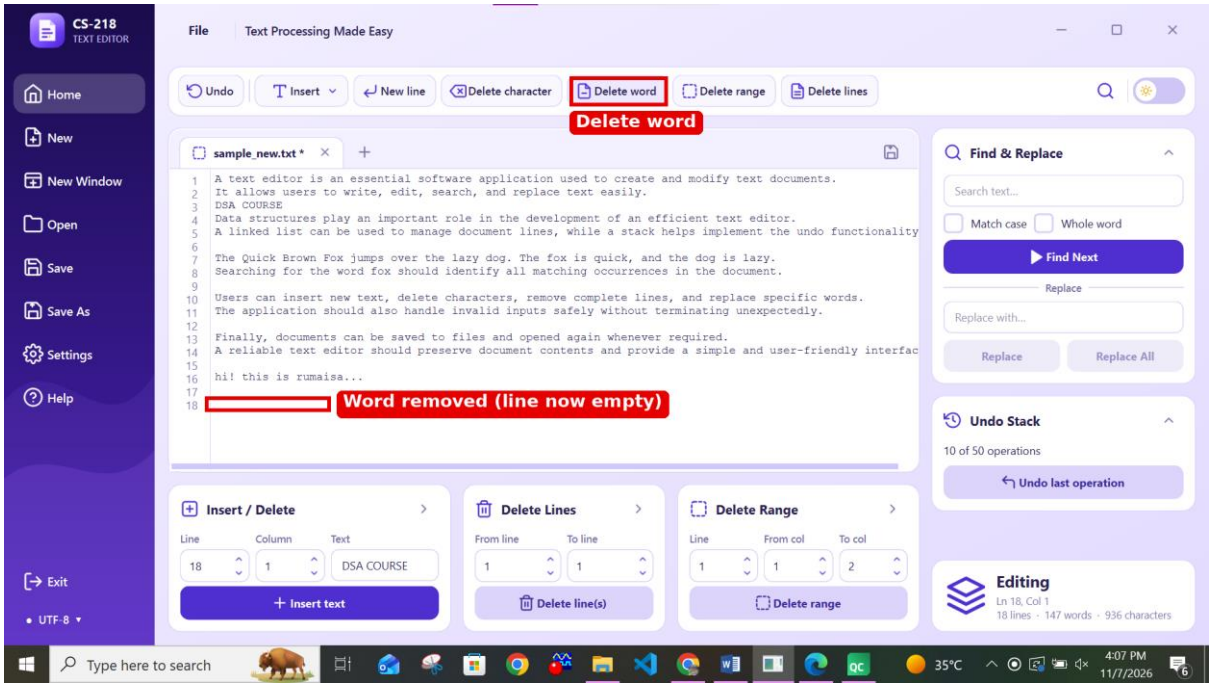
### 9.4.3.2 Delete a Character, Word, or Text Range




CS-218
TEXT EDITOR
File
Text Processing Made Easy
Home
New
New Window
Open
Save
Save As
Settings
Help
Exit
UTF-8
Undo
Insert
New line
Delete character
Delete word
Delete range
Delete lines
Delete character
sample_new.txt *
A text editor is an essential software application used to create and modify text documents.
It allows users to write, edit, search, and replace text easily.
DSA COURSE
Data structures play an important role in the development of an efficient text editor.
A linked list can be used to manage document lines, while a stack helps implement the undo functionality
The Quick Brown Fox jumps over the lazy dog. The fox is quick, and the dog is lazy.
Searching for the word fox should identify all matching occurrences in the document.
Users can insert new text, delete characters, remove complete lines, and replace specific words.
The application should also handle invalid inputs safely without terminating unexpectedly.
Finally, documents can be saved to files and opened again whenever required.
A reliable text editor should preserve document contents and provide a simple and user-friendly interfac
hi! this is rumaisa...
DELETE!!
Target line
Find & Replace
Search text...
Match case
Whole word
Find Next
Replace
Replace with...
Replace
Replace All
Undo Stack
9 of 50 operations
Undo last operation
Insert / Delete
Line
Column
Text
18
2
DSA COURSE
Insert text
Delete Lines
From line
To line
1
1
Delete line(s)
Delete Range
Line
From col
To col
1
1
2
Delete range
Editing
Ln 18, Col 2
18 lines · 148 words · 944 characters
Type here to search
35°C
4:06 PM
11/7/2026


CS-218
TEXT EDITOR
File
Text Processing Made Easy
Home
New
New Window
Open
Save
Save As
Settings
Help
Exit
UTF-8
Undo
Insert
New line
Delete character
Delete word
Delete range
Delete lines
Delete character
sample_new.txt *
A text editor is an essential software application used to create and modify text documents.
It allows users to write, edit, search, and replace text easily.
DSA COURSE
Data structures play an important role in the development of an efficient text editor.
A linked list can be used to manage document lines, while a stack helps implement the undo functionality
The Quick Brown Fox jumps over the lazy dog. The fox is quick, and the dog is lazy.
Searching for the word fox should identify all matching occurrences in the document.
Users can insert new text, delete characters, remove complete lines, and replace specific words.
The application should also handle invalid inputs safely without terminating unexpectedly.
Finally, documents can be saved to files and opened again whenever required.
A reliable text editor should preserve document contents and provide a simple and user-friendly interfac
hi! this is rumaisa...
DELETE!!
Character removed
Find & Replace
Search text...
Match case
Whole word
Find Next
Replace
Replace with...
Replace
Replace All
Undo Stack
10 of 50 operations
Undo last operation
Insert / Delete
Line
Column
Text
18
2
DSA COURSE
Insert text
Delete Lines
From line
To line
1
1
Delete line(s)
Delete Range
Line
From col
To col
1
1
2
Delete range
Updated character count
Editing
Ln 18, Col 2
18 lines · 148 words · 943 characters
Type here to search
35°C
4:06 PM
11/7/2026


CS-218
TEXT EDITOR
File
Text Processing Made Easy
Home
New
New Window
Open
Save
Save As
Settings
Help
Exit
UTF-8
Undo
Insert
New line
Delete character
Delete word
Delete range
Delete lines
Delete word
sample_new.txt *
A text editor is an essential software application used to create and modify text documents.
It allows users to write, edit, search, and replace text easily.
DSA COURSE
Data structures play an important role in the development of an efficient text editor.
A linked list can be used to manage document lines, while a stack helps implement the undo functionality
The Quick Brown Fox jumps over the lazy dog. The fox is quick, and the dog is lazy.
Searching for the word fox should identify all matching occurrences in the document.
Users can insert new text, delete characters, remove complete lines, and replace specific words.
The application should also handle invalid inputs safely without terminating unexpectedly.
Finally, documents can be saved to files and opened again whenever required.
A reliable text editor should preserve document contents and provide a simple and user-friendly interfac
hi! this is rumaisa...
DELETE!!
Word to delete
Find & Replace
Search text...
Match case
Whole word
Find Next
Replace
Replace with...
Replace
Replace All
Undo Stack
9 of 50 operations
Undo last operation
Insert / Delete
Line
Column
Text
18
1
DSA COURSE
Insert text
Delete Lines
From line
To line
1
1
Delete line(s)
Delete Range
Line
From col
To col
1
1
2
Delete range
Editing
Ln 18, Col 1
18 lines · 148 words · 944 characters
Type here to search
35°C
4:07 PM
11/7/2026


CS-218
TEXT EDITOR
File
Text Processing Made Easy
Home
New
New Window
Open
Save
Save As
Settings
Help
Exit
UTF-8
Undo
Insert
New line
Delete character
Delete word
Delete range
Delete lines
Delete word
sample_new.txt *
A text editor is an essential software application used to create and modify text documents.
It allows users to write, edit, search, and replace text easily.
DSA COURSE
Data structures play an important role in the development of an efficient text editor.
A linked list can be used to manage document lines, while a stack helps implement the undo functionality
The Quick Brown Fox jumps over the lazy dog. The fox is quick, and the dog is lazy.
Searching for the word fox should identify all matching occurrences in the document.
Users can insert new text, delete characters, remove complete lines, and replace specific words.
The application should also handle invalid inputs safely without terminating unexpectedly.
Finally, documents can be saved to files and opened again whenever required.
A reliable text editor should preserve document contents and provide a simple and user-friendly interfac
hi! this is rumaisa...
Word removed (line now empty)
Find & Replace
Search text...
Match case
Whole word
Find Next
Replace
Replace with...
Replace
Replace All
Undo Stack
10 of 50 operations
Undo last operation
Insert / Delete
Line
Column
Text
18
1
DSA COURSE
Insert text
Delete Lines
From line
To line
1
1
Delete line(s)
Delete Range
Line
From col
To col
1
1
2
Delete range
Editing
Ln 18, Col 1
18 lines · 147 words · 936 characters
Type here to search
35°C
4:07 PM
11/7/2026

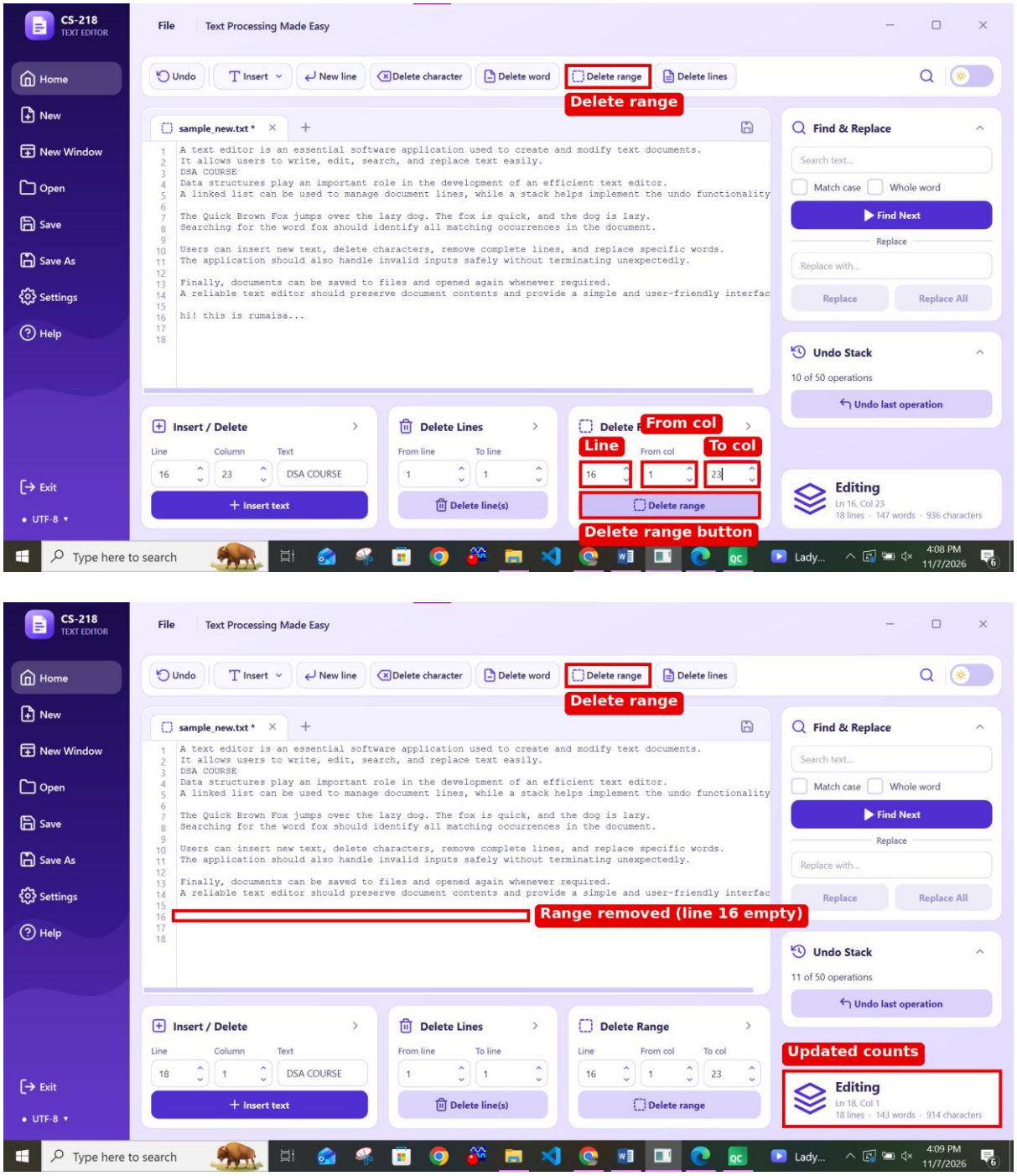
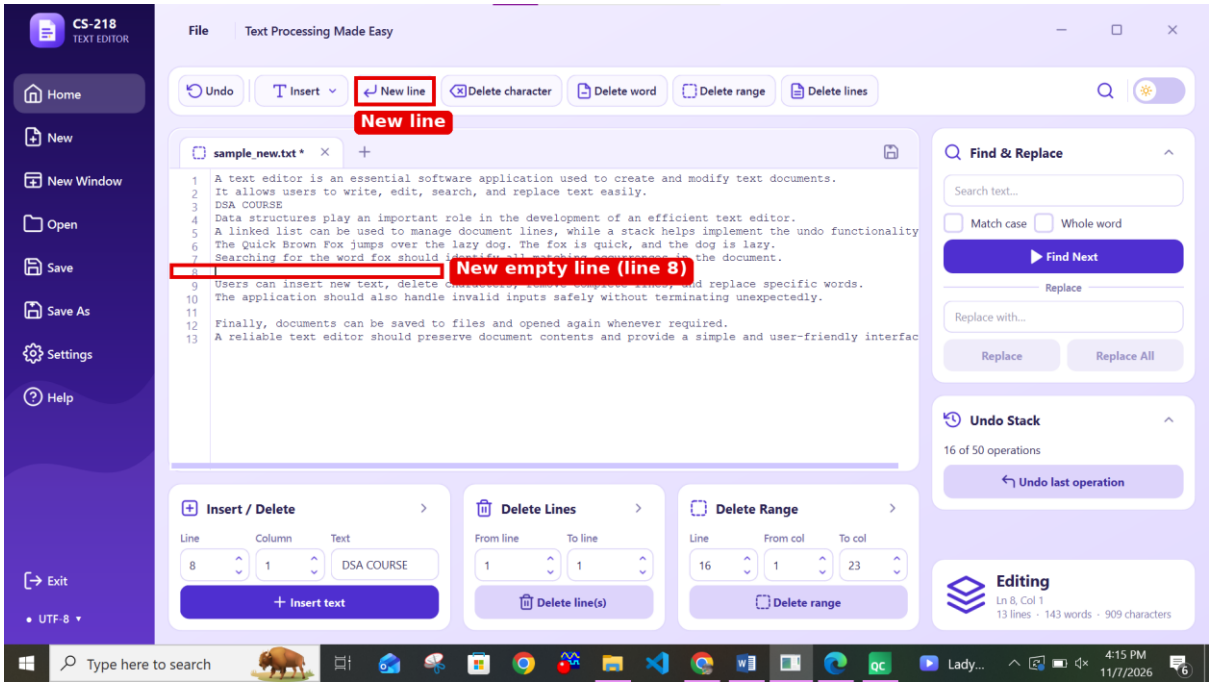
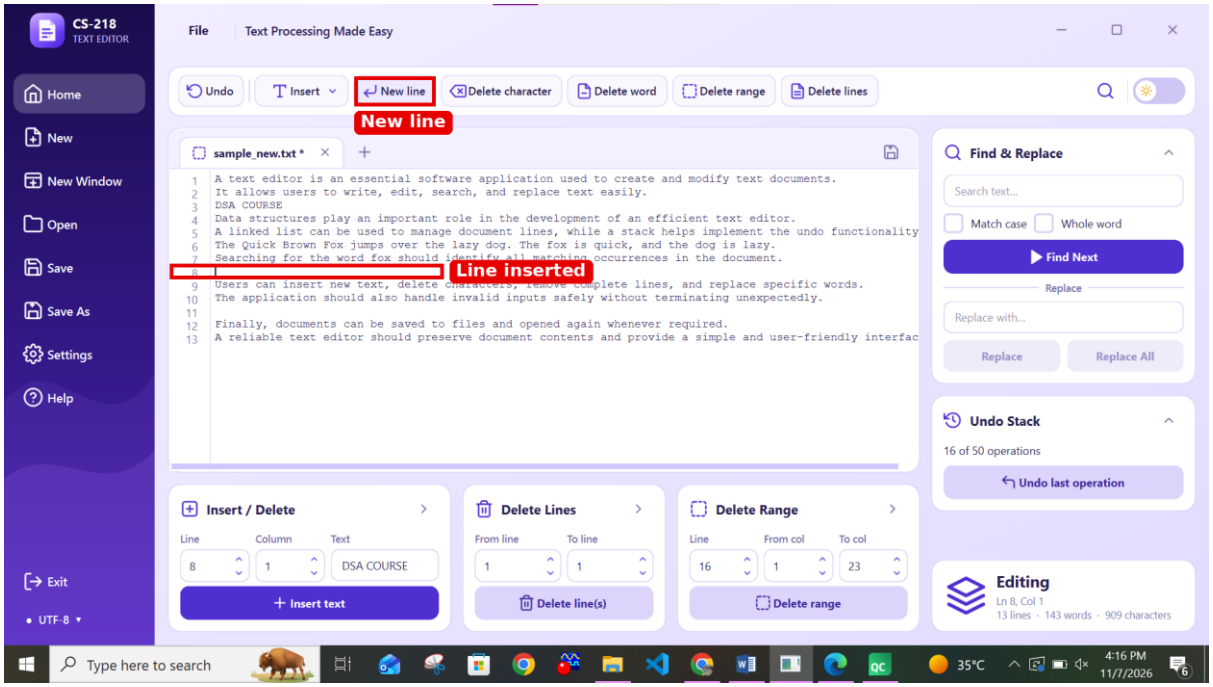



*Figure 9.12: Text Deletion Functionality*

Figure 9.12 demonstrates the deletion functionality. Users can remove unwanted characters, words, or selected ranges of text from the document.

## 9.4.3.3 Insert a New Line



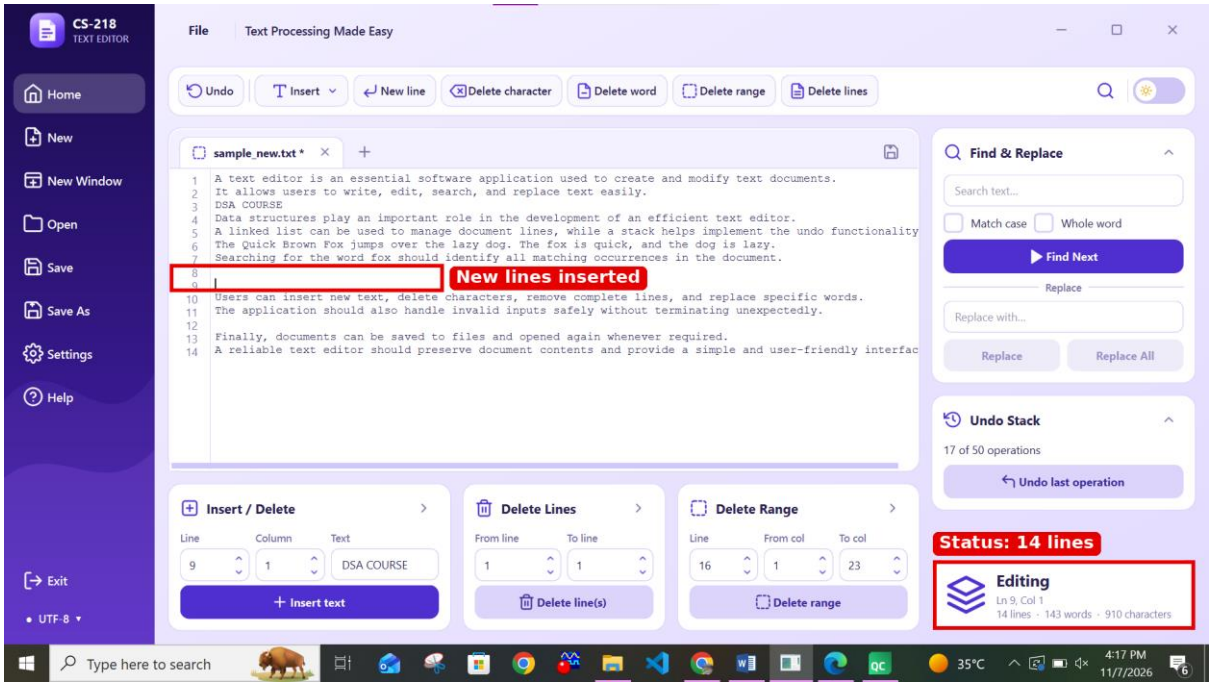
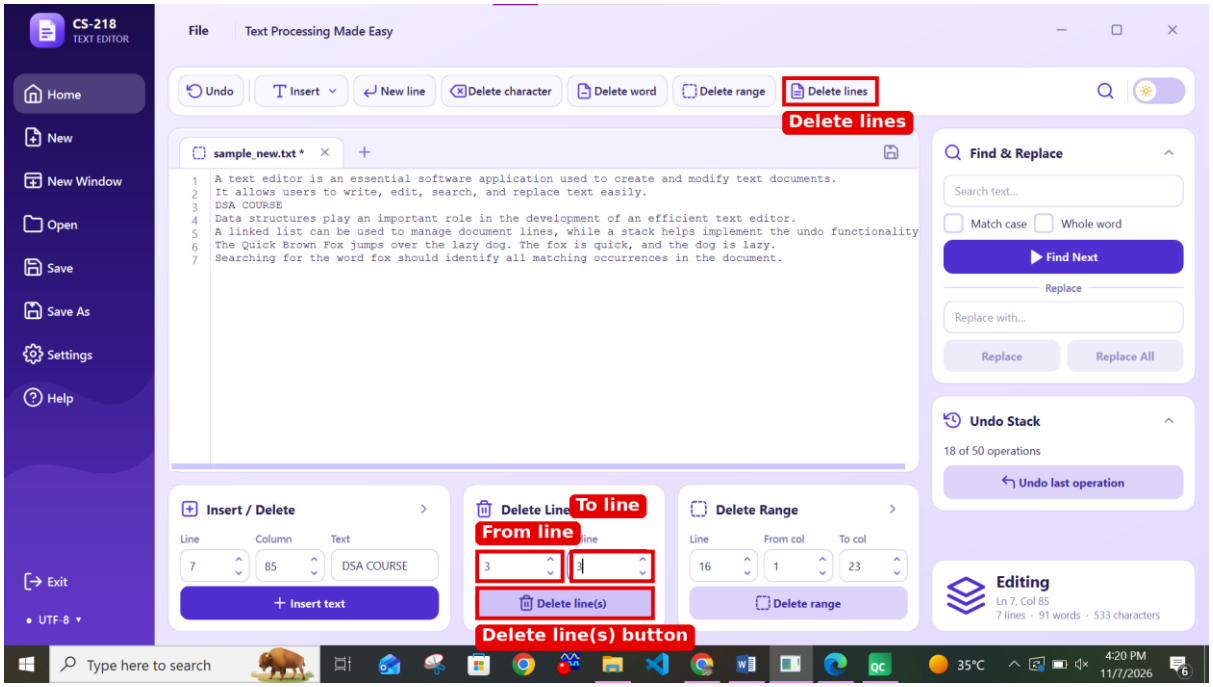
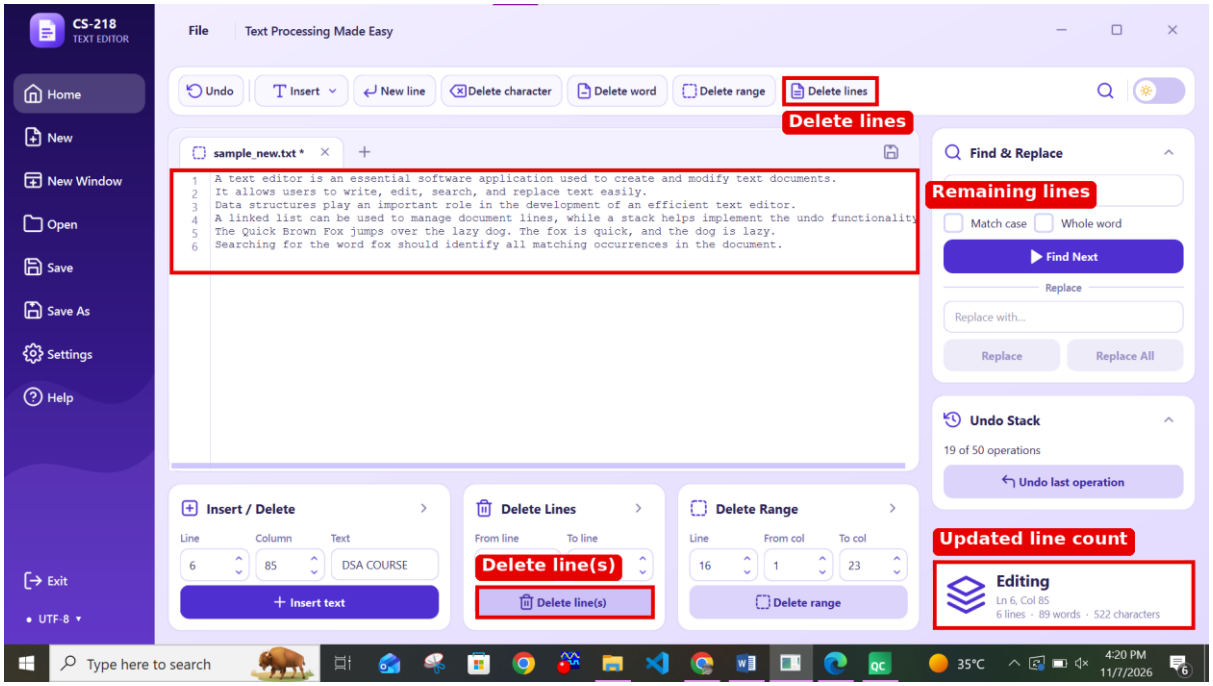


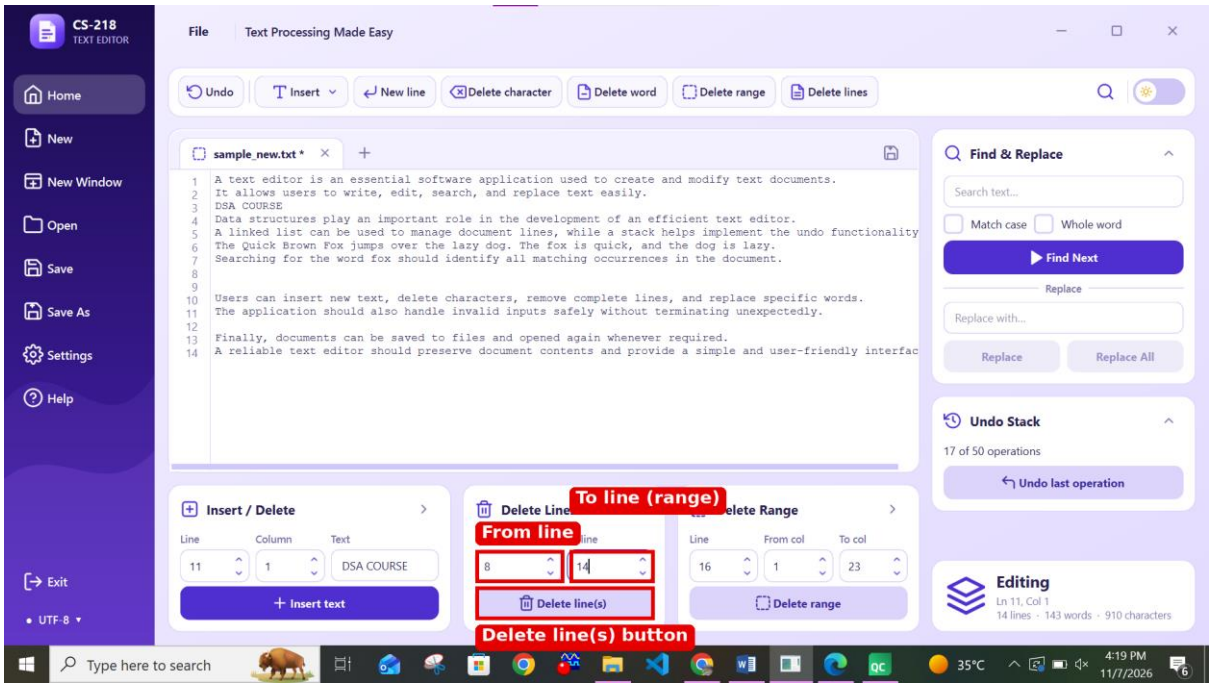

*Figure 9.13: Inserting a New Line*

Figure 9.13 demonstrates how users can create additional lines in the document. The editor updates the document structure to display the newly inserted line.

### 9.4.3.4 Delete a Line or Range of Lines




CS-218
TEXT EDITOR
File
Text Processing Made Easy
Home
New
New Window
Open
Save
Save As
Settings
Help
Exit
UTF-8
Undo
Insert
New line
Delete character
Delete word
Delete range
Delete lines
Delete lines
sample_new.txt *
1 A text editor is an essential software application used to create and modify text documents.
2 It allows users to write, edit, search, and replace text easily.
3 Data structures play an important role in the development of an efficient text editor.
4 A linked list can be used to manage document lines, while a stack helps implement the undo functionality
5 The Quick Brown Fox jumps over the lazy dog. The fox is quick, and the dog is lazy.
6 Searching for the word fox should identify all matching occurrences in the document.
Find & Replace
Remaining lines
Match case
Whole word
Find Next
Replace
Replace with...
Replace
Replace All
Undo Stack
19 of 50 operations
Undo last operation
Insert / Delete
Line
Column
Text
6
85
DSA COURSE
Insert text
Delete Lines
From line
To line
Delete line(s)
Delete Range
Line
From col
To col
16
1
23
Delete range
Updated line count
Editing
Ln 6, Col 85
6 lines · 89 words · 522 characters
Type here to search
35°C
4:20 PM
11/7/2026


CS-218
TEXT EDITOR
File
Text Processing Made Easy
Home
New
New Window
Open
Save
Save As
Settings
Help
Exit
UTF-8
Undo
Insert
New line
Delete character
Delete word
Delete range
Delete lines
sample_new.txt *
1 A text editor is an essential software application used to create and modify text documents.
2 It allows users to write, edit, search, and replace text easily.
3 DSA COURSE
4 Data structures play an important role in the development of an efficient text editor.
5 A linked list can be used to manage document lines, while a stack helps implement the undo functionality
6 The Quick Brown Fox jumps over the lazy dog. The fox is quick, and the dog is lazy.
7 Searching for the word fox should identify all matching occurrences in the document.
8
9
10 Users can insert new text, delete characters, remove complete lines, and replace specific words.
11 The application should also handle invalid inputs safely without terminating unexpectedly.
12
13 Finally, documents can be saved to files and opened again whenever required.
14 A reliable text editor should preserve document contents and provide a simple and user-friendly interfac
Find & Replace
Search text...
Match case
Whole word
Find Next
Replace
Replace with...
Replace
Replace All
Undo Stack
17 of 50 operations
Undo last operation
To line (range)
From line
Insert / Delete
Line
Column
Text
11
1
DSA COURSE
Insert text
Delete Lines
8
14
Delete line(s)
Delete Range
Line
From col
To col
16
1
23
Delete range
Delete line(s) button
Editing
Ln 11, Col 1
14 lines · 143 words · 910 characters
Type here to search
35°C
4:19 PM
11/7/2026

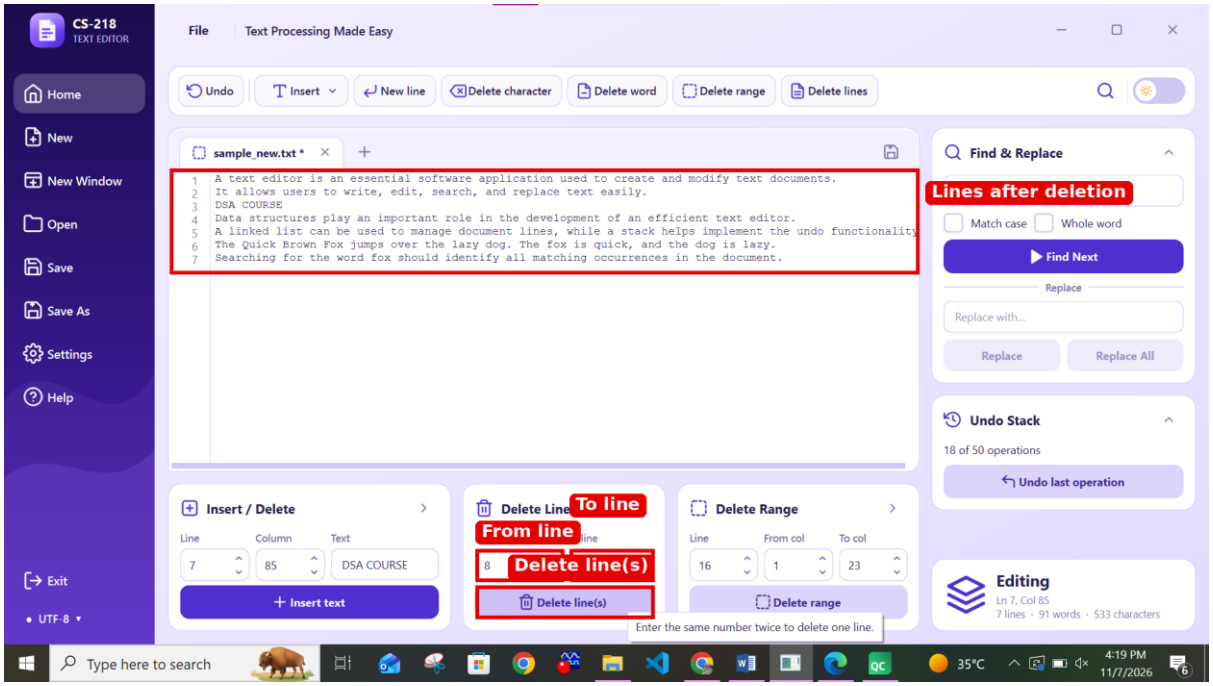
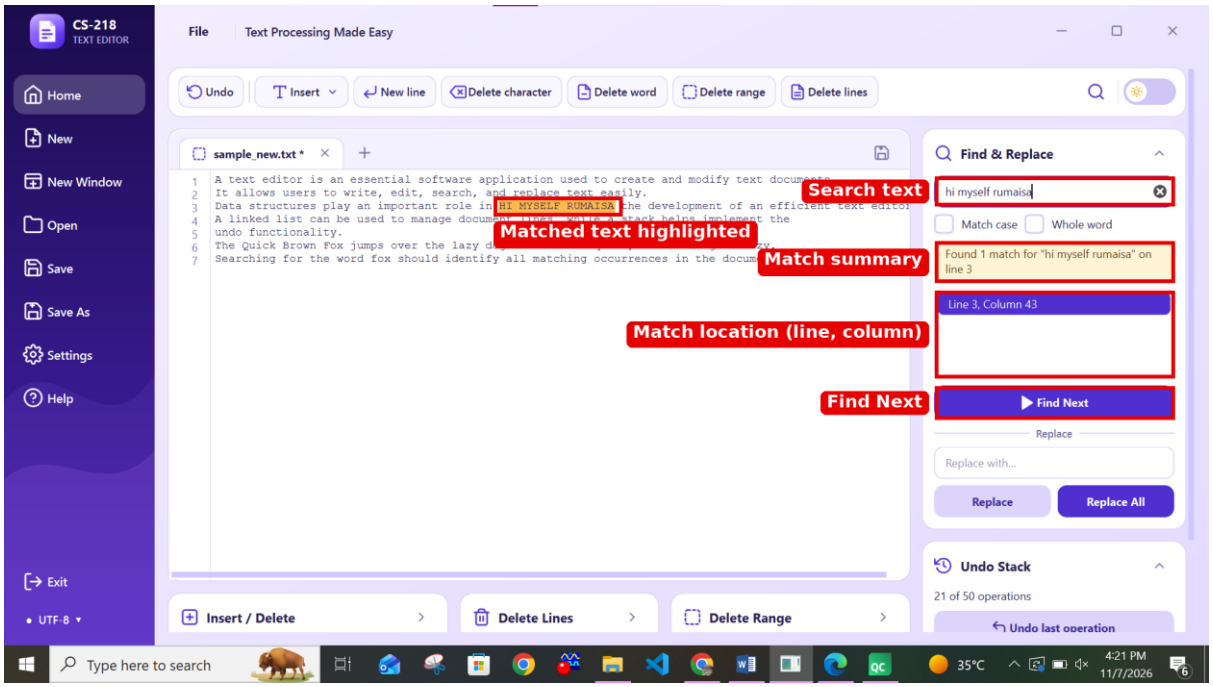



*Figure 9.14: Deleting Lines from the Document*

Figure 9.14 demonstrates the removal of a line or a range of lines from the document. The remaining lines are maintained in their correct order.

## 9.4.4 Search Functionality

### 9.4.4.1 Search for a Word or Text Pattern



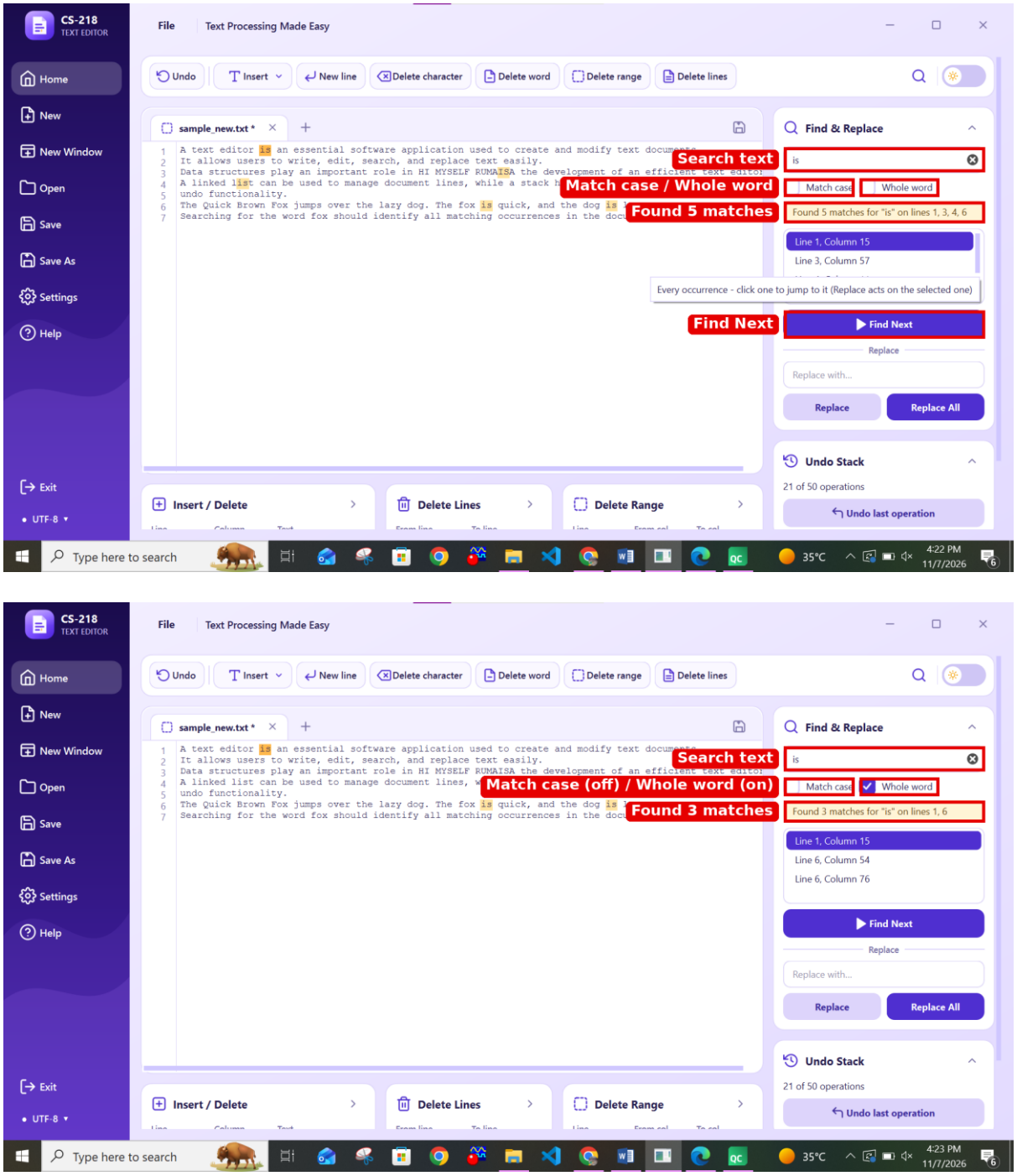



*Figure 9.15: Searching for Text in the Document*

Figure 9.15 demonstrates the search functionality of the graphical interface. Users can enter a word or sequence of characters to search for matching occurrences within the document.

### 9.4.4.2 Display Search Results and Match Locations

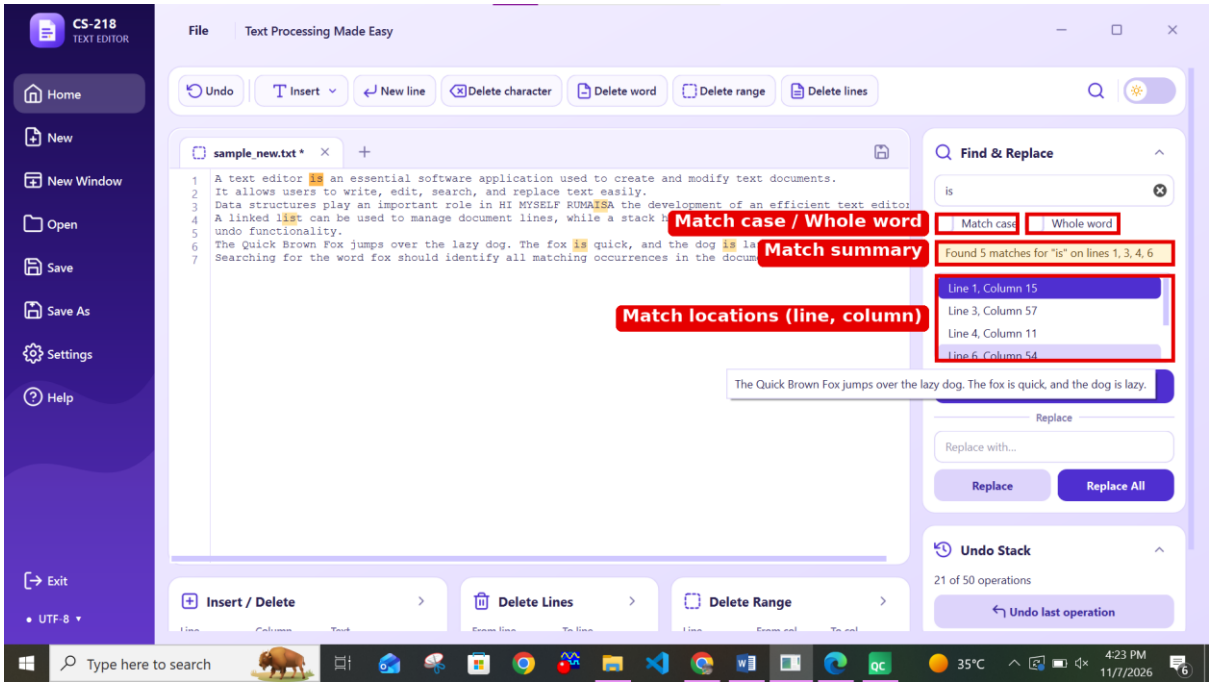



*Figure 9.16: Displaying Search Results and Match Locations*

Figure 9.16 demonstrates how the application displays the locations of matching search results. Each result identifies the corresponding line and column number, allowing users to locate matching text easily.

The search panel also provides case-sensitive and whole-word search options. Selecting a search result moves the editor cursor to the corresponding match.

## 9.4.5 Replace Functionality

### 9.4.5.1 Replace a Specific Occurrence

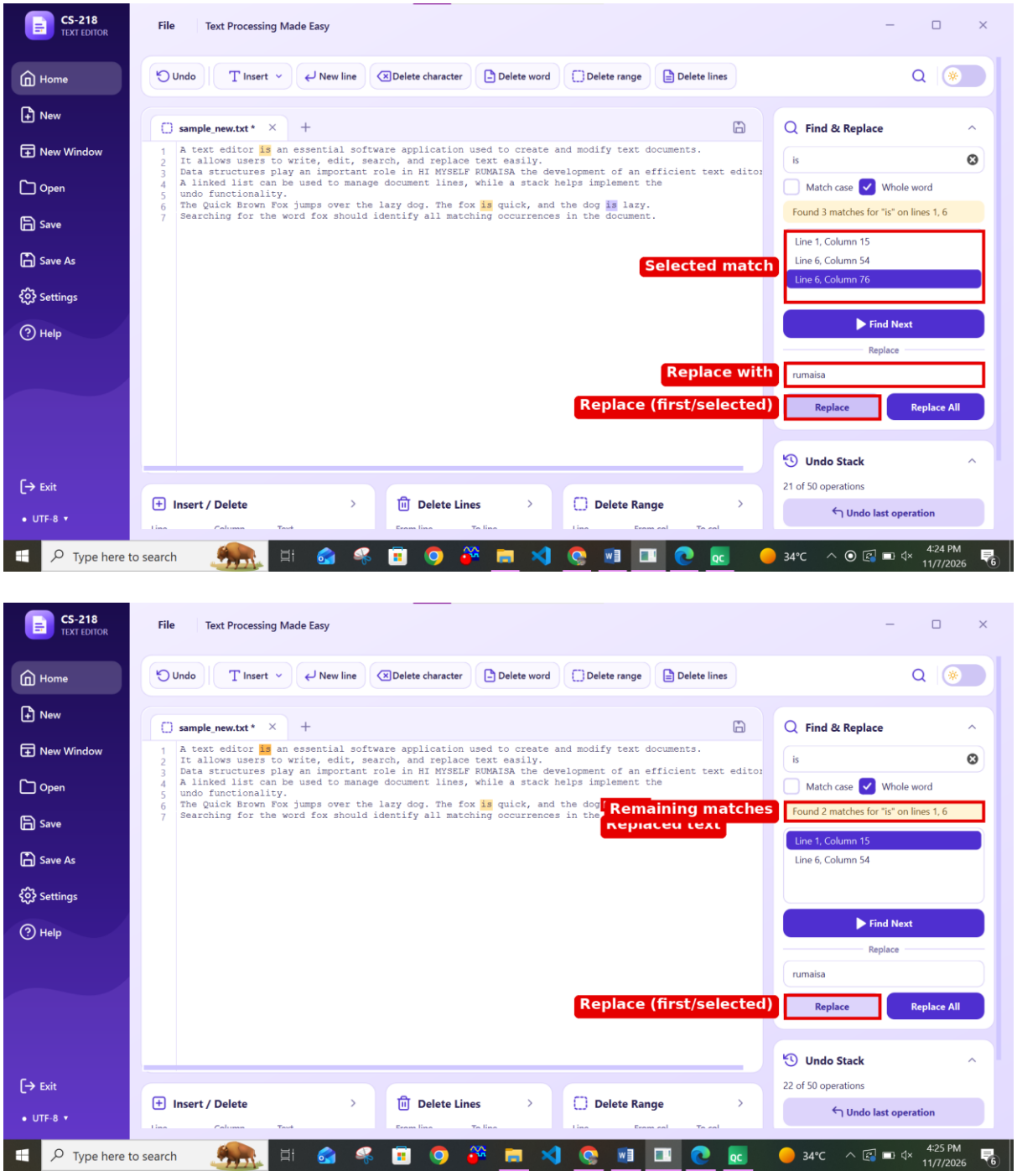



*Figure 9.17: Replacing a Specific Occurrence*

Figure 9.17 demonstrates the Replace First functionality. It allows users to replace a selected or first matching occurrence of a specified word or text pattern with replacement text.

### 9.4.5.2 Replace All Occurrences

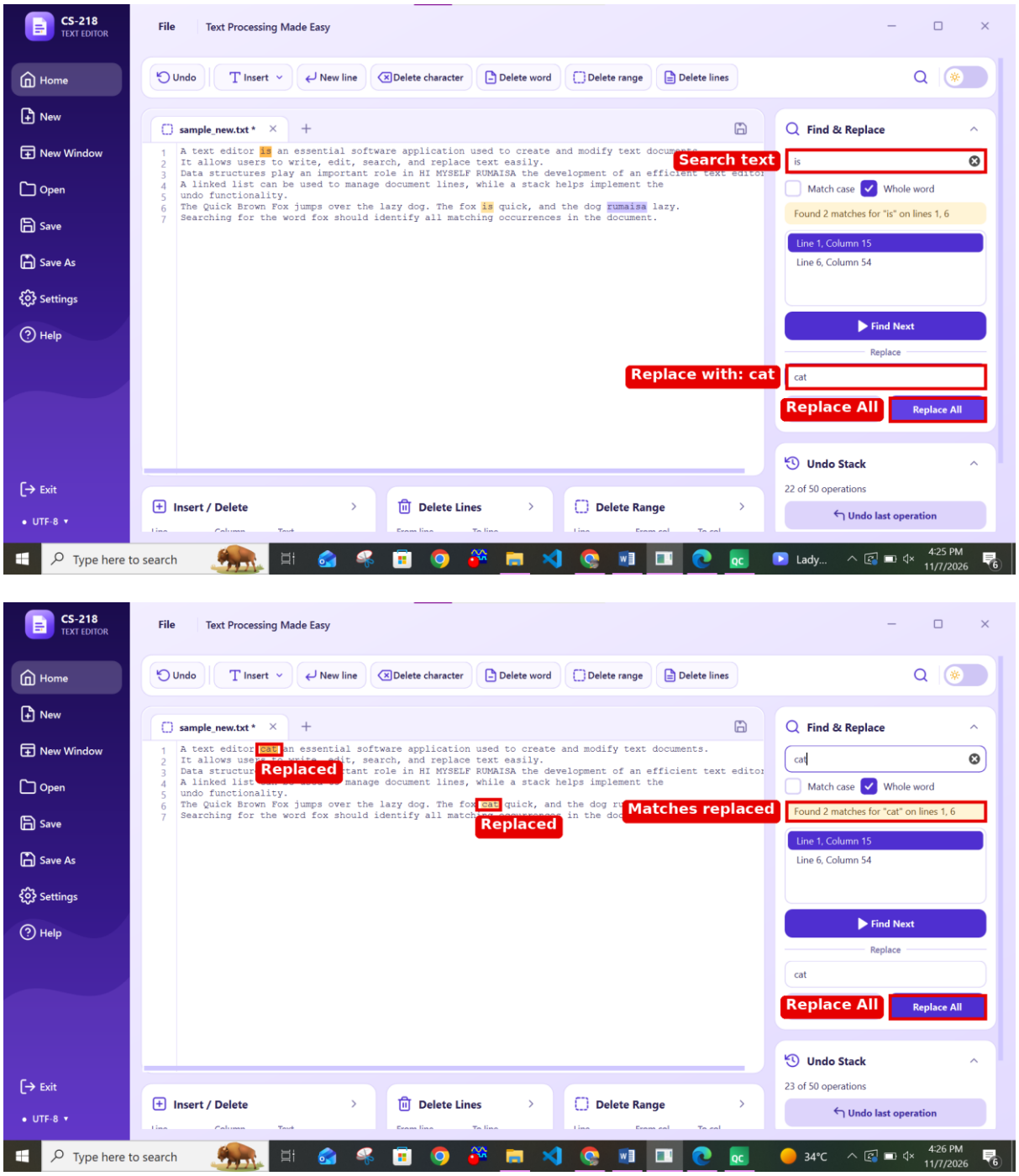
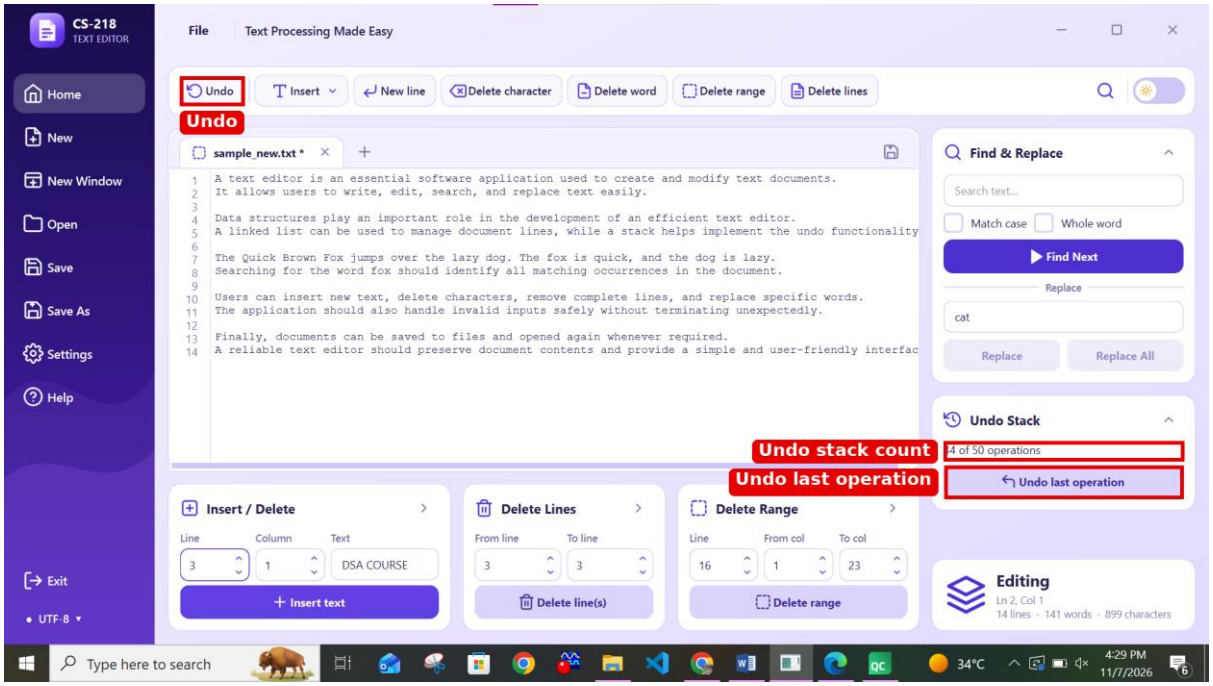


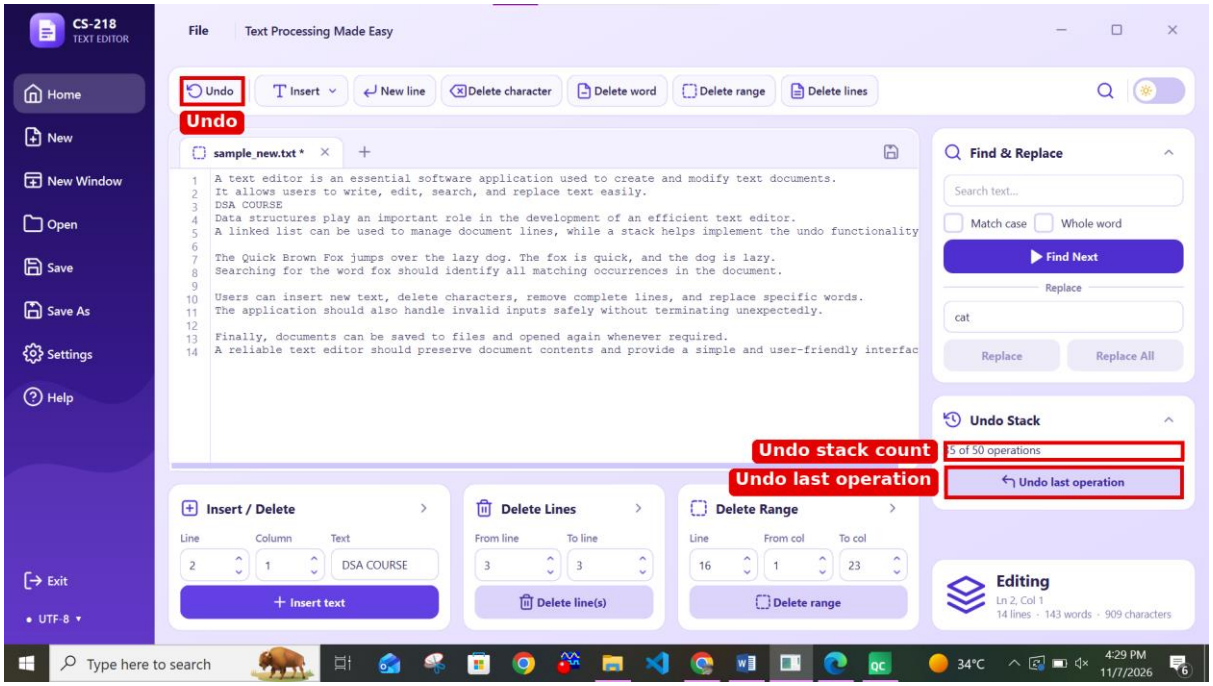

*Figure 9.18: Replacing All Occurrences*

Figure 9.18 demonstrates the Replace All functionality. The application searches for all matching occurrences and replaces them with the specified replacement text. The updated document reflects the completed replacement operation.

## 9.4.6 Undo Functionality

### 9.4.6.1 Undo Recent Editing Operations



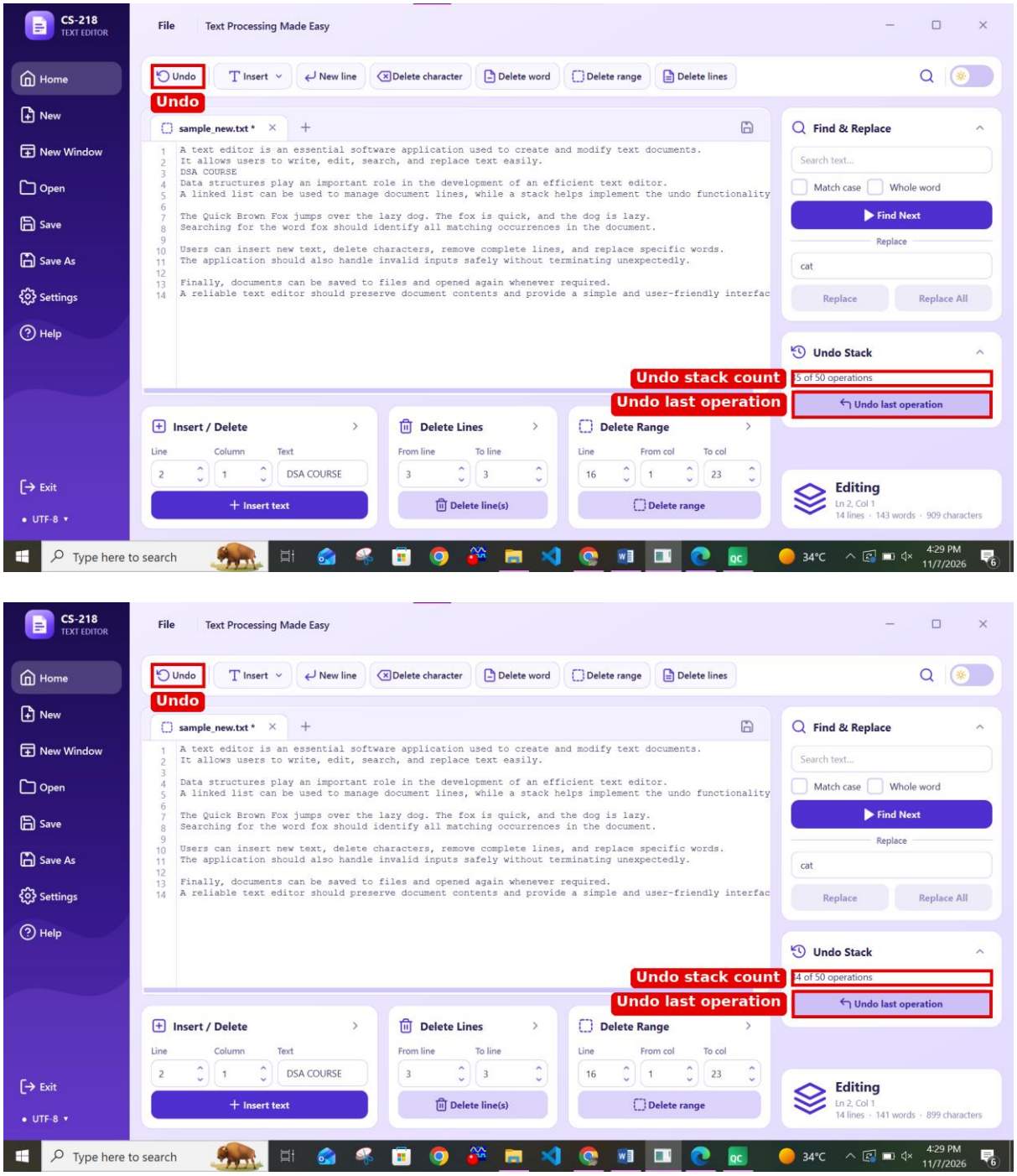



*Figure 9.19: Undo Functionality*

Figure 9.19 demonstrates the Undo functionality provided by the graphical interface. Users can reverse recent editing operations, including insertion, deletion, and replacement.

The undo mechanism uses the stack-based approach implemented in the backend. This allows previous document states to be restored according to the available undo history.

## 9.4.7 Basic Error Handling

### 9.4.7.1 Invalid Search Input

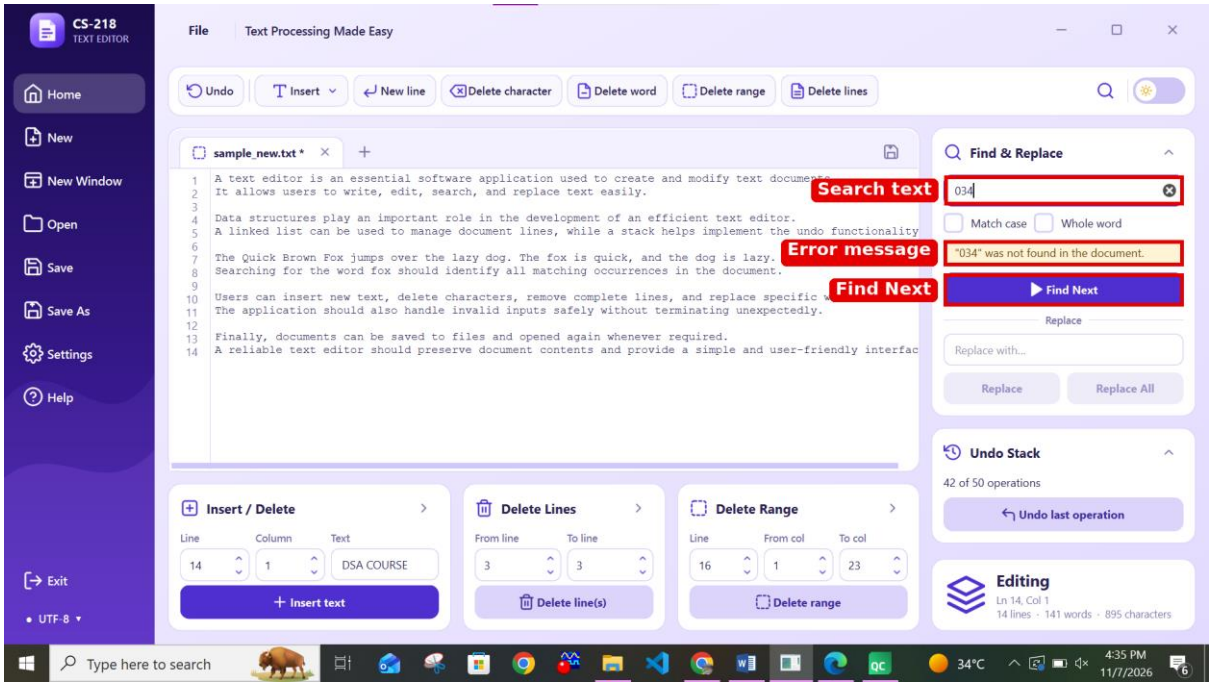



*Figure 9.20: Handling an Empty Search Pattern*

Figure 9.20 demonstrates the application's response when a user attempts to search using an empty string. The application rejects the invalid request and displays an appropriate error message instead of terminating unexpectedly.

### 9.4.7.2 File-Opening Error

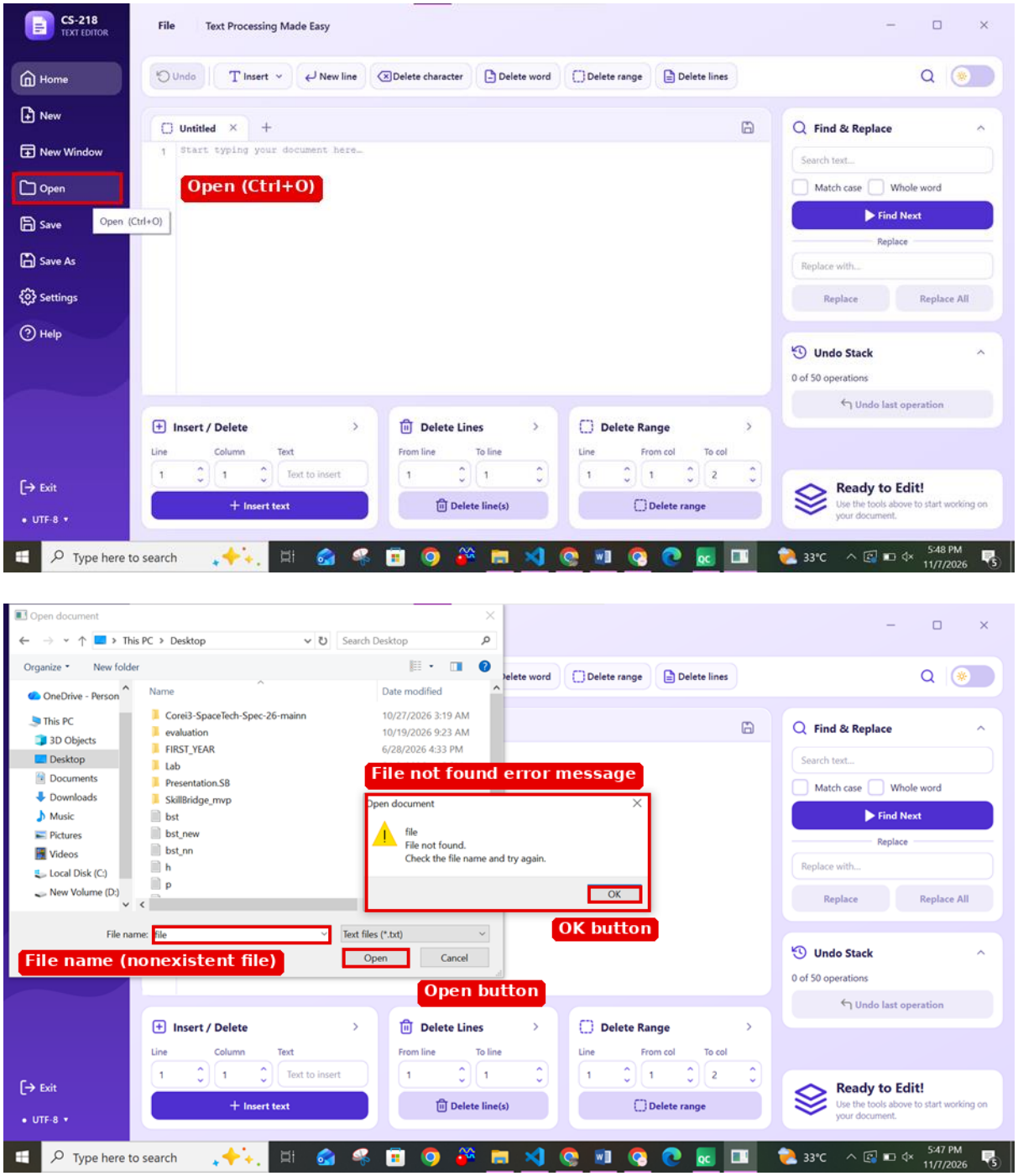



*Figure 9.21: Handling File-Opening Errors*

Figure 9.21 demonstrates how the application handles an unsuccessful file-opening operation. When a requested file cannot be opened, an appropriate error message is displayed and the application remains operational.

### 9.4.7.3 Invalid Editing Operations

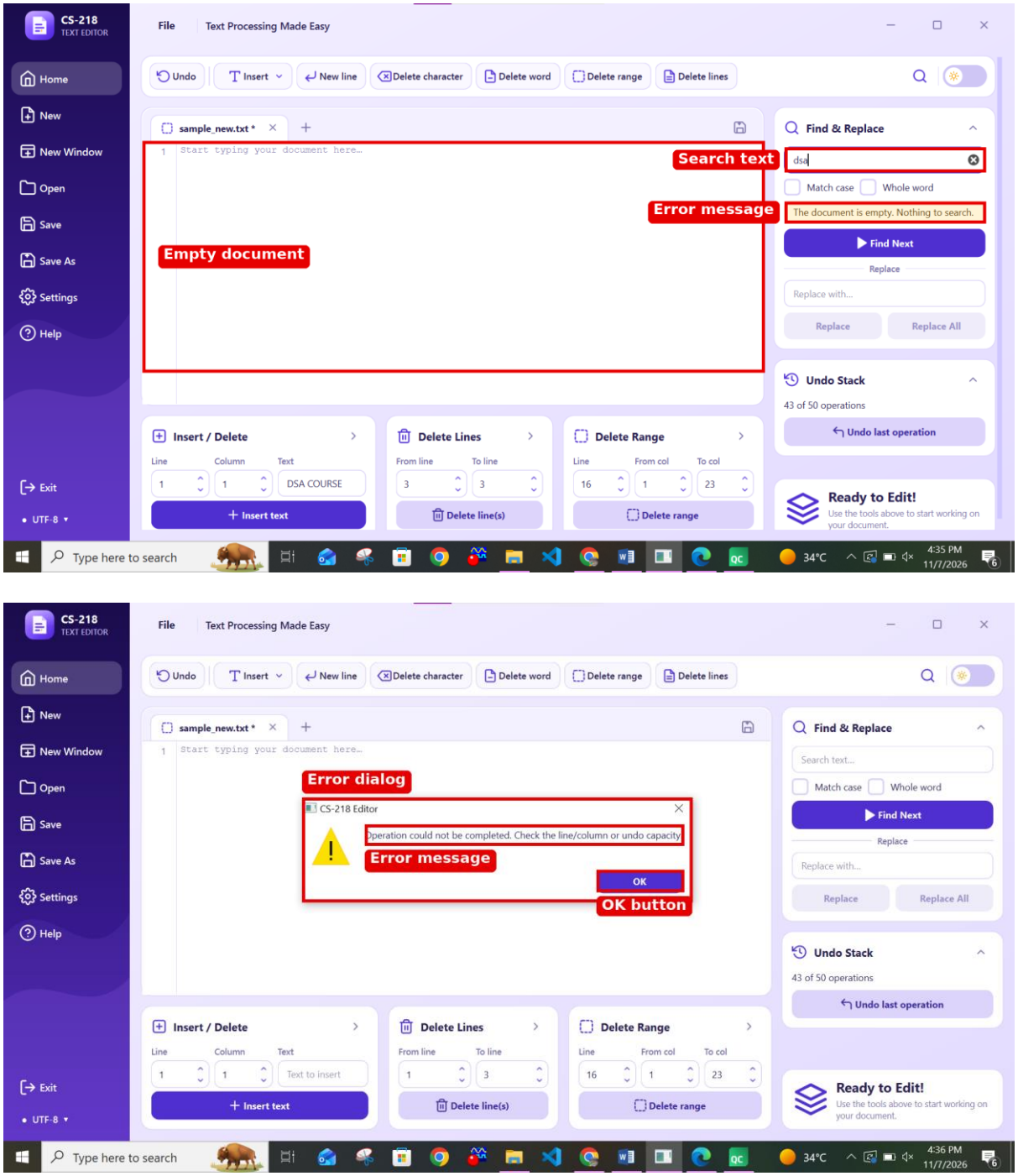



*Figure 9.22: Handling Invalid Editing Operations*

Figure 9.22 demonstrates the application's handling of invalid editing requests, such as an invalid position or an operation performed on an empty document. Invalid requests are rejected safely without causing unexpected application termination.

## 9.5 Summary of Frontend Functionality

The Qt graphical user interface provides the essential functionality required for a text editor.

The implemented file operations allow users to create new documents, open existing files, save documents, save documents under new filenames, and exit the application.

The text display features allow users to view complete document contents, examine individual lines, and access useful document information.

The editing features support text insertion, character and word deletion, line insertion, and line deletion. The search and replacement facilities allow users to locate matching text, view its locations, replace individual occurrences, and replace all occurrences.

The Undo functionality allows users to reverse recent editing operations using the backend's stack-based undo mechanism. Basic error handling also helps prevent invalid requests from interrupting normal application execution.

Overall, the Qt frontend provides an interactive way to access the text editor's essential features while sharing the same underlying backend implementation as the console application.

## 9.6 Chapter Summary

This chapter presented the automated backend test results and the essential features of the Qt graphical user interface.

The backend testing screenshots confirmed that all 55 automated test cases passed successfully, with zero failures.

The GUI screenshots demonstrate the application's file-management, text-display, editing, search, replacement, undo, and error-handling functionality.

Together, these results provide visual evidence of the backend testing process and the frontend features implemented in the text editor.

# 10. LIMITATIONS OF THE IMPLEMENTATION

## 10.1 Interface Limitations

The console interface is text-based and handles one command at a time. The main screen displays only a preview of the first five lines. Users must select a viewing option to see the remaining contents, and option 05 displays the entire document without paging. Column positions for insertion and deletion use zero-based offsets, which requires users to count characters manually. The console interface does not provide a graphical cursor, mouse interaction, or direct keyboard shortcuts for editing.

The Qt graphical interface provides a more convenient editing experience through a graphical text area, toolbar actions, and search controls. However, it requires a compatible Qt installation and currently supports one open document at a time. Both interfaces use the same backend components, but their user interactions and available controls differ.

## 10.2 Document Representation Limitations

Each document line is stored as a separate node in a doubly linked list, with an individual dynamically allocated character array. Consequently, each line requires approximately 40 bytes of node overhead, in addition to at least 16 bytes of buffer and allocator overhead.

Accessing a line by its number requires traversal from the nearer end of the linked list. Therefore, repeatedly accessing lines by index can result in quadratic time complexity when processing an entire document. This limitation affects operations such as searching, Replace All, and displaying all lines.

Long lines also increase editing costs because insertion and deletion may require copying or shifting a substantial portion of the character buffer.

Lines are stored without their line-ending characters. A document ending with a newline is represented using an additional empty final line. Line endings are normalised when files are loaded. A lone CR is converted to LF, and if a file contains CRLF line endings, the application uses CRLF when saving the document. Therefore, files containing mixed line-ending styles are not preserved byte for byte.

The implementation supports UTF-8 text but does not support other character encodings or binary files.

## 10.3 Editing Limitations

Text passed to the insertion functions cannot contain line breaks. New lines must be created using the Insert Line operation.

The console interface does not provide dedicated operations for splitting a line, joining two lines, deleting text across line boundaries, or performing cut, copy, and paste operations.

The Delete Word operation removes the specified word together with the whitespace immediately preceding it. Character editing operations are restricted to individual lines.

The application also does not support rich-text formatting, such as font styles, colours, text sizes, or embedded images.

## 10.4 Search and Replacement Limitations

Searching is performed separately on each document line. Consequently, a search pattern cannot match text extending across two lines.

The search implementation uses the Knuth-Morris-Pratt (KMP) algorithm and supports case-sensitive, case-insensitive, and whole-word matching. However, regular-expression searching is not supported.

Case-insensitive matching performs ASCII-based case folding only. Therefore, non-ASCII uppercase and lowercase characters, such as É and é, are not treated as equivalent. Whole-word matching also treats non-ASCII bytes as word characters.

Replacement text cannot contain line breaks. Furthermore, although the backend and Qt graphical interface support case-insensitive and whole-word search options, the console menu performs searching and replacement using case-sensitive, non-whole-word matching.

## 10.5 Undo Limitations

The undo history has a fixed capacity. The console implementation stores up to 100 operations, whereas the Qt graphical interface stores up to 50 operations. When the history becomes full, the oldest records are discarded to make room for newer operations.

The undo history is cleared when a new document is created or an existing document is opened. File operations themselves cannot be undone.

The modified flag is set after an undo operation, even when the document contents become identical to their previously saved state.

Each undo record stores copies of the affected text. Therefore, operations involving large deletions or multiple lines can require additional memory.

Consecutive typing operations are merged into a single undo record. As a result, one Undo operation may remove up to 64 bytes of consecutively typed text.

## 10.6 Performance Limitations

The linked-list implementation makes accessing a particular line dependent on its position in the document. Operations that repeatedly retrieve lines by index, including searching, Replace All, and displaying all lines, can therefore have quadratic traversal costs.

The performance measurements presented in Table 7.3 demonstrate this limitation. For example, searching through 50,000 lines required approximately 1.8 seconds.

Character insertion can also be expensive because the character buffer may need to be reallocated and its existing contents copied.

The console interface redraws document information by processing the document contents, including copying text to calculate character statistics.

Opening a file may temporarily require approximately three times the file size in memory for a short period, depending on the file contents and loading process.

These limitations become more noticeable when working with very large documents.

## 10.7 Future Improvements

Several improvements could make the application more efficient, flexible, and user-friendly:

- Replace repeated indexed linked-list access with sequential traversal or caching of the last visited node to reduce the quadratic cost of searching, Replace All, and document display.
- Implement geometric growth of character buffers to improve the average performance of text insertion.
- Support searching across line boundaries and introduce regular-expression searching.
- Provide case-insensitive and whole-word search options in the console interface.
- Add cut, copy, and paste operations to the console version.
- Improve modified-state tracking so that the document is marked as saved when its contents return to the saved state.
- Support undo for file operations.
- Introduce more advanced document representations, such as gap buffers, piece tables, or ropes, for handling very large documents.
- Implement Unicode-aware case folding and character counting.
- Allow users to configure the undo history capacity.
- Support multiple open documents through tabs or separate editor windows.
- Improve the Qt graphical interface with additional editing shortcuts and document-management features.

These improvements could enhance the application's performance and usability while maintaining its existing data structures and backend architecture where appropriate.

# 11. CONCLUSION

This project involved the design and implementation of a text editor using C++17, with both a console interface and a Qt graphical user interface. Both interfaces share a common backend, allowing document management, editing, searching, replacement, file handling, and undo operations to use the same underlying implementation.

The application supports creating new documents, opening existing text files, saving documents, and saving documents under new filenames. It also provides document display and information features, including line and character statistics.

The editing functionality includes text insertion, character deletion, word deletion, line insertion, line deletion, and line-range deletion. The search functionality uses the Knuth-Morris-Pratt algorithm and supports case-sensitive, case-insensitive, and whole-word matching. The replacement functionality allows users to replace the first matching occurrence or all matching occurrences of a specified string.

The undo functionality is implemented using a bounded stack-based approach built with a doubly linked list. It allows users to reverse recent insertion, deletion, and replacement operations while maintaining a limited history of previous changes.

The implementation uses manually developed data structures rather than relying entirely on standard container classes. A doubly linked list manages the document lines, dynamic character arrays store individual line contents, and a bounded undo stack maintains editing history.

Testing was performed using an automated backend test driver (`tests.cpp`) and a scripted console session. The automated test driver executed 55 test cases covering document management, editing operations, searching, replacement, UTF-8 handling, undo stack behaviour, and input validation.

All 55 automated tests passed successfully, with zero failures. The console application was also tested using scripted input to exercise its menu options. In addition, the Qt graphical interface was demonstrated to verify its document-management and editing features.

The project also highlighted the importance of analysing the performance of selected data structures. Although the linked list provides convenient line insertion and deletion, repeated indexed access introduces quadratic traversal costs in certain operations. The performance measurements presented in Chapter 7 helped identify this limitation and suggest possible improvements.

Overall, the project demonstrates the practical application of fundamental data structures and algorithms in developing a functional text editor. It also demonstrates how a shared backend can support multiple user interfaces while maintaining consistent document-processing logic. The implementation fulfils the main functional requirements while providing opportunities for future improvements in performance, Unicode support, editing capabilities, and graphical usability.

# APPENDIX A: SOURCE CODE STRUCTURE

The final project is organised into separate source files to maintain modularity and make the application easier to understand, maintain, and extend.

The project contains a common backend, two frontend implementations (console and Qt graphical interface), a CMake build configuration, and an automated test driver.

## A.1 Project Directory Structure

The final project directory is organised as follows:

```
CS-218_TextEditor/
|
|-- CMakeLists.txt
|-- README.md
|-- .gitignore
|
|-- tests/
|   `-- tests.cpp
|
`-- CS-218_SourceCode/
    |
    |-- TextDocument.h
    |-- TextDocument.cpp
    |
    |-- UndoStack.h
    |-- UndoStack.cpp
    |
    |-- EditOperations.h
    |-- EditOperations.cpp
    |
    |-- TextSearch.h
    |-- TextSearch.cpp
    |
    |-- TextReplace.h
    |-- TextReplace.cpp
    |
    |-- FileManager.h
    |-- FileManager.cpp
    |
    |-- InputValidator.h
    |-- InputValidator.cpp
    |
    |-- DisplayManager.h
    |-- DisplayManager.cpp
    |
    |-- PathUtil.h
    |
    |-- main.cpp
    |
    `-- QtMain.cpp
```

## A.2 Description of Source Files

The following table describes the purpose and organisation of the main source files used in the project.

Table A.1: Source Files and Their Responsibilities

| Source file | Lines (.h / .cpp) | Purpose |
|---|---|---|
| `TextDocument.h / .cpp` | 48 / 196 | Implements the document using a doubly linked list of lines and dynamic character buffers. |
| `UndoStack.h / .cpp` | 69 / 129 | Defines EditType, UndoRecord, and the bounded undo history. |
| `EditOperations.h / .cpp` | 40 / 452 | Implements text insertion, deletion, line operations, replacement, and undo functionality. |
| `TextSearch.h / .cpp` | 53 / 163 | Implements KMP searching, case matching, and whole-word matching. |
| `TextReplace.h / .cpp` | 57 / 135 | Implements Replace First, replacement at a specified location, and Replace All. |
| `FileManager.h / .cpp` | 67 / 365 | Handles creating, opening, saving, Save As, and unsaved-change management. |
| `InputValidator.h / .cpp` | 30 / 241 | Validates user input, line numbers, ranges, and file names. |
| `DisplayManager.h / .cpp` | 36 / 83 | Handles console document display, line information, and document statistics. |
| `PathUtil.h` | 19 | Provides UTF-8 file-name conversion to `std::filesystem::path`. |
| `main.cpp` | 301 | Implements the console frontend and its numbered menu. |
| `QtMain.cpp` | 1,243 | Implements the Qt graphical frontend through the EditorWindow class. |
| `tests.cpp` | 89 | Contains the automated backend test driver used to execute the 55 test cases. |

Line counts are based on the project figures supplied for this report and should be checked against the final submitted source files.

## A.3 Build Configuration

The project uses CMake to configure and build the application. The `CMakeLists.txt` file is located in the root directory.

It defines separate executable targets for the Qt graphical application and the console application. Both executables use the shared backend source files.

The Qt graphical application uses the Qt Widgets module, while the console application is built independently without Qt dependencies.

The automated test driver is maintained separately in the `tests` directory and can be compiled independently for backend testing.

## A.4 Automated Testing File

The `tests.cpp` file contains the automated test driver used to verify the backend functionality.

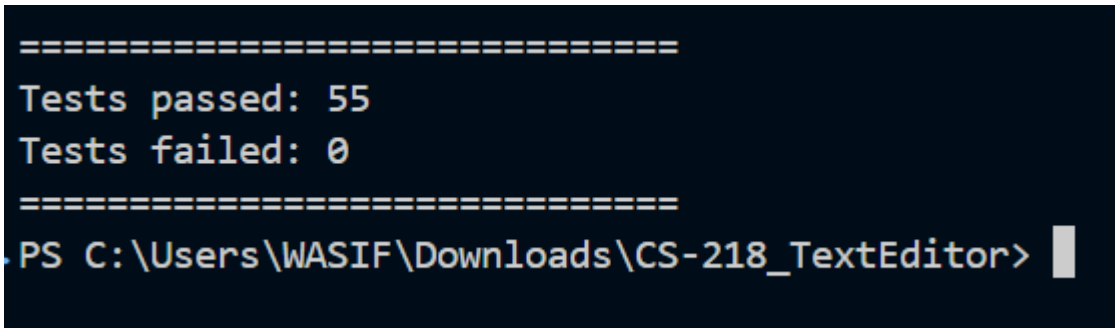
It executes 55 test cases covering the following areas:

- Document construction and text loading.
- Text insertion and deletion.
- Line insertion and deletion.
- Undo operations.
- Search and match identification.
- Replace First and Replace All.
- UTF-8 character handling.
- Undo stack capacity and record management.
- Input validation and invalid-operation handling.

The test driver reports the result of each test using PASS or FAIL messages and displays a final summary of successful and failed tests.

The final execution produced the following result:

### Automated Test Summary



# GROUP MEMBERS' CONTRIBUTION

## Individual Contributions

**Member 1: Rumaisa Waseem (CS-25052)**
**Text Insertion, Deletion, Undo, and Project Documentation**

- Implemented text insertion at specified positions.
- Worked on character, word, and range deletion.
- Implemented line insertion and deletion operations.
- Developed the stack-based Undo mechanism.
- Implemented undo functionality for insertion, deletion, and replacement operations.
- Prepared and organized the complete project report, including system design, implementation details, algorithms, complexity analysis, testing, results, limitations, and conclusion.

**Member 2: Yashfeen Raza (CS-25055)**
**Search, Replace, and Supporting Functionality**

- Implemented text searching using the KMP algorithm.
- Worked on identifying and displaying matching locations.
- Implemented Replace First functionality.
- Implemented Replace All functionality.
- Contributed to search and replacement options.
- Implemented case-sensitive and whole-word matching options for searching.

**Member 3: Manahil Imran (CS-25061)**
**File Operations, Text Display, and Supporting Functionality**

- Contributed to document creation and management.
- Worked on opening existing text files.
- Implemented and supported saving documents.
- Contributed to Save As functionality.
- Worked on displaying complete document contents.
- Contributed to line-by-line text display.
- Worked on displaying document statistics, including line and character counts.

## Collaborative Contributions

The following responsibilities were handled jointly by all three team members:

- **Error Handling:** Validating user input, rejecting invalid operations, handling empty search strings, and managing missing-file errors.
- **Main Program Integration:** Integrating individual modules into the main console application and ensuring that the components worked together correctly.
- **File Handling:** Coordinating document creation, opening, saving, Save As, and file-related error handling across the application.
- **Testing and Debugging:** Identifying errors, checking functionality, and verifying that the integrated application behaved as expected.